СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ 7

1.1. Описание предметной области 7

1.2. Составление технического задания 8

1.3. Структурная схема предприятия 9

1.4. UML-диаграмма вариантов использования 10

1.5. UML-диаграмма классов 12

1.6. ER-диаграмма базы данных 13

1.7. Правила ссылочной целостности 16

1.8. Диаграммы последовательности и состояний 17

2. РАЗРАБОТКА И ИНТЕГРАЦИЯ МОДУЛЕЙ СИСТЕМЫ 19

2.1. Настройка среды разработки и Git-репозитория 19

2.2. Создание базы данных и миграций 20

2.3. Модуль авторизации 22

2.4. Разграничение ролей 25

2.5. Модуль справочников 26

2.5.1. Справочник транспортных средств 27

2.5.2. Справочник неисправностей 28

2.6. Модуль заявок 30

2.7. Слой доступа к данным 33

2.8. Фильтрация, сортировка и пагинация 34

2.9. Обеспечение ссылочной целостности 36

3. МАТЕМАТИЧЕСКОЕ МОДЕЛИРОВАНИЕ И СТАТИСТИКА 38

3.1. Постановка задачи статистики 38

3.2. Реализация расчёта в коде 39
3.3. Интеграция расчёта в отчёт 41
4. ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА И ВНЕШНИЕ ИНТЕГРАЦИИ 43
4.1. Подключение логгера Serilog 43
4.2. Конфигурация приложения 44
4.3. Интеграция с внешним API курсов валют 45
5. ТЕСТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА 47
5.1. Модульные тесты 47
5.2. Интеграционные тесты 48
5.3. Тест-план 49
6. ОТЧЁТНОСТЬ И ВИЗУАЛИЗАЦИЯ ДАННЫХ 51
6.1. Отчёт «Заявки за период» 51
6.2. Экспорт отчёта в Excel и PDF 52
6.3. Графики статистики 54
7. ДОКУМЕНТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ 57
7.1. Руководство пользователя 57
7.2. Описание программы по ГОСТ 19.402-78 58
7.3. Оформление листингов 59
7.4. Сборка релиза 59
7.5. Инструкция по развёртыванию 60
ЗАКЛЮЧЕНИЕ 62
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 64
ПРИЛОЖЕНИЕ А 66
ПРИЛОЖЕНИЕ Б 70

ПРИЛОЖЕНИЕ В 75

ПРИЛОЖЕНИЕ Г 82

ПРИЛОЖЕНИЕ Д 84

ПРИЛОЖЕНИЕ Е 88

ВВЕДЕНИЕ

Современная станция технического обслуживания автомобилей ежедневно обрабатывает десятки обращений — от плановой замены масла до сложного ремонта двигателя. Каждая заявка проходит несколько стадий — приём, согласование, назначение механика, выполнение работ, расчёт стоимости и закрытие. Сведения о заявках нередко ведутся в бумажном журнале или в разрозненных файлах Excel.

Автоматизация учёта заявок устраняет эти проблемы — все обращения хранятся в единой базе данных, статус изменяется по строго определённым правилам, а по накопленным данным рассчитываются средние показатели, доверительные интервалы и вероятности завершения ремонта за заданное время. Именно поэтому тема учебной практики — разработка модуля учёта заявок на ремонт транспорта с базой данных и статусами, с расчётом среднего времени ремонта — является актуальной и имеет прямое практическое применение.

Цель работы — спроектировать и разработать информационную систему учёта заявок на ремонт транспорта на платформе C# WinForms с базой данных SQLite, разграничением ролей, жизненным циклом заявки и статистическим анализом длительности ремонта.

Для достижения цели поставлены следующие задачи

1. изучить предметную область деятельности станции технического обслуживания, описать участников, бизнес-процессы и требования к автоматизации;
2. составить техническое задание по ГОСТ 19.201-78, определить функциональные и нефункциональные требования;
3. построить структурную схему предприятия, UML-диаграммы вариантов использования и классов, ER-диаграмму и проверить схему базы данных на соответствие третьей нормальной форме;
4. настроить среду разработки, Git-репозиторий и решение из трёх проектов — библиотеки Core, приложения WinForms и проекта модульных тестов;

5. создать базу данных SQLite средствами Entity Framework Core, выполнить миграцию и заполнить базу демонстрационными данными;
6. реализовать авторизацию с хэшированием паролей, разграничение ролей и CRUD-формы справочников транспорта и неисправностей;
7. разработать модуль заявок с полным жизненным циклом статусов, историей изменений, фильтрацией, сортировкой и постраничным выводом;
8. реализовать расчёт описательной статистики времени ремонта, экспоненциальную модель распределения и доверительный интервал;
9. подключить журналирование Serilog, конфигурацию appsettings.json и внешний API курсов валют с обработкой ошибок;
10. покрыть ключевую логику модульными и интеграционными тестами, составить тест-план;
11. реализовать отчёт за период с экспортом в Excel и PDF и визуализацией статистики;
12. подготовить руководство пользователя, описание программы по ГОСТ 19.402-78, инструкцию по развёртыванию и презентацию к защите.

Объект исследования — процесс учёта заявок на ремонт транспортных средств в сервисной организации. Предмет исследования — методы, модели и программные средства автоматизации этого процесса, включая статистическую обработку длительностей ремонта.

При выполнении работы использовались методы системного и объектно-ориентированного анализа, UML-моделирования, проектирования реляционных баз данных и нормализации, математической статистики и теории вероятностей, модульного и интеграционного тестирования, а также инструменты контроля версий Git.

Практическая значимость работы состоит в том, что созданное приложение может применяться на небольшой станции технического обслуживания — оно не требует отдельного сервера, работает с локальной базой SQLite, поддерживает роли клиента, менеджера и администратора, формирует отчётность и экспортирует её в распространённые форматы.

Кроме того, реализованная математическая модель позволяет оценивать вероятность завершения ремонта за заданное время и обоснованно планировать загрузку постов.

1. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

1.1. Описание предметной области

Предметной областью является деятельность небольшой станции технического обслуживания автомобилей «АвтоМастер», которая выполняет текущий и капитальный ремонт легкового и лёгкого коммерческого транспорта. Организация располагает приёмным пунктом, производственным участком, складом запчастей, бухгалтерией и административной службой. Штат насчитывает шесть сотрудников — генеральный директор, менеджеры по приёму заявок (они же выполняют функции механиков), кладовщик, бухгалтер и администратор информационной системы.

Основной бизнес-процесс начинается с обращения клиента — владельца транспортного средства. Менеджер фиксирует данные автомобиля (марка, модель, государственный номер, VIN, год выпуска, пробег), вид неисправности, описание жалобы и желаемый срок выполнения. Затем заявке присваивается статус «Новая», и она попадает в очередь. После согласования объёма работ менеджер принимает заявку, назначает механика и переводит её в статус «В работе». По завершении ремонта фиксируются фактическая стоимость работ и запчастей, после чего заявка закрывается. Бухгалтерия использует накопленные данные для расчёта выручки, а руководство — для оценки загрузки постов и среднего времени ремонта.

В существующем («как есть») процессе сведения о заявках ведутся в бумажном журнале и частично дублируются в электронных таблицах. Это порождает ряд проблем

— заявки теряются, а история изменений не сохраняется — невозможно установить, кто и когда принял автомобиль в работу;

— статус заявки известен только менеджеру, клиент вынужден звонить для уточнения;

— данные об автомобилях и неисправностях вводятся повторно, что приводит к опечаткам и противоречиям;

— отсутствует единая нормативно-справочная информация о трудоёмкости и базовой стоимости работ;

— руководство не получает оперативных показателей — количества активных заявок, просроченных обращений, среднего времени ремонта и простоев;

— формирование отчёта за период занимает несколько часов и выполняется вручную.

Целевой процесс предполагает единую базу данных, в которой хранятся пользователи, транспортные средства, справочник неисправностей, заявки и история изменения их статусов. Каждая роль получает собственный набор операций — клиент видит только свои автомобили и заявки, менеджер обрабатывает все обращения и ведёт справочники, администратор дополнительно отвечает за работоспособность базы и резервное копирование. Отчёт формируется автоматически за выбранный период и выгружается в Excel или PDF.

Ключевые понятия предметной области. Заявка — обращение клиента, связанное с необходимостью ремонта конкретного автомобиля; статус — этап жизненного цикла заявки; неисправность — вид работ с нормативной трудоёмкостью и базовой ценой; простой — интервал от создания заявки до её завершения; время ремонта — интервал от начала работ до их выполнения. Именно время ремонта рассматривается как случайная величина, статистические характеристики которой рассчитываются в разделе 3.

1.2. Составление технического задания

Программное средство предназначено для автоматизации учёта заявок на ремонт транспорта, ведения справочников, контроля статусов и формирования отчётности на станции технического обслуживания. Система должна эксплуатироваться на персональном компьютере под управлением Windows 10/11, не требовать выделенного сервера и поддерживать работу одного пользователя в рамках локальной базы данных.

Техническое задание разработано в соответствии с ГОСТ 19.201-78 «Единая система программной документации. Техническое задание. Требования к содержанию и оформлению». Полный текст документа приведён в приложении А.

1.3. Структурная схема предприятия

Организационная структура станции технического обслуживания «АвтоМастер» является линейно-функциональной — генеральному директору подчиняются пять подразделений, каждое из которых отвечает за свой участок работ. Структурная схема приведена на рисунке 1.1.

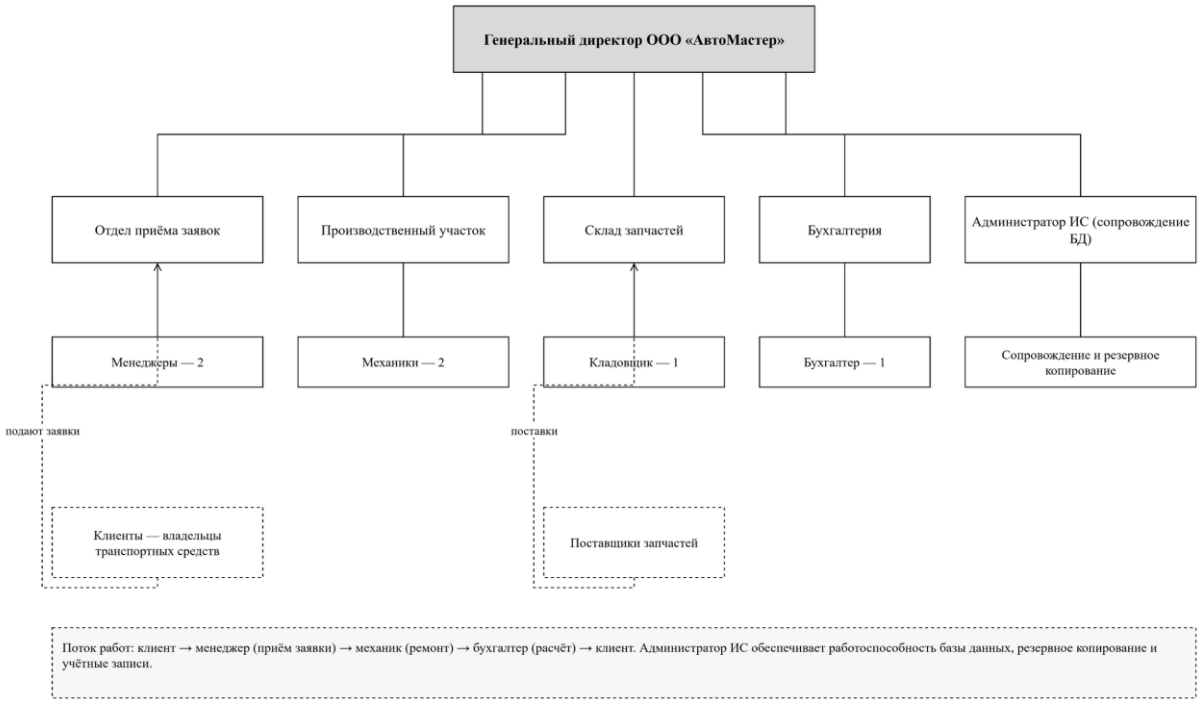


Рисунок 1.1 — Структурная схема предприятия

Отдел приёма заявок выполняет первичную коммуникацию с клиентом — принимает обращения, проверяет комплектность документов, согласует объём работ и сроки, контролирует выдачу автомобиля. Производственный участок отвечает непосредственно за ремонт — менеджеры совмещают функции механиков, распределяют работы по постам и фиксируют фактическое время. Склад запчастей обеспечивает наличие расходных материалов и запасных частей, взаимодействует с поставщиками. Бухгалтерия рассчитывает стоимость работ и запчастей, ведёт учёт выручки и формирует финансовую отчётность. Администратор информационной

системы сопровождает базу данных, управляет учётными записями, выполняет резервное копирование и обновление программного обеспечения.

Информационные потоки между подразделениями можно описать следующим образом — клиент подаёт заявку в отдел приёма; отдел передаёт согласованную заявку на производственный участок; участок запрашивает запчасти на складе и сообщает о завершении работ; бухгалтерия получает данные о стоимости и закрывает заявку; руководство анализирует сводные показатели. Разрабатываемая система автоматизирует эти потоки, заменяя бумажный документооборот единой базой данных и электронными статусами.

1.4. UML-диаграмма вариантов использования

Для определения границ системы и функций пользователей построена диаграмма вариантов использования (use case diagram). Акторами являются клиент, менеджер и администратор. Диаграмма представлена на рисунке 1.2.

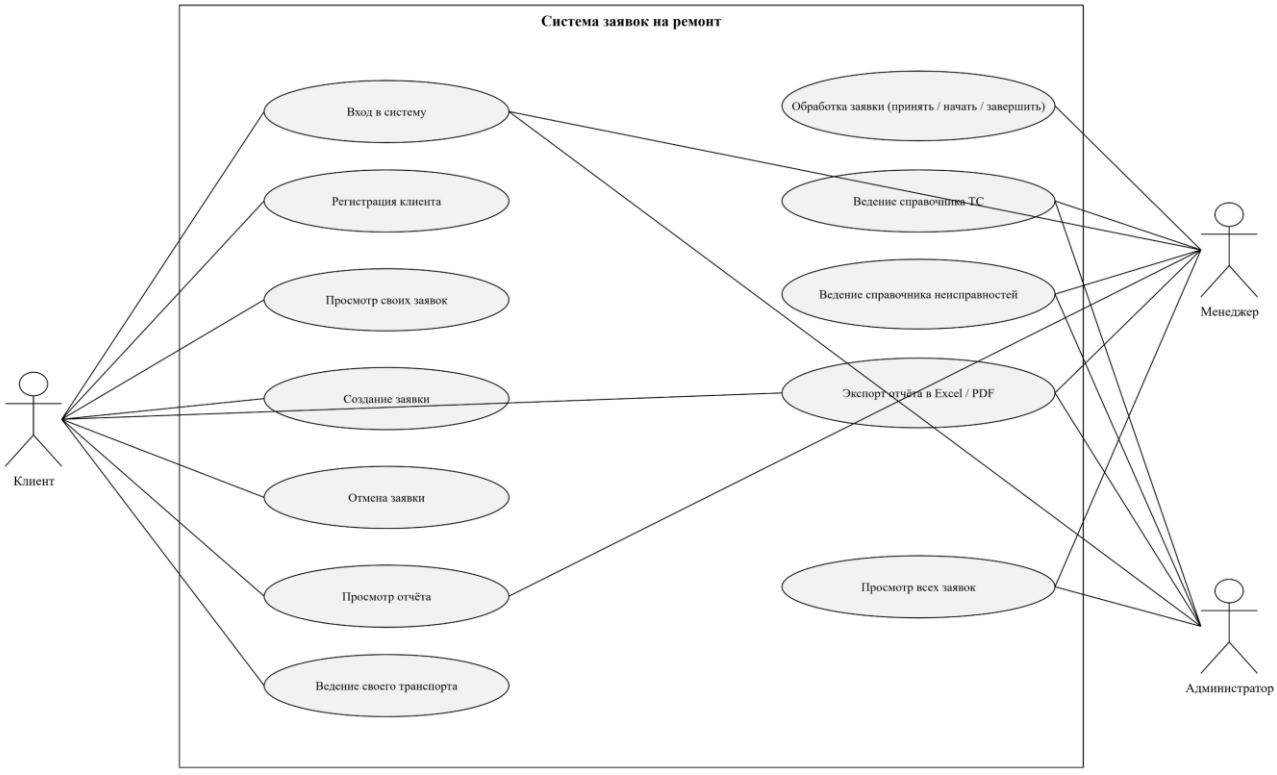


Рисунок 1.2 — UML-диаграмма вариантов использования

Клиент после входа получает доступ к своим заявкам и своему транспорту, может создать новую заявку, отменить её до начала работ, просмотреть отчёт по своим обращениям и выгрузить его в Excel или PDF.

Менеджер обрабатывает все заявки — принимает, начинает работы, завершает и закрывает их, назначает механика и стоимость, ведёт справочники транспорта и неисправностей, формирует общий отчёт. Администратор обладает правами менеджера и дополнительно отвечает за сопровождение базы данных; отдельного пользовательского интерфейса администрирования в текущей версии не предусмотрено, управление учётными записями выполняется на уровне базы данных.

В таблице 1.1 приведено соответствие акторов и вариантов использования, а также основные условия выполнения.

Таблица 1.1 — Акторы и варианты использования

Актор	Варианты использования	Условия
Клиент	Вход, просмотр своих заявок, создание заявки, отмена заявки, просмотр отчёта, экспорт, ведение своего транспорта	Видит только записи, где он указан владельцем или клиентом
Менеджер	Вход, обработка заявки, ведение справочника ТС, ведение справочника неисправностей, просмотр всех заявок, отчёт, экспорт	Полный доступ к операциям обработки заявок
Администратор	Вход, ведение справочников, просмотр всех заявок, отчёт, экспорт	Права менеджера плюс сопровождение БД

Рассмотрим основной сценарий «Создание заявки». Предусловие — клиент авторизован и имеет хотя бы один автомобиль. Основной поток — клиент открывает раздел «Мои заявки», нажимает «Добавить», выбирает автомобиль и неисправность, указывает приоритет, плановую дату и описание, сохраняет запись. Система проверяет данные, формирует уникальный номер вида ЗР-2026-0045, присваивает статус «Новая», создаёт первую запись истории и отображает заявку в списке. Постусловие — заявка сохранена в базе и доступна менеджеру. Альтернативный поток — при пустом описании или отсутствии автомобиля система выводит сообщение и не сохраняет запись.

Сценарий «Обработка заявки» выполняется менеджером. После открытия заявки система показывает только те действия, которые допустимы для текущего статуса. Например, для статуса «Новая» доступны кнопки «Принять заявку» и «Отменить заявку»; для статуса «Выполнена» — только

«Закрыть заявку». Попытка недопустимого перехода блокируется как на уровне интерфейса, так и в сервисе бизнес-логики.

1.5. UML-диаграмма классов

Статическая структура приложения отражена на диаграмме классов (рисунок 1.3). Центральной сущностью является класс RepairRequest, связанный с пользователем-клиентом, автомобилем, неисправностью, механиком и историей статусов.

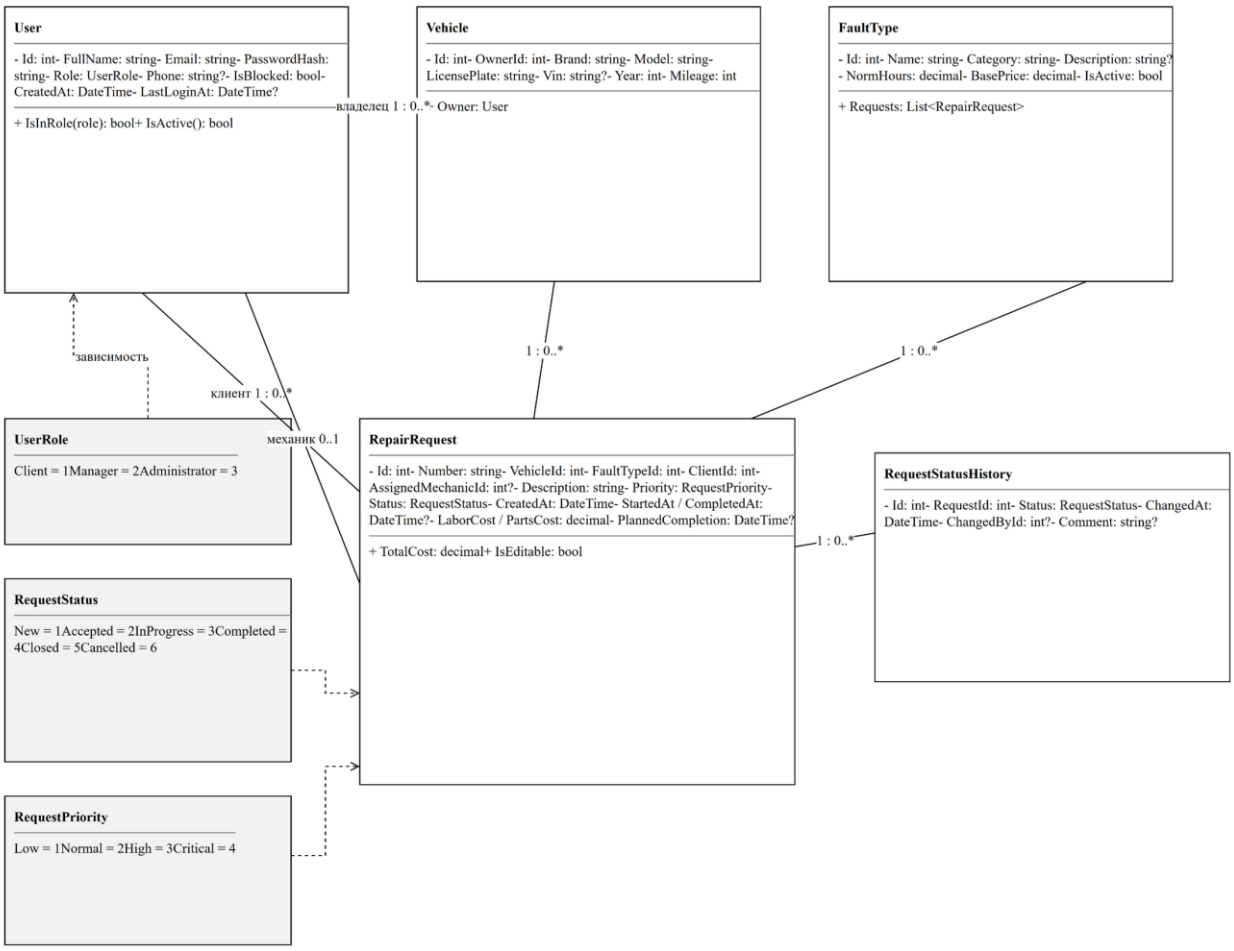


Рисунок 1.3 — UML-диаграмма классов

Класс User описывает учётную запись — идентификатор, ФИО, электронную почту, хэш пароля, роль, телефон, признак блокировки, даты регистрации и последнего входа. Класс Vehicle содержит сведения об автомобиле — владельца, марку, модель, государственный номер, VIN, год выпуска и пробег. Класс FaultType представляет вид неисправности с категорией, нормативной трудоёмкостью, базовой ценой и признаком активности.

Класс RepairRequest объединяет данные заявки — номер, ссылки на автомобиль, неисправность, клиента и механика, описание, приоритет, статус, временные отметки (создание, принятие, начало, завершение, закрытие), стоимости работ и запчастей, сведения о валюте и комментарий. Вычисляемые свойства TotalCost, RepairHours, WaitingHours, TotalDowntimeHours и IsOverdue не хранятся в базе, а рассчитываются на основе временных отметок и стоимостей. Класс RequestStatusHistory фиксирует каждое изменение статуса — кто, когда и с каким комментарием его выполнил.

Перечисления UserRole, RequestStatus и RequestPriority задают допустимые значения и обеспечивают типобезопасность. Связи между классами имеют кратности «один ко многим» — пользователь владеет несколькими автомобилями, автомобиль участвует во многих заявках, неисправность встречается в многих заявках, заявка содержит несколько записей истории. На уровне архитектуры классы разделены на модели предметной области, репозитории, сервисы и формы; такое разделение соответствует принципу единственной ответственности и упрощает тестирование.

Таблица 1.2 — Основные классы предметной области

Класс	Назначение	Ключевые элементы
User	Учётная запись	Id, FullName, Email, PasswordHash, Role, IsBlocked
Vehicle	Транспортное средство	Id, OwnerId, Brand, Model, LicensePlate, Vin, Year, Mileage
FaultType	Справочник неисправностей	Id, Name, Category, NormHours, BasePrice, IsActive
RepairRequest	Заявка на ремонт	Number, VehicleId, FaultTypeId, ClientId, Status, Priority, стоимости, даты
RequestStatusHistory	История статусов	RequestId, Status, ChangedAt, ChangedById, Comment

1.6. ER-диаграмма базы данных

Логическая модель базы данных построена в нотации «сущность — связь» (ER-диаграмма) и представлена на рисунке 1.4. Модель включает пять таблиц — Users, Vehicles, FaultTypes, RepairRequests и RequestStatusHistory.

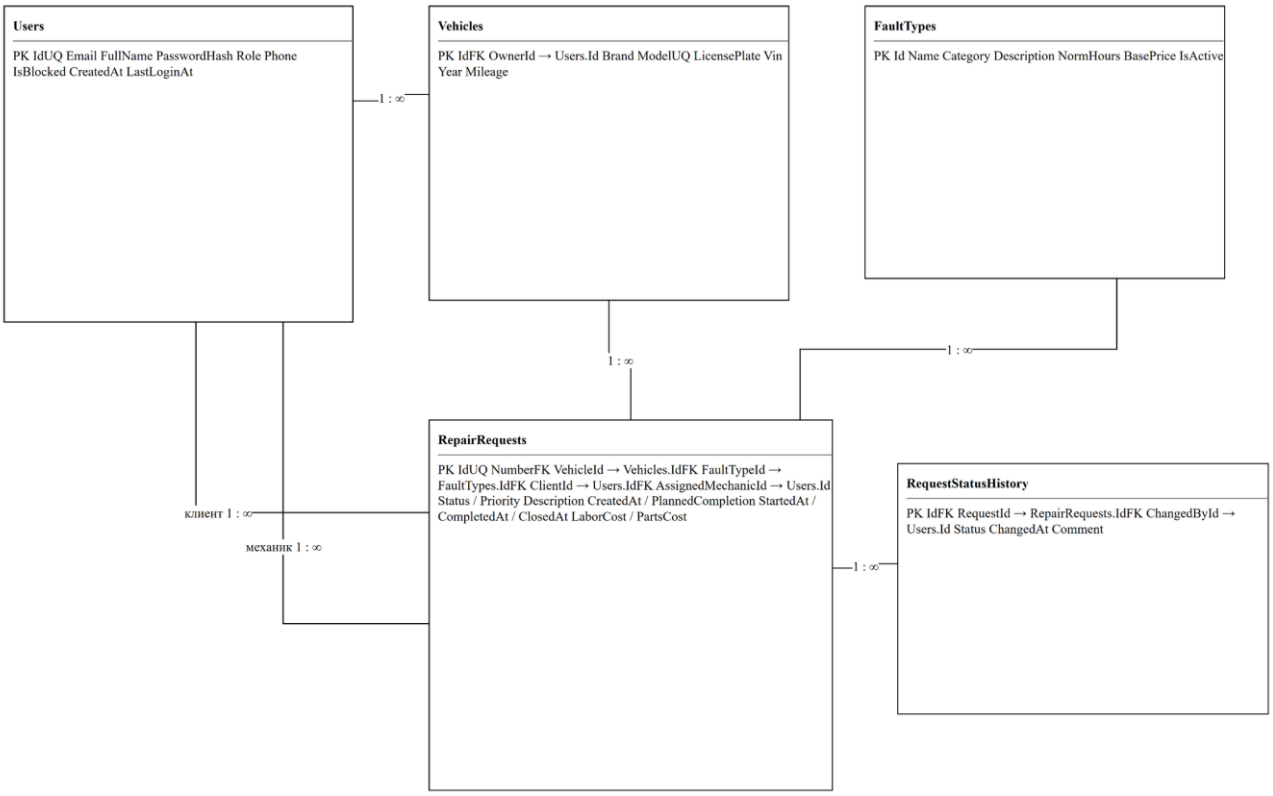


Рисунок 1.4 — ER-диаграмма базы данных

Таблица Users хранит учётные записи. Первичный ключ Id имеет автоинкремент; поле Email объявлено уникальным, что исключает регистрацию двух пользователей с одинаковой почтой. Таблица Vehicles связана с Users внешним ключом OwnerId — каждый автомобиль принадлежит ровно одному пользователю, а пользователь может иметь несколько автомобилей. Государственный номер также уникален.

Таблица FaultTypes является независимым справочником видов работ. Таблица RepairRequests ссылается на Vehicles, FaultTypes и Users тремя внешними ключами (VehicleId, FaultTypeId, ClientId), а также содержит необязательный внешний ключ AssignedMechanicId. Таблица RequestStatusHistory связана с RepairRequests отношением «один ко многим» и хранит аудит изменений. Для ускорения выборок созданы индексы по дате создания, статусу, клиенту, автомобилю, неисправности и механику.

Таблица 1.3 — Сущности базы данных

Таблица	Назначение	Первичный ключ	Внешние ключи
Users	Пользователи и роли	Id	—
Vehicles	Транспортные средства	Id	OwnerId → Users.Id

Таблица	Назначение	Первичный ключ	Внешние ключи
FaultTypes	Справочник неисправностей	Id	—
RepairRequests	Заявки на ремонт	Id	VehicleId → Vehicles.Id; FaultTypeId → FaultTypes.Id; ClientId → Users.Id; AssignedMechanicId → Users.Id
RequestStatusHistory	История статусов	Id	RequestId → RepairRequests.Id; ChangedById → Users.Id

Нормализация. Исходное отношение «Заявка» содержало бы сведения о клиенте, его телефоне, автомобиле, марке, модели, госномере, неисправности, категории, нормативе и стоимости в одной таблице. При этом возникают транзитивные зависимости — номер заявки определяет клиента, а клиент — его телефон; номер заявки определяет автомобиль, а автомобиль — марку, модель и владельца; номер заявки определяет неисправность, а неисправность — категорию и норматив. Это нарушает третью нормальную форму. Декомпозиция на таблицы Users, Vehicles, FaultTypes и RepairRequests устраняет дублирование и аномалии вставки, обновления и удаления (рисунок 1.5).

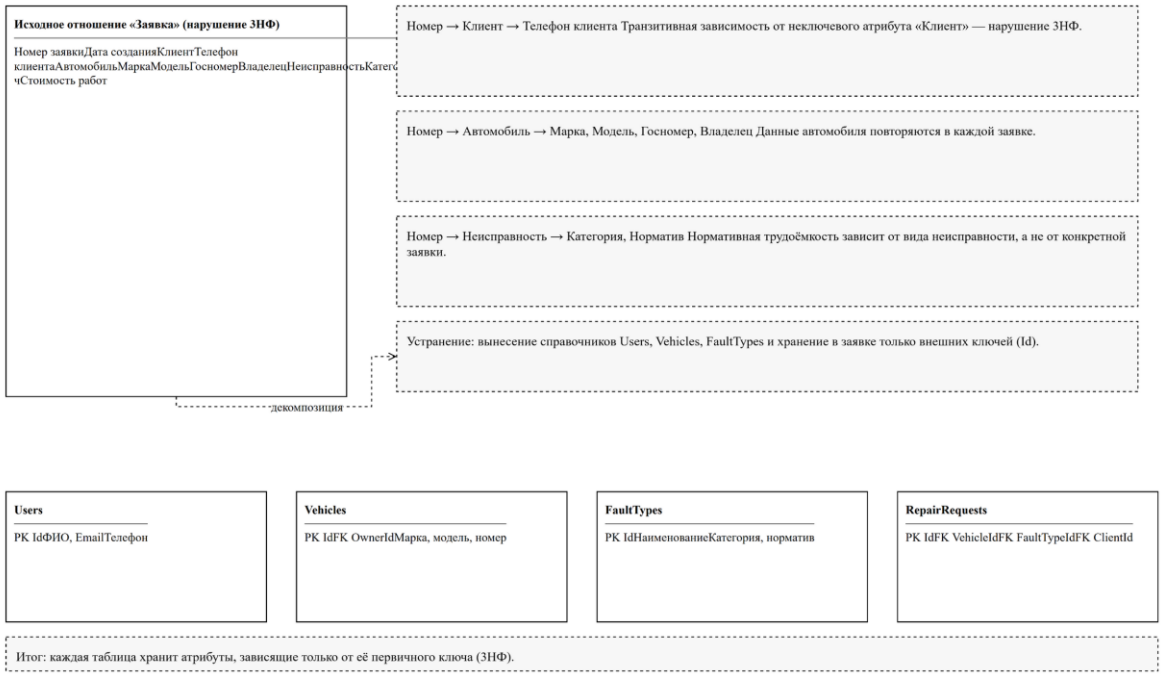


Рисунок 1.5 — Приведение схемы к третьей нормальной форме

После декомпозиции каждая таблица хранит атрибуты, зависящие только от её первичного ключа. В Users — сведения о пользователе, в Vehicles — об автомобиле, в FaultTypes — о виде работ, в RepairRequests —

только внешние ключи и параметры конкретного обращения. Схема находится в третьей нормальной форме. Следует отметить особенность SQLite — из-за динамической типизации десятичные числа EF Core сохраняет в текстовом виде, а точность задаётся на уровне модели (HasPrecision), что не влияет на логическую нормализацию.

1.7. Правила ссылочной целостности

Ссылочная целостность обеспечивается на трёх уровнях — ограничениями базы данных, правилами Entity Framework Core и проверками в репозиториях и формах. Основные правила приведены в таблице 1.4.

Таблица 1.4 — Правила ссылочной целостности

Связь	Правило	Поведение при удалении
Vehicles.OwnerId → Users.Id	Владелец обязателен	RESTRICT — пользователя с автомобилями удалить нельзя
RepairRequests.VehicleId → Vehicles.Id	Автомобиль обязателен	RESTRICT — автомобиль с заявками удалить нельзя
RepairRequests.FaultTypeId → FaultTypes.Id	Неисправность обязательна	RESTRICT — вид работ с заявками удалить нельзя
RepairRequests.ClientId → Users.Id	Клиент обязателен	RESTRICT
RepairRequests.AssignedMechanicId → Users.Id	Механик необязателен	RESTRICT
RequestStatusHistory.RequestId → RepairRequests.Id	История принадлежит заявке	CASCADE — при удалении заявки история удаляется
RequestStatusHistory.ChangedById → Users.Id	Автор изменения необязателен	RESTRICT

Каскадное удаление применено только для истории статусов, поскольку она является составной частью заявки и не имеет самостоятельной ценности. Все справочные записи защищены запретом удаления — если на автомобиль или неисправность ссылается хотя бы одна заявка, операция отклоняется. В пользовательском интерфейсе это реализовано предварительной проверкой HasRequestsAsync — пользователь получает понятное сообщение вместо исключения базы данных. Дополнительно действуют ограничения уникальности для Email, государственного номера и номера заявки, а также ограничения длины и диапазонов значений, заданные атрибутами DataAnnotations.

1.8. Диаграммы последовательности и состояний

Взаимодействие объектов при создании заявки показано на диаграмме последовательности (рисунок 1.6). Клиент открывает раздел «Мои заявки», нажимает «Добавить» и заполняет поля формы. Форма вызывает Validate(), затем RequestService.CreateAsync(dto) — сервис запрашивает у репозитория последний номер за год, формирует номер вида ЗР-2026-0045, присваивает статус «Новая» и сохраняет заявку и запись истории в SQLite. При некорректных данных выводится сообщение об ошибке, запись не создаётся.

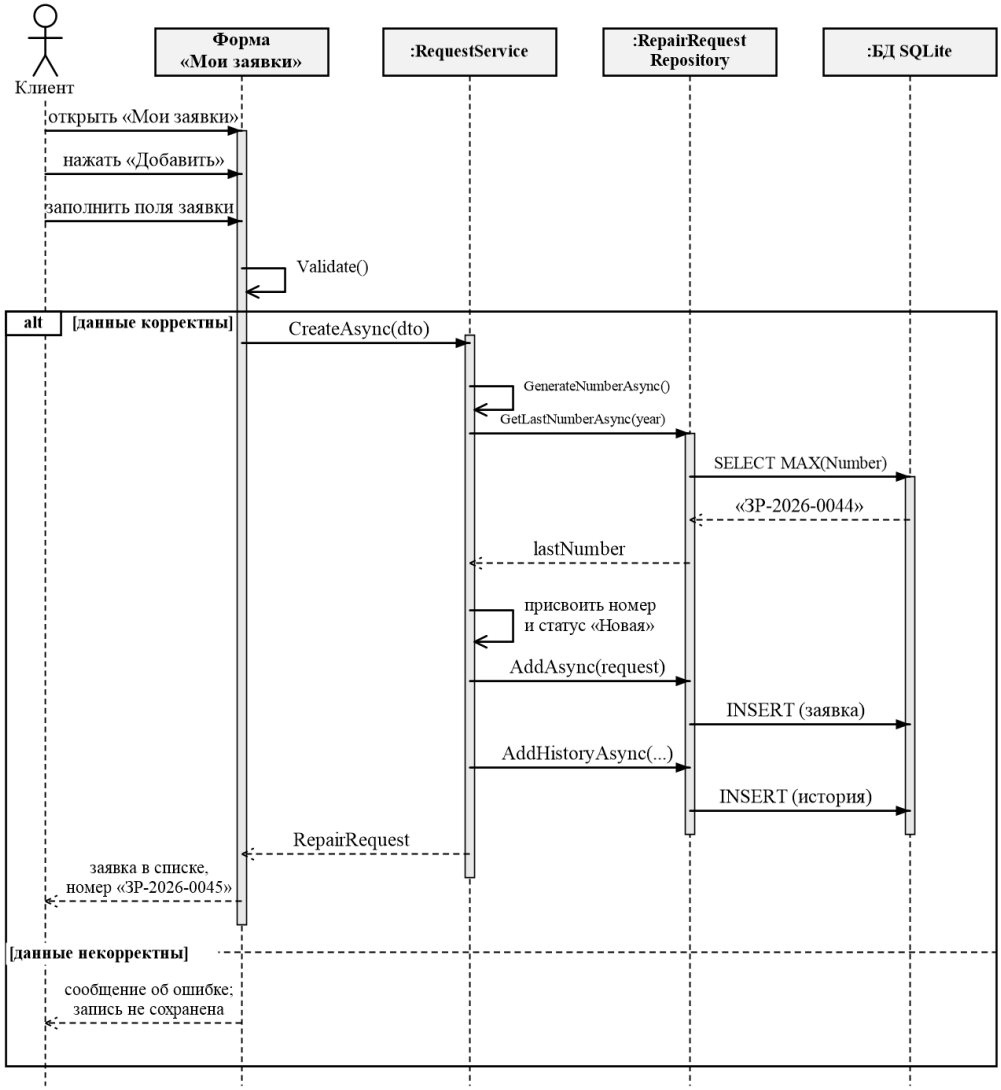


Рисунок 1.6 — Диаграмма последовательности создания заявки

Жизненный цикл заявки отражён на диаграмме состояний (рисунок 1.7). Заявка последовательно проходит состояния «Новая», «Принята», «В

работе», «Выполнена» и «Закрыта»; из промежуточных состояний допускается отмена с указанием причины. При каждом переходе фиксируются временная отметка и запись в историю статусов.

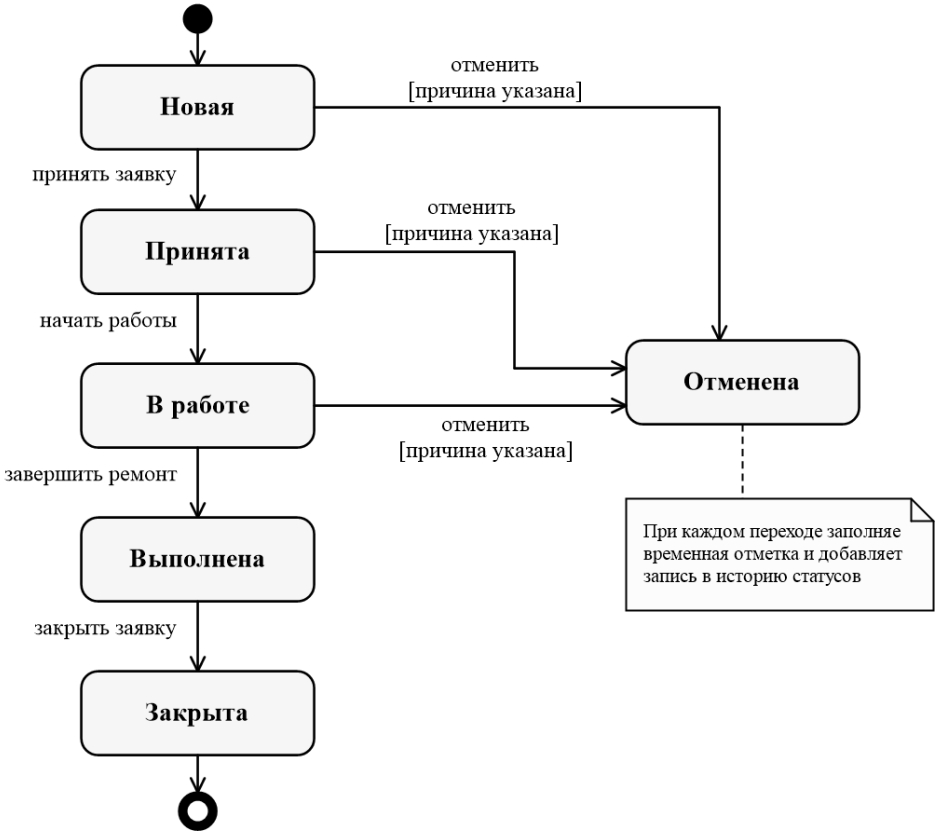


Рисунок 1.7 — Диаграмма состояний заявки

Таким образом, на этапе проектирования определены границы системы, участники, функциональные и нефункциональные требования, структура предприятия, варианты использования, классы, схема базы данных и правила целостности. Эти результаты служат основой для реализации, описанной в следующем разделе.

2. РАЗРАБОТКА И ИНТЕГРАЦИЯ МОДУЛЕЙ СИСТЕМЫ

2.1. Настройка среды разработки и Git-репозитория

Разработка велась на персональном компьютере под управлением Windows 11. В качестве платформы выбрана .NET 10 и технология Windows Forms — она позволяет быстро создавать настольные приложения с богатым набором элементов управления, не требует установки веб-сервера и хорошо подходит для учебного проекта, развёртываемого на одном компьютере. Основные инструменты приведены в таблице 2.1.

Таблица 2.1 — Инструменты разработки

Инструмент	Версия	Назначение
.NET SDK	10.0	Сборка и запуск приложений C#
Visual Studio / Rider	2022+ / 2024+	Среда разработки, отладка
Entity Framework Core	10.0.13	ORM, миграции, доступ к SQLite
SQLite	3.x (встроенная)	Локальная база данных
ClosedXML	0.105.1	Экспорт отчётов в Excel
QuestPDF	2026.9.1	Формирование PDF-отчётов
Serilog	4.4.0	Журналирование событий
xUnit	2.9.3	Модульные и интеграционные тесты
Git	2.4x	Контроль версий
draw.io Desktop	24.x	Диаграммы в редактируемом формате

Решение состоит из трёх проектов, разделённых по слоям (таблица 2.2). Такое разделение позволяет тестировать бизнес-логику без запуска графического интерфейса и упрощает дальнейшее развитие приложения.

Таблица 2.2 — Структура решения

Проект	Тип	Содержимое
RepairRequests.Core	Библиотека net10.0	Модели, перечисления, DbContext, миграции, репозитории, сервисы, экспорт
RepairRequests.WinForms	WinExe net10.0-windows	Формы, тема оформления, конфигурация, журналирование
RepairRequests.Tests	xUnit net10.0	133 модульных и интеграционных теста

Архитектура приложения приведена на рисунке 2.1. Слой представления вызывает сервисы и репозитории, расположенные в Core; те проекты взаимодействуют с базой данных через EF Core. Внешние библиотеки (ClosedXML, QuestPDF, Serilog) подключены только там, где они необходимы.

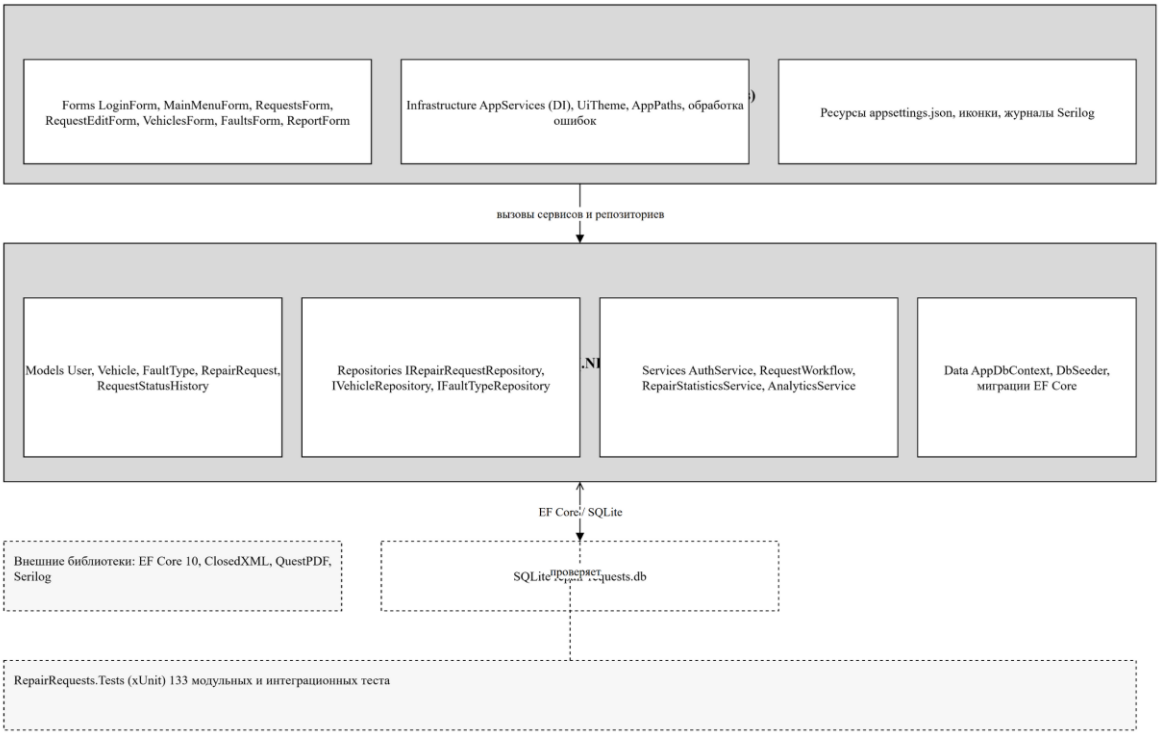


Рисунок 2.1 — Архитектура приложения

Для контроля версий инициализирован Git-репозиторий. Файл .gitignore исключает сборочные артефакты (bin/, obj/), файлы базы данных (*.db, *.db-shm, *.db-wal), журналы, каталоги IDE и служебные файлы Office. Первый коммит 83bb374 содержит каркас решения, модели, миграцию и тесты. Основные команды, использованные при настройке

```
git init
git config core.autocrlf true
git add .
git commit -m "Каркас решения: Core, WinForms, тесты, миграция InitialCreate"
git log --oneline
```

Сборка и запуск выполняются стандартными командами .NET

```
dotnet build RepairRequests.slnx -v q --nologo
dotnet run --project src/RepairRequests.WinForms
dotnet test tests/RepairRequests.Tests/RepairRequests.Tests.csproj -v q --nologo
```

Сборка завершается без ошибок и предупреждений, что подтверждает корректность настроек и отсутствие устаревших API.

2.2. Создание базы данных и миграций

В качестве СУБД выбрана SQLite — она не требует установки сервера, хранит всю базу в одном файле и полностью поддерживается Entity Framework Core. Строка подключения находится в appsettings.json и по умолчанию указывает на файл data/repair-requests.db рядом с исполняемым

файлом. При первом запуске приложение создаёт каталоги data, logs и reports, применяет миграции и, если база пуста, заполняет её демонстрационными данными.

Контекст базы данных AppDbContext наследует DbContext и настраивает таблицы, индексы и связи. Фрагмент конфигурации заявки приведён ниже.

```
modelBuilder.Entity<RepairRequest>(entity =>
{
    entity.HasIndex(r => r.Number).IsUnique();
    entity.HasIndex(r => r.CreatedAt);
    entity.HasIndex(r => r.Status);
    entity.Property(r => r.Status).HasConversion<string>().HasMaxLength(20);
    entity.Property(r => r.Priority).HasConversion<string>().HasMaxLength(20);
    entity.Property(r => r.LaborCost).HasPrecision(12, 2);
    entity.Property(r => r.PartsCost).HasPrecision(12, 2);

    entity.HasOne(r => r.Vehicle)
        .WithMany(v => v.Requests)
        .HasForeignKey(r => r.VehicleId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

Перечисления хранятся в виде строк (HasConversion<string>()), что делает базу читаемой при просмотре в любом SQLite-клиенте и защищает от изменения порядка числовых значений. Десятичные суммы имеют точность 12,2, нормативы — 6,2. Для временных отметок используется тип TEXT, штатно поддерживаемый SQLite.

Миграция InitialCreate создана командой dotnet tool run dotnet-ef migrations add InitialCreate. Она содержит пять таблиц, внешние ключи, уникальные индексы и индексы для выборок. Применение миграции выполняется автоматически при старте, а также вручную командой dotnet tool run dotnet-ef database update. Откат возможен командой database update <имя_предыдущей_миграции>.

Демонстрационные данные генерируются детерминированно с помощью Random(2026), поэтому при повторном развёртывании получается одинаковый набор, удобный для проверки отчётов. Состав данных приведён в таблице 2.3.

Таблица 2.3 — Демонстрационные данные

Сущность	Количество	Примеры
Пользователи	6	администратор, 2 менеджера-механика, 3 клиента

Сущность	Количество	Примеры
Транспортные средства	7	Lada Vesta, Toyota Camry, Ford Transit, ГАЗ Газель Next, Mercedes-Benz Sprinter, Kia Rio, Skoda Octavia
Неисправности	10	замена тормозных колодок, ремонт двигателя, замена масла, замена амортизатора, ремонт генератора, замена сцепления и др.
Заявки	42	34 завершённые и закрытые, 3 отменённые, 5 активных
Записи истории	182	создание, принятие, начало работ, завершение, закрытие, отмена

Заявки распределены по пяти месяцам (июнь–ноябрь 2026 года). Длительность ремонта формируется по экспоненциальному закону с масштабом, зависящим от норматива неисправности, что делает демонстрационную выборку пригодной для статистического анализа. Всего в базе 34 завершённых ремонта, что достаточно для расчёта среднего, медианы, дисперсии и доверительного интервала.

2.3. Модуль авторизации

Модуль авторизации включает модель User, сервис аутентификации AuthService, хэширование паролей и форму входа. Пароль никогда не сохраняется в открытом виде — при регистрации вычисляется хэш PBKDF2-HMAC-SHA256 с 120 000 итераций, случайной солью длиной 16 байт и ключом длиной 32 байта. Строка хранения имеет формат pbkdf2-sha256$120000$соль$хэш, где соль и хэш закодированы в Base64.

```
public string Hash(string password)
{
    var salt = RandomNumberGenerator.GetBytes(16);
    var key = Rfc2898DeriveBytes.Pbkdf2(
        password, salt, 120_000, HashAlgorithmName.SHA256, 32);
    return $"pbkdf2-sha256${120_000}${Convert.ToBase64String(salt)}"
         + $"${Convert.ToBase64String(key)}";
}

public bool Verify(string password, string storedHash)
{
    // разбор строки, повторное вычисление ключа с той же солью
    var actual = Rfc2898DeriveBytes.Pbkdf2(
        password, salt, iterations, HashAlgorithmName.SHA256, expected.Length);
    return CryptographicOperations.FixedTimeEquals(actual, expected);
}
```

Сравнение выполняется за постоянное время (FixedTimeEquals), что снижает риск атак по времени. Соль уникальна для каждого пароля, поэтому одинаковые пароли разных пользователей дают разные хэши. Итерации

сохраняются в строке хэша, что позволяет в дальнейшем повышать их количество без потери совместимости.

Сервис AuthService реализует вход и регистрацию. При входе проверяются пустые поля, наличие пользователя, правильность пароля и признак блокировки. При успешном входе обновляется дата последнего входа. При регистрации проверяются длина ФИО, формат электронной почты, длина пароля (не менее 8 символов), наличие букв и цифр, совпадение паролей и уникальность почты. Новый пользователь получает роль Client.

```
var user = await _users.GetByEmailAsync(email);
if (user is null || !_hasher.Verify(password, user.PasswordHash))
    return AuthResult.Fail("Неверная электронная почта или пароль.");

if (user.IsBlocked)
    return AuthResult.Fail("Учётная запись заблокирована. Обратитесь к администратору.");

user.LastLoginAt = DateTime.Now;
await _users.UpdateAsync(user);
return AuthResult.Ok(user);
```

Форма входа содержит поля электронной почты и пароля, кнопку «Войти», ссылку на регистрацию и переключатель показа пароля. Для демонстрации предусмотрены подсказки с учётными данными администратора; в реальной эксплуатации они отключаются. При ошибке форма выводит понятное сообщение и не раскрывает, что именно неверно — почта или пароль. Внешний вид формы приведён на рисунке 2.2.

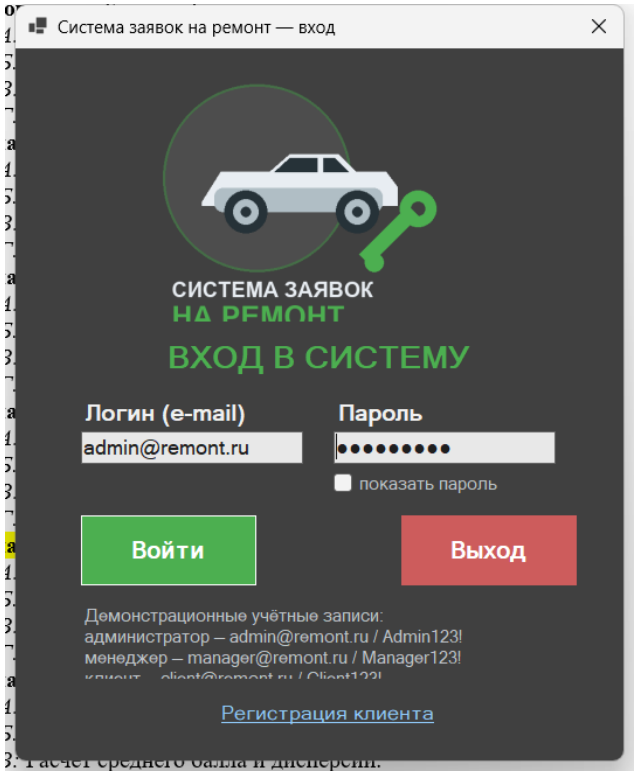


Рисунок 2.2 — Форма входа в систему

Форма регистрации (рисунок 2.3) создаёт учётную запись клиента. После успешной регистрации пользователь возвращается на форму входа; автоматический вход не выполняется, чтобы исключить случайное создание учётной записи с ошибочными данными.

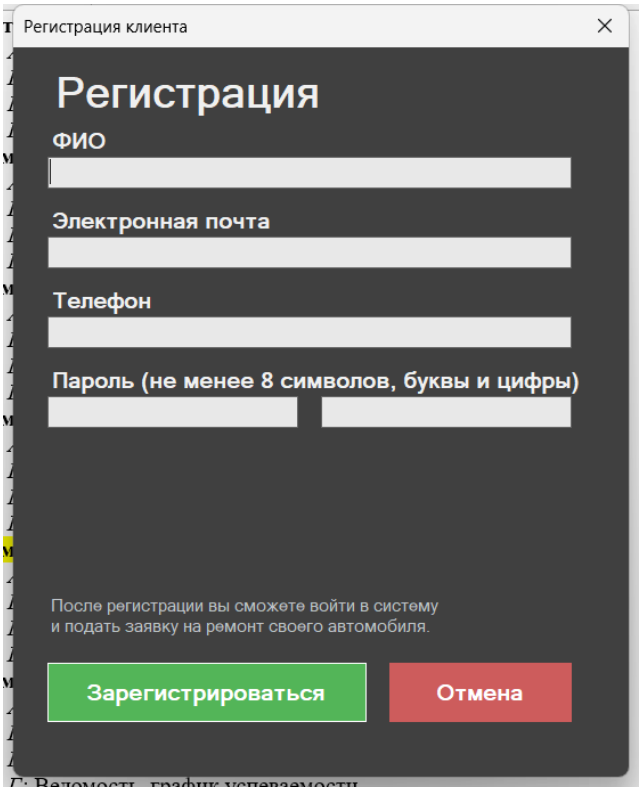


Рисунок 2.3 — Форма регистрации клиента

2.4. Разграничение ролей

В системе определены три роли — Client, Manager и Administrator. Роль хранится в поле Role таблицы Users и преобразуется в строку при сохранении. После входа текущий пользователь сохраняется в статическом свойстве AppServices.CurrentUser, а признаки IsManager и ClientScope используются формами для выбора режима работы.

Матрица прав приведена в таблице 2.4. Администратор в текущей версии не имеет отдельного интерфейса управления пользователями — он обладает полными правами менеджера и отвечает за сопровождение базы данных, резервное копирование и обновление приложения. Клиент видит только собственные записи — в запросы принудительно добавляется условие ClientId = текущий пользователь.

Таблица 2.4 — Матрица прав доступа

Операция	Клиент	Менеджер	Администратор
Вход и регистрация	+	+	+
Просмотр своих заявок	+	+	+
Создание заявки	+	+	+
Отмена своей заявки	+	—	—
Просмотр всех заявок	—	+	+
Изменение статусов	—	+	+
Назначение механика и стоимости	—	+	+
Ведение справочника ТС	свои	все	все
Ведение справочника неисправностей	—	+	+
Отчёт и экспорт	свои	все	все
Сопровождение БД	—	—	+

Разграничение реализовано не только скрытием кнопок, но и проверками в коде. Например, удаление заявки доступно только менеджеру — кнопка скрыта для клиента, а метод DeleteSelectedAsync дополнительно проверяет AppServices.IsManager. Аналогично клиент не может изменить стоимость работ или назначить механика — соответствующие поля блокируются при построении формы. Такая двойная защита исключает обход ограничений через горячие клавиши или программный вызов.

Главное меню адаптируется под роль — у менеджера и администратора отображаются разделы «Заявки», «Автомобили», «Неисправности», «Отчёт» и «Выход»; у клиента — «Мои заявки», «Мой транспорт», «Отчёт» и «Выход». Внешний вид меню менеджера приведён на рисунке 2.4, меню клиента — на рисунке 2.5.

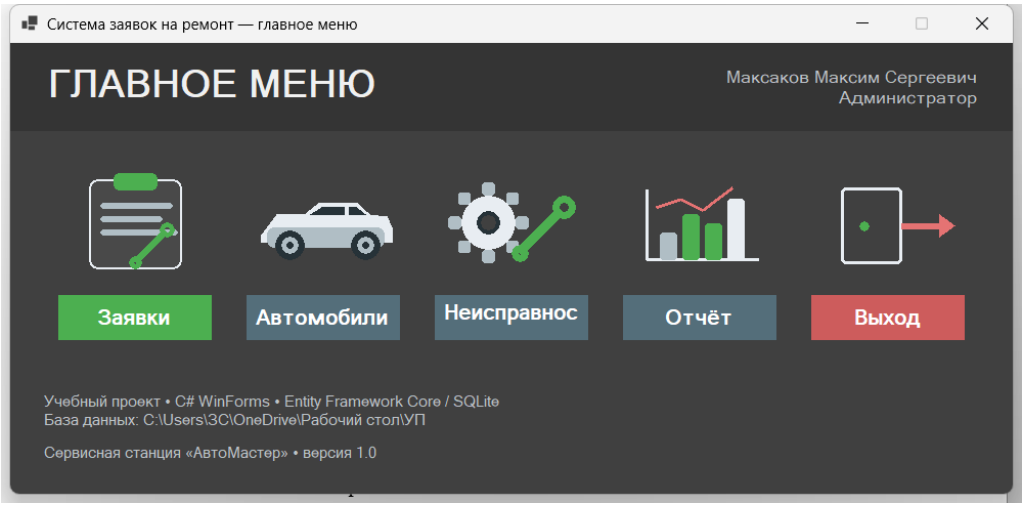


Рисунок 2.4 — Главное меню менеджера (администратора)

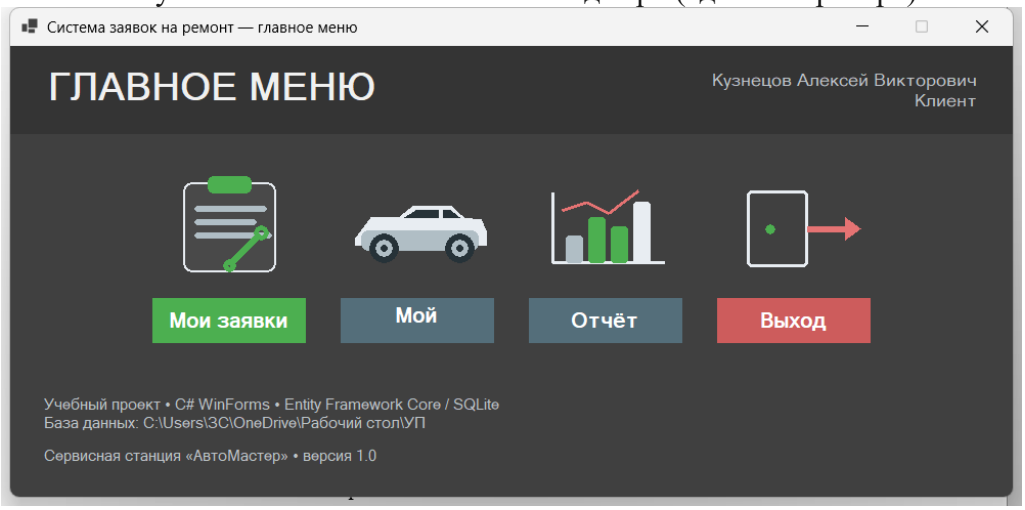


Рисунок 2.5 — Главное меню клиента

Кнопка «Выход» завершает сеанс и возвращает пользователя на форму входа; приложение при этом не закрывается. Полное завершение работы выполняется кнопкой «Выход» на самой форме входа. Такое поведение исключает случайную потерю несохранённых данных при смене пользователя.

2.5. Модуль справочников

В предметной области станции технического обслуживания роль справочников выполняют транспортные средства и виды неисправностей.

Оба справочника поддерживают полный набор операций CRUD, поиск, валидацию и контроль связанных записей.

2.5.1. Справочник транспортных средств

Модель Vehicle содержит владельца, марку, модель, государственный номер, VIN, год выпуска, пробег и дату добавления. Для полей заданы ограничения. Марка — от 2 до 60 символов, модель — от 1 до 60, год — от 1950 до 2100, пробег — неотрицательное число. Государственный номер проверяется регулярным выражением, допускающим кириллические и латинские буквы — ^[АВЕКМНОРСТУХА-Z]\d{3}[АВЕКМНОРСТУХА-Z]{2}\d{2,3}$. VIN необязателен, но если указан, должен содержать ровно 17 символов и не содержать букв I, O, Q.

Форма справочника (рисунок 2.6) отображает таблицу с колонками «Марка», «Модель», «Госномер», «Год», «Пробег» и «Владелец», поле поиска и кнопки «Добавить», «Изменить», «Удалить», «Обновить». Поиск выполняется по марке, модели, номеру и ФИО владельца. Для клиента список ограничен его автомобилями, а колонка «Владелец» скрыта.

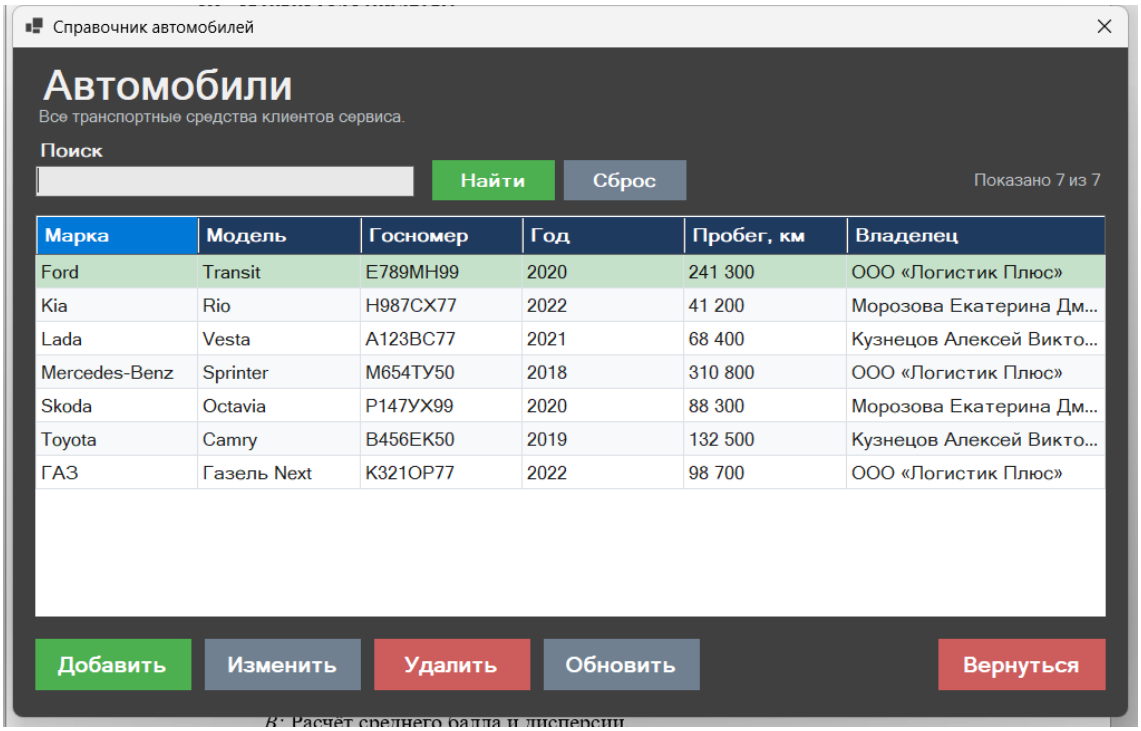


Рисунок 2.6 — Справочник транспортных средств

Форма редактирования (рисунок 2.7) содержит поля марки, модели, номера, VIN, года, пробега и владельца. При сохранении выполняются

проверки DataAnnotations, проверка уникальности государственного номера и проверка существования владельца. Если запись связана с заявками, удаление блокируется с сообщением «Нельзя удалить автомобиль — по нему уже есть заявки на ремонт».

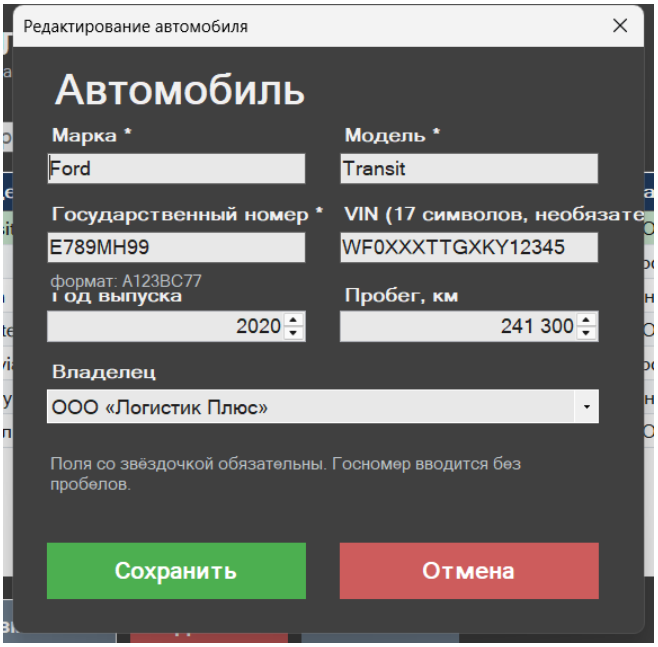


Рисунок 2.7 — Редактирование автомобиля

2.5.2. Справочник неисправностей

Модель FaultType включает наименование, категорию, описание, нормативную трудоёмкость в часах, базовую стоимость работ и признак активности. Норматив ограничен диапазоном 0,1–200 часов, стоимость — неотрицательным значением. Неактивные записи сохраняются для истории, но не предлагаются при создании новых заявок.

Форма справочника (рисунок 2.8) поддерживает поиск по наименованию и категории, фильтрацию по категории и отображение признака активности. Форма редактирования (рисунок 2.9) позволяет изменить все поля и снабжена пояснением о том, что снятие признака активности не удаляет запись и не влияет на уже созданные заявки.

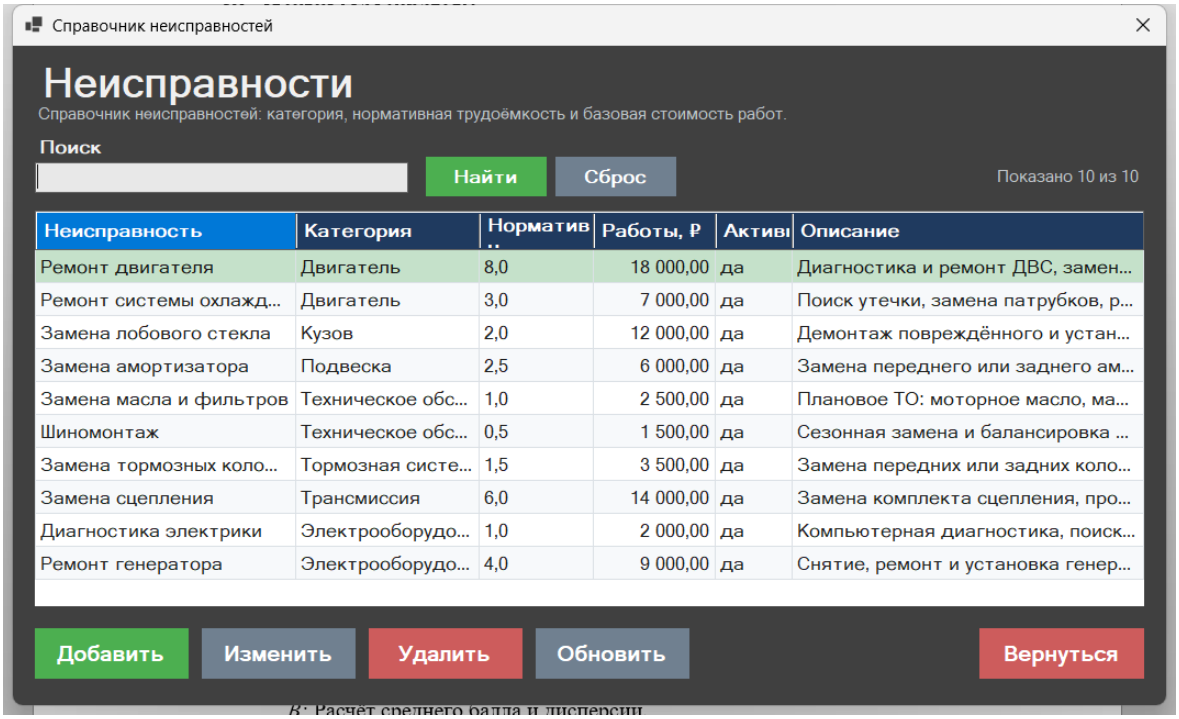


Рисунок 2.8 — Справочник неисправностей

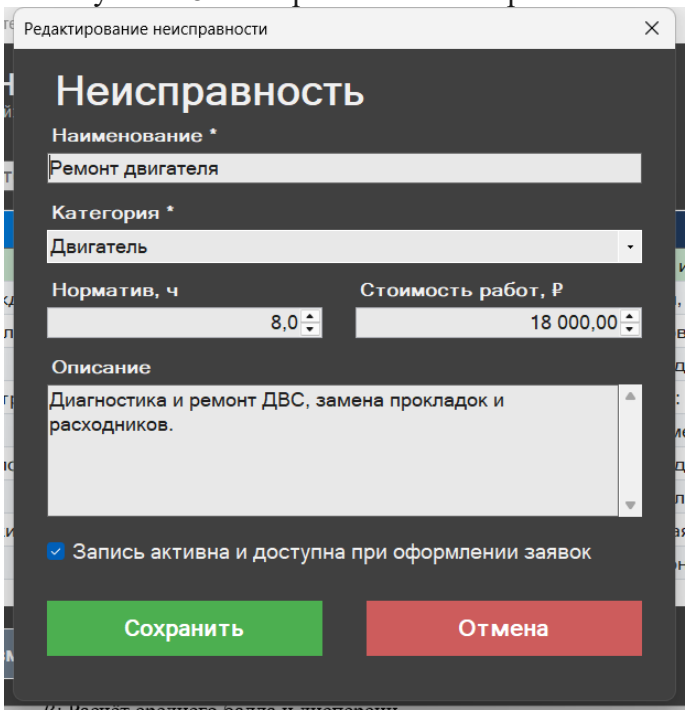


Рисунок 2.9 — Редактирование неисправности

Оба справочника используют общий подход — загрузка данных в DataGridView, привязка к анонимным или проектным строкам, асинхронные вызовы репозиториев, централизованный вывод ошибок через UiTheme.Error. Поиск выполняется на стороне базы данных, что позволяет работать с большими объёмами без потери производительности.

2.6. Модуль заявок

Заявка — центральная сущность системы. Она содержит номер, ссылки на автомобиль, неисправность, клиента и механика, описание, приоритет, статус, временные отметки, стоимости работ и запчастей, сведения о валюте и комментарий. Номер формируется автоматически в формате ЗР-ГГГГ-NNNN, где NNNN — следующий порядковый номер в пределах года.

```
public async Task<string> GenerateNumberAsync()
{
    var year = DateTime.Now.Year;
    var prefix = $"ЗР-{year}-";
    var last = await db.RepairRequests
        .Where(r => r.Number.StartsWith(prefix))
        .OrderByDescending(r => r.Number)
        .Select(r => r.Number)
        .FirstOrDefaultAsync();
    var next = last is not null && int.TryParse(last[^4..], out var parsed)
        ? parsed + 1 : 1;
    return prefix + next.ToString("D4");
}
```

Жизненный цикл заявки включает шесть статусов — «Новая», «Принята», «В работе», «Выполнена», «Закрыта» и «Отменена». Допустимые переходы заданы явно и проверяются сервисом RequestWorkflow. Например, из статуса «Новая» можно перейти только в «Принята» или «Отменена»; из «Выполнена» — только в «Закрыта». Отмена требует обязательного комментария с причиной. Диаграмма состояний приведена на рисунке 1.7.

Таблица 2.5 — Жизненный цикл заявки

Текущий статус	Допустимые переходы	Действие в интерфейсе
Новая	Принята, Отменена	Принять заявку / Отменить заявку
Принята	В работе, Отменена	Начать работы / Отменить заявку
В работе	Выполнена, Отменена	Завершить ремонт / Отменить заявку
Выполнена	Закрыта	Закрыть заявку
Закрыта	—	Только просмотр
Отменена	—	Только просмотр

При каждом переходе сервис проставляет соответствующую временную отметку (AcceptedAt, StartedAt, CompletedAt, ClosedAt) и добавляет запись в историю с указанием автора и комментария. Недопустимый переход вызывает исключение InvalidOperationException, которое перехватывается формой и выводится пользователю. Фрагмент реализации приведён ниже.

```
public static void Apply(RepairRequest request, RequestStatus target,
                         int changedById, string? comment = null)
{
    if (!CanTransition(request.Status, target))
        throw new InvalidOperationException(
            $"Переход «{request.Status.Ru()}» → «{target.Ru()}» недопустим.");

    if (target == RequestStatus.Cancelled && string.IsNullOrWhiteSpace(comment))
        throw new InvalidOperationException(
            "Для отмены заявки необходимо указать причину.");

    request.Status = target;
    // установка AcceptedAt / StartedAt / CompletedAt / ClosedAt
    request.History.Add(new RequestStatusHistory
    {
        Status = target, ChangedAt = now,
        ChangedById = changedById, Comment = comment
    });
}
```

Форма списка заявок (рисунок 2.10) содержит поле поиска, фильтры по статусу, приоритету и датам, таблицу с десятью колонками и кнопки управления. Двойной щелчок по строке открывает форму редактирования. Для клиента заголовок меняется на «Мои заявки», кнопка «Удалить» скрывается, а в выборку добавляется ограничение по клиенту.

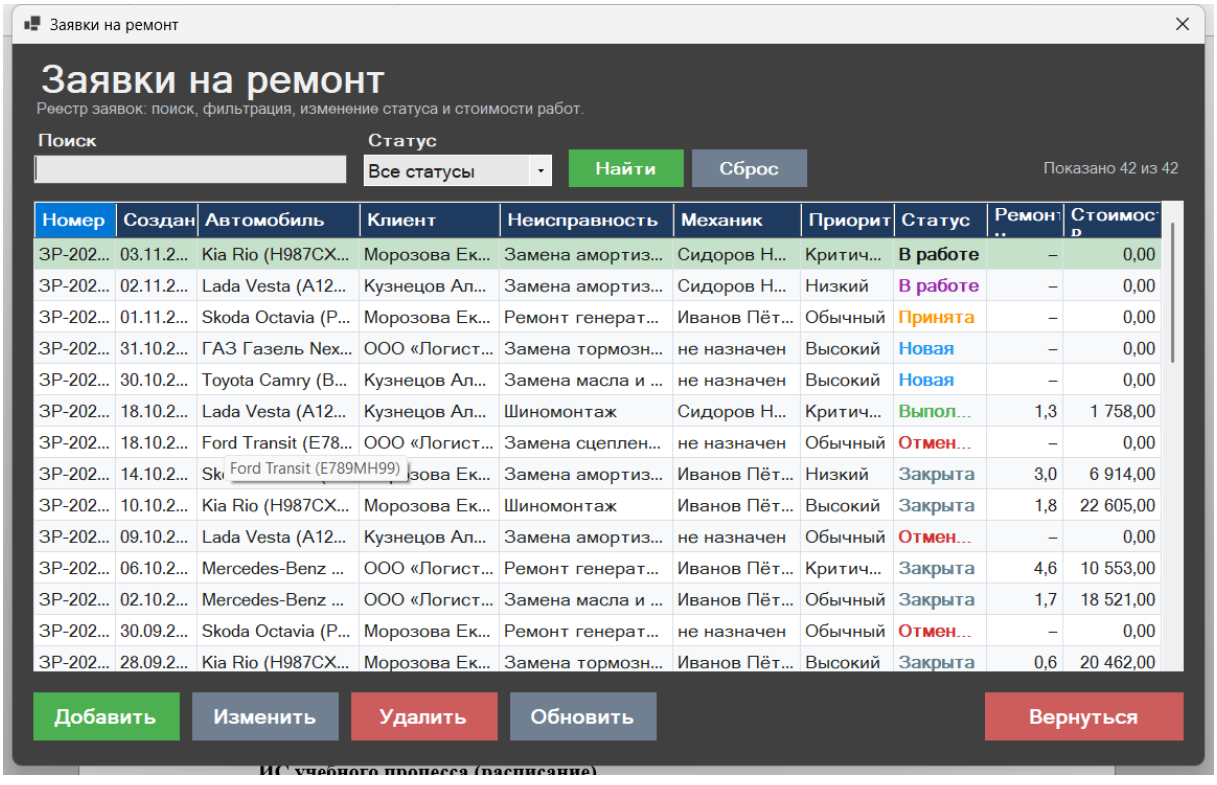


Рисунок 2.10 — Список заявок (менеджер)

Форма редактирования (рисунок 2.11) разделена на три зоны — данные заявки, финансовая информация и история статусов. В верхней части отображаются номер, статус, клиент, автомобиль, неисправность, приоритет, механик, плановая дата и описание. В правой колонке расположены кнопки

допустимых переходов. В нижней части показана история изменений с датой, статусом, пользователем и комментарием. Для клиента поля стоимости, механика и служебного комментария скрыты, а доступна только отмена заявки.

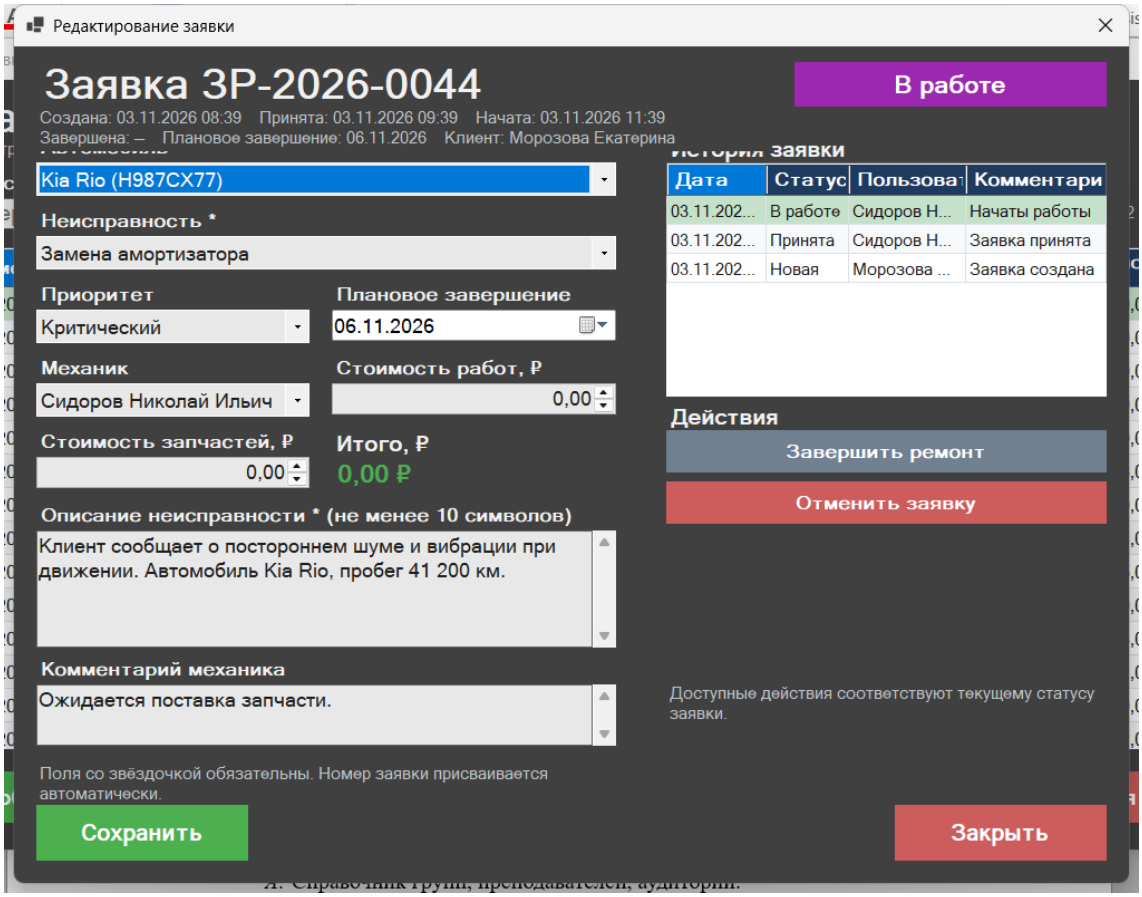


Рисунок 2.11 — Редактирование заявки

Клиентский режим показан на рисунке 2.12. Клиент видит только свои заявки, может создать новую, отменить её до начала работ и просмотреть историю. Стоимость работ и запчастей доступна только для чтения, поскольку её определяет менеджер.

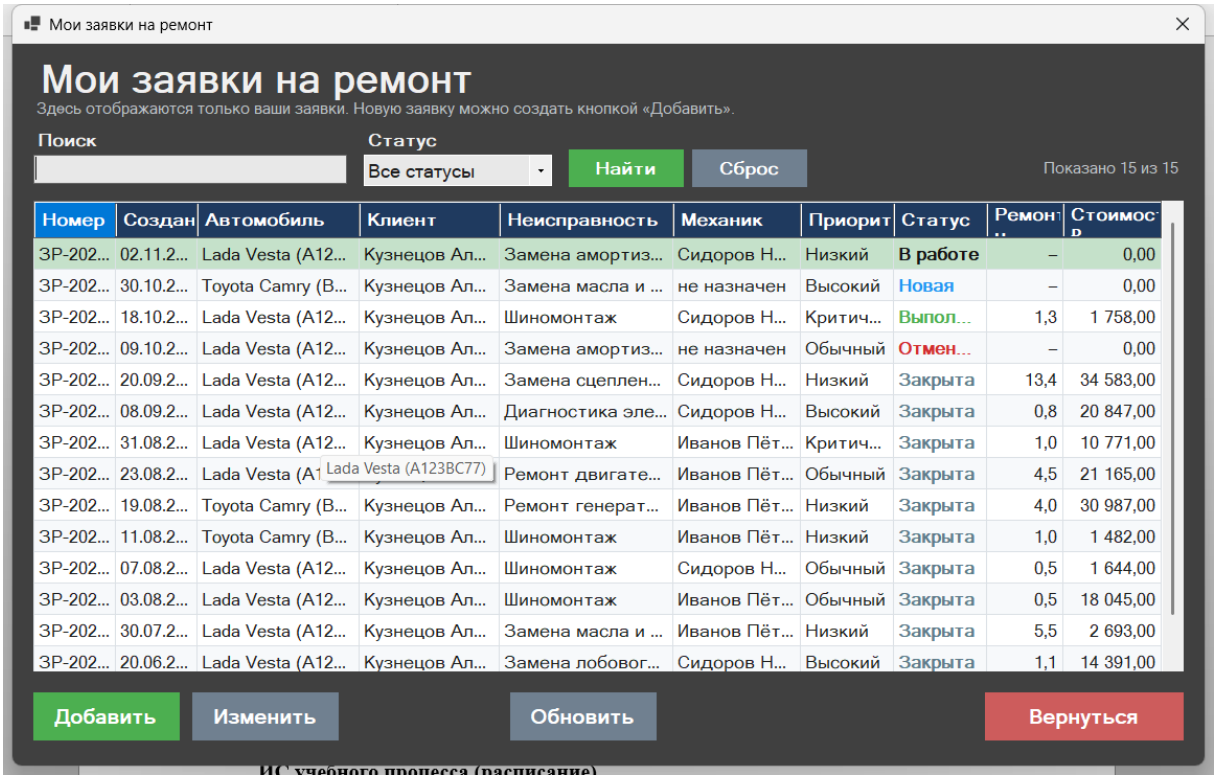


Рисунок 2.12 — Список заявок клиента

При сохранении заявки выполняются проверки — описание должно содержать от 10 до 1000 символов, стоимости не могут быть отрицательными, плановая дата не должна быть в прошлом (для новых заявок), автомобиль должен принадлежать выбранному клиенту, неисправность должна быть активной. Проверки выполняются как атрибутами DataAnnotations, так и явными методами формы, что даёт понятные сообщения на русском языке.

2.7. Слой доступа к данным

Доступ к данным организован по шаблону «репозиторий». Для каждой сущности определен интерфейс (IRepairRequestRepository, IVehicleRepository, IFaultTypeRepository, IUserRepository) и реализация на EF Core. Интерфейсы находятся в Core и не зависят от WinForms, что позволяет использовать их в тестах и потенциально в других клиентах.

Репозитории получают IDbContextFactory<AppDbContext> и создают новый контекст на каждую операцию. Это исключает проблемы с длительным отслеживанием сущностей, утечками памяти и конфликтами при параллельных вызовах. Для операций чтения применяется AsNoTracking(),

для загрузки связанных данных — Include и ThenInclude. При обновлении заявки навигационные свойства отсоединяются, чтобы EF Core не пытался повторно вставить автомобиль, клиента или неисправность.

Таблица 2.6 — Методы репозиториев

Репозиторий	Основные методы
IRepairRequestRepository	GetPagedAsync, GetByIdAsync, GetRecentAsync, GetForPeriodAsync, GetCompletedInPeriodAsync, GetDashboardStatsAsync, GenerateNumberAsync, AddAsync, UpdateAsync, DeleteAsync, HasRequestsForVehicleAsync, HasRequestsForFaultTypeAsync
IVehicleRepository	GetPagedAsync, GetAllAsync, GetByIdAsync, AddAsync, UpdateAsync, DeleteAsync, HasRequestsAsync, LicensePlateExistsAsync
IFaultTypeRepository	GetPagedAsync, GetActiveAsync, GetCategoriesAsync, GetByIdAsync, AddAsync, UpdateAsync, DeleteAsync, HasRequestsAsync, NameExistsAsync
IUserRepository	GetPagedAsync, GetMechanicsAsync, GetClientsAsync, GetByEmailAsync, GetByIdAsync, AddAsync, UpdateAsync, EmailExistsAsync, CountAsync

Фрагмент реализации выборки заявок показывает типичный подход — построение запроса, подсчёт общего количества, нормализация размера страницы, сортировка и постраничная выборка.

```
var query = db.RepairRequests
    .AsNoTracking()
    .Include(r => r.Vehicle)
    .Include(r => r.FaultType)
    .Include(r => r.Client)
    .Include(r => r.AssignedMechanic)
    .AsQueryable();

var total = await query.CountAsync();
var page = Math.Max(1, filter.Page);
var pageSize = Math.Clamp(filter.PageSize, 5, 200);
var items = await ApplySorting(query, filter.SortBy)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
```

Такой подход обеспечивает единообразие выборок и позволяет использовать один и тот же репозиторий в форме списка, в отчёте и в тестах. Все методы асинхронные, что предотвращает блокировку интерфейса при обращении к базе.

2.8. Фильтрация, сортировка и пагинация

Для выборки заявок используется класс RequestFilter, содержащий поисковую строку, статус, приоритет, идентификаторы автомобиля, неисправности и клиента, диапазон дат, параметр сортировки, номер

страницы и размер страницы. Значения по умолчанию — сортировка по дате убыванию и 20 записей на страницу.

Таблица 2.7 — Параметры фильтрации

Параметр	Назначение	Допустимые значения
Search	Поиск по номеру, описанию, автомобилю, клиенту, неисправности	строка
Status	Фильтр по статусу	New … Cancelled или пусто
Priority	Фильтр по приоритету	Low … Critical или пусто
VehicleId	Фильтр по автомобилю	идентификатор или пусто
FaultTypeId	Фильтр по неисправности	идентификатор или пусто
ClientId	Ограничение по клиенту	идентификатор или пусто
DateFrom / DateTo	Диапазон дат создания (включительно)	дата или пусто
SortBy	Сортировка	date_desc, date_asc, number, priority, status, cost
Page / PageSize	Постраничный вывод	Page ≥ 1; PageSize 5–200

Диапазон дат обрабатывается аккуратно — начальная дата приводится к началу дня, конечная — к началу следующего дня, а сравнение выполняется строгим неравенством <. Это включает все заявки, созданные в последний день периода, независимо от времени суток.

```
if (filter.DateFrom.HasValue)
{
    var from = filter.DateFrom.Value.Date;
    query = query.Where(r => r.CreatedAt >= from);
}
if (filter.DateTo.HasValue)
{
    var to = filter.DateTo.Value.Date.AddDays(1);
    query = query.Where(r => r.CreatedAt < to);
}
```

Сортировка реализована через выражение switch — по дате возрастанию и убыванию, по номеру, по приоритету (критические выше), по статусу (новые выше), по стоимости. Для стабильности результатов при равных значениях добавляется сортировка по идентификатору. Постраничный вывод возвращает объект PagedResult<T>, содержащий элементы, общее количество, номер страницы, размер страницы, вычисляемые свойства TotalPages, HasPrevious и HasNext.

В пользовательском интерфейсе пагинация отображается строкой «Страница X из Y, всего N записей» и кнопками «Назад» и «Вперёд». При изменении фильтра страница сбрасывается на первую, чтобы пользователь не оказался на несуществующей странице. Размер страницы ограничен диапазоном 5–200, что защищает приложение от случайного запроса слишком большого объёма данных.

2.9. Обеспечение ссылочной целостности

Ссылочная целостность обеспечивается комбинацией ограничений базы данных и предварительных проверок в приложении. В таблице 2.8 перечислены ограничения, реализованные в схеме.

Таблица 2.8 — Ограничения целостности

Ограничение	Реализация	Реакция приложения
Уникальность Email	UNIQUE INDEX IX_Users_Email	Сообщение «Пользователь с такой почтой уже зарегистрирован»
Уникальность госномера	UNIQUE INDEX IX_Vehicles_LicensePlate	Сообщение о занятом номере
Уникальность номера заявки	UNIQUE INDEX IX_RepairRequests_Number	Номер генерируется автоматически
Запрет удаления владельца с автомобилями	FOREIGN KEY … ON DELETE RESTRICT	Ошибка базы данных, обрабатывается формой
Запрет удаления автомобиля с заявками	FOREIGN KEY … ON DELETE RESTRICT + HasRequestsForVehicleAsync	Понятное сообщение, операция отменяется
Запрет удаления неисправности с заявками	FOREIGN KEY … ON DELETE RESTRICT + HasRequestsForFaultTypeAsync	Понятное сообщение, операция отменяется
Каскадное удаление истории	FOREIGN KEY … ON DELETE CASCADE	История удаляется вместе с заявкой
Обязательность полей	NOT NULL, DataAnnotations	Подсветка полей, сообщение
Диапазоны значений	DataAnnotations Range	Сообщение о допустимом диапазоне

Проверки в приложении выполняются до вызова удаления, чтобы пользователь получил понятное сообщение, а не техническую ошибку. Например, при удалении автомобиля форма сначала вызывает HasRequestsForVehicleAsync, и только при отрицательном результате выполняет удаление. Если запись всё же будет удалена другим

пользователем, исключение DbUpdateException перехватывается и выводится в общем обработчике ошибок.

Дополнительно реализована защита от «осиротевших» записей — при обновлении заявки навигационные свойства отсоединяются, а история сохраняется отдельно. Это гарантирует, что изменение заявки не создаёт дубликаты клиентов, автомобилей или неисправностей. Таким образом, данные остаются согласованными на всех уровнях — база данных, репозиторий и интерфейс.

3. МАТЕМАТИЧЕСКОЕ МОДЕЛИРОВАНИЕ И СТАТИСТИКА

3.1. Постановка задачи статистики

Время ремонта автомобиля рассматривается как непрерывная случайная величина T, измеряемая в часах. Для каждой завершённой заявки известны момент начала работ StartedAt и момент завершения CompletedAt; их разность даёт наблюдаемое значение t_i. Совокупность таких значений образует выборку объёмом n. В демонстрационной базе n = 34, что достаточно для оценки среднего, медианы, дисперсии и построения доверительного интервала, хотя для промышленной эксплуатации желательно накапливать несколько сотен наблюдений.

Цель статистического анализа — получить обоснованные ответы на практические вопросы — сколько в среднем длится ремонт, какая доля заявок завершается за 2, 4 или 8 часов, насколько велик разброс и можно ли планировать загрузку поста, исходя из экспоненциальной модели. Формулы, используемые в работе, приведены в таблице 3.1.

Таблица 3.1 — Формулы статистического анализа

Показатель	Формула	Смысл
Среднее арифметическое	$\bar{x} = (1/n) \cdot \Sigma x_i$	Оценка математического ожидания
Медиана	$Me = x_{(n/2)}$ при чётном n — полусумма двух центральных	Устойчива к выбросам
Мода	Середина интервала с наибольшей частотой	Наиболее типичная длительность
Дисперсия	$s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1)$	Несмещённая оценка рассеяния
Стандартное отклонение	$s = \sqrt{s^2}$	Разброс в часах
Коэффициент вариации	$CV = s / \bar{x}$	Относительный разброс
Параметр экспоненциальной модели	$\lambda = 1 / \bar{x}$	Интенсивность завершения ремонта
Функция распределения	$F(t) = 1 - e^{(-\lambda t)}$	Вероятность завершить за t часов
Вероятность превышения	$P(T > t) = e^{(-\lambda t)}$	Вероятность, что ремонт затянется
Медиана экспоненциальной модели	$Me = \ln 2 / \lambda$	Проверка согласия модели
90-й процентиль	$t_{0,9} = -\ln(0,1) / \lambda$	Время, за которое завершится 90 % ремонтов

Показатель	Формула	Смысл
Доверительный интервал	$\bar{x} \pm 1{,}96 \cdot s / \sqrt{n}$	95 % интервал для среднего

Экспоненциальная модель выбрана потому, что она широко применяется в теории массового обслуживания для описания длительности обслуживания и обладает удобным свойством — интенсивность завершения λ постоянна. Модель не является точной копией реального процесса — время ремонта зависит от вида неисправности, квалификации механика и наличия запчастей, — однако она даёт полезную верхнеуровневую оценку и позволяет рассчитывать вероятности без сложных вычислений. Для более точного прогноза в дальнейшем можно строить отдельные модели по категориям неисправностей.

При расчёте используются только завершённые ремонты, у которых заполнены обе временные отметки. Незавершённые и отменённые заявки исключаются, поскольку для них время ремонта неизвестно. Отрицательные и нечисловые значения отбрасываются. Смещение, связанное с тем, что в выборку попадают только завершённые заявки, следует учитывать при интерпретации — она характеризует длительность успешно выполненных ремонтов.

3.2. Реализация расчёта в коде

Расчёт сосредоточен в сервисе RepairStatisticsService. Метод Calculate принимает последовательность длительностей в часах, сортирует значения, вычисляет показатели и возвращает неизменяемую запись RepairTimeStatistics. Если выборка пуста, возвращается объект Empty, а все свойства равны нулю — это позволяет форме отчёта работать без специальных проверок.

```
var data = repairHours.Where(h => !double.IsNaN(h) && !double.IsInfinity(h) && h >= 0)
   .OrderBy(h => h).ToArray();
var n = data.Length;
var mean = data.Average();
var median = n % 2 == 1
   ? data[n / 2]
   : (data[n / 2 - 1] + data[n / 2]) / 2.0;
var variance = n > 1 ? data.Sum(h => Math.Pow(h - mean, 2)) / (n - 1) : 0;
var std = Math.Sqrt(variance);
var lambda = mean > 0 ? 1 / mean : 0;
var margin = n > 1 ? 1.96 * std / Math.Sqrt(n) : 0;
```

Мода определяется как середина часового интервала, в который попало наибольшее число наблюдений. Такой подход устойчивее, чем поиск точного повторяющегося значения, поскольку длительности в часах редко совпадают до секунды. Для экспоненциальной модели дополнительно рассчитываются медиана $\ln 2 / \lambda$ и 90-й процентиль $-\ln(0{,}1) / \lambda$.

Результаты расчёта по демонстрационной базе приведены в таблице 3.2. Значения получены встроенными средствами приложения и совпадают с независимой проверкой запросами к SQLite.

Таблица 3.2 — Статистика времени ремонта

Показатель	Значение	Показатель	Значение
Объём выборки n	34	Минимум, ч	0,50
Среднее $\bar{x}$, ч	2,81	Максимум, ч	13,41
Медиана Me, ч	1,69	Коэффициент вариации CV	0,96
Мода, ч	1,5	Параметр λ, 1/ч	0,3559
Дисперсия s^2	7,29	Медиана модели, ч	1,95
Стандартное отклонение s, ч	2,70	90-й процентиль, ч	6,47
Доверительный интервал	1,90–3,72 ч	—	—

Среднее время ремонта составило 2,81 часа, медиана — 1,69 часа. Медиана заметно меньше среднего, что указывает на правостороннюю асимметрию — большинство ремонтов завершается быстро, но отдельные сложные работы (до 13,41 часа) увеличивают среднее. Коэффициент вариации 0,96 близок к единице — это характерный признак экспоненциального распределения, для которого теоретическое отношение стандартного отклонения к среднему равно 1. Следовательно, выбранная модель согласуется с данными.

Доверительный интервал 1,90–3,72 часа означает, что с вероятностью 95 % истинное среднее время ремонта находится в этом диапазоне. Ширина интервала объясняется небольшим объёмом выборки и высоким разбросом. По мере накопления данных интервал будет сужаться — например, при n = 100 и той же дисперсии он составил бы примерно 2,28–3,34 часа.

3.3. Интеграция расчёта в отчёт

Статистика интегрирована в форму отчёта. После выбора периода и нажатия кнопки «Сформировать» приложение загружает заявки, вычисляет ReportData и отображает ключевые показатели в верхней панели — общее количество заявок, число выполненных и их долю, количество активных, среднее время ремонта, средний простой, медиану, стандартное отклонение и суммарную стоимость. Ниже выводится пояснение к математической модели — $\lambda = 1/\bar{x}$; $F(t) = 1 - e^{(-\lambda t)}$.

В отдельной вкладке «Статистика» доступны вероятности завершения ремонта за 1, 2, 4, 8 и 24 часа, рассчитанные по функции распределения. Например, при $\lambda = 0{,}3559$ вероятность завершить ремонт за 2 часа составляет $1 - e^{(-0{,}7118)} \approx 0{,}509$, за 4 часа — 0,759, за 8 часов — 0,942. Вероятность того, что ремонт продлится дольше 4 часов, равна $e^{(-1{,}4236)} \approx 0{,}241$.

Таблица 3.3 — Вероятности завершения ремонта

Интервал t, ч	$F(t) = P(T \le t)$	$P(T > t)$
1	0,300	0,700
2	0,509	0,491
4	0,759	0,241
8	0,942	0,058
24	0,9998	0,0002

Эти значения помогают менеджеру отвечать клиенту — «С вероятностью около 76 % ваш автомобиль будет готов в течение четырёх часов». Разумеется, прогноз следует корректировать с учётом вида неисправности и загрузки поста, однако базовая оценка становится объективной и воспроизводимой.

Форма отчёта также показывает распределение по статусам и приоритетам, средние простои по категориям неисправностей и автомобилям, месячную динамику. Все эти показатели рассчитываются в AnalyticsService на основе реальных заявок и отображаются в таблицах, что делает отчёт удобным для руководителя. Внешний вид формы приведён на рисунке 3.1.

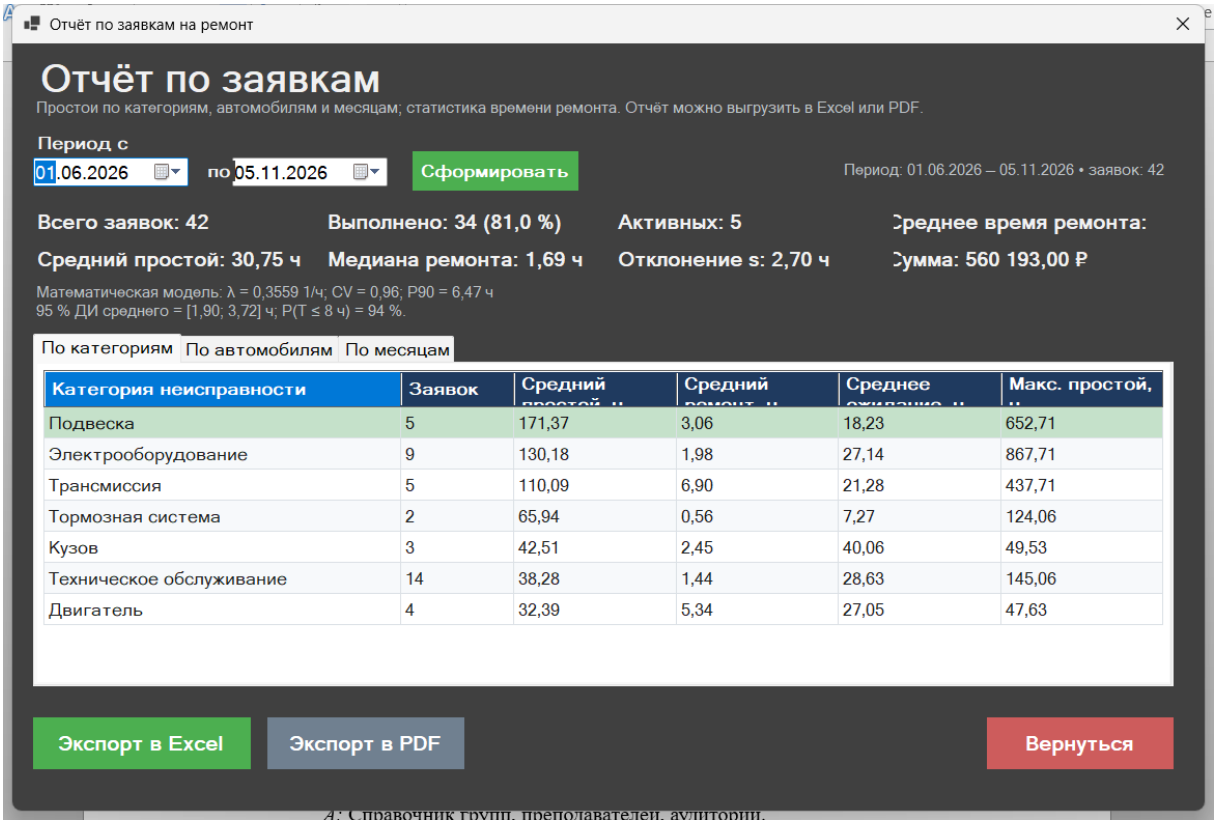


Рисунок 3.1 — Форма отчёта со статистикой

Расчёт устойчив к пустым периодам — если за выбранный интервал заявок нет, форма показывает нулевые значения и поясняет, что данных недостаточно, вместо аварийного завершения. Это проверено отдельным тестом Build_EmptyPeriod_ReturnsZeroesWithoutExceptions.

4. ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА И ВНЕШНИЕ ИНТЕГРАЦИИ

4.1. Подключение логгера Serilog

Для журналирования используется библиотека Serilog. Логгер настраивается в точке входа приложения — минимальный уровень Information, вывод в консоль и в файл с суточной ротацией. Файлы журналов хранятся в каталоге logs рядом с исполняемым файлом, сохраняются 14 дней, шаблон сообщения включает дату, время, уровень, текст и, при наличии, стек исключения.

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File(
        Path.Combine(AppPaths.LogDirectory, "app-.log"),
        rollingInterval: RollingInterval.Day,
        retainedFileCountLimit: 14,
        outputTemplate: "{Timestamp:yyyy-MM-dd HH:mm:ss} [{Level:u3}] "
                + "{Message:lj}{NewLine}{Exception}")
    .CreateLogger();
```

Уровни логирования используются следующим образом. Debug — подробная диагностика, в текущей версии не применяется; Information — запуск приложения, инициализация базы, количество пользователей; Warning — некритичные ситуации, например недоступность внешнего API; Error — необработанные ошибки интерфейса; Fatal — невозможность запуска или критическое падение. Пример реальной записи журнала

```
2026-11-05 15:39:03 [INF] Запуск приложения. База данных:
  ...\bin\Debug\net10.0-windows\data/repair-requests.db
2026-11-05 15:39:03 [INF] База данных готова. Пользователей: 6
```

Глобальные обработчики Application.ThreadException и AppDomain.UnhandledException записывают исключения в журнал и показывают пользователю понятное сообщение. Это позволяет анализировать инциденты после завершения работы приложения, не требуя установки отладчика. При необходимости уровень можно повысить через конфигурацию Serilog без пересборки программы.

Таблица 4.1 — Уровни журналирования

Уровень	Когда используется	Пример
Information	Штатные события жизненного цикла	Запуск, инициализация БД
Warning	Ожидаемая, но нештатная ситуация	Недоступен API курсов валют
Error	Ошибка, обработанная приложением	Не удалось сохранить отчёт

Уровень	Когда используется	Пример
Fatal	Критическая ошибка, работа невозможна	Не удалось открыть базу данных

4.2. Конфигурация приложения

Параметры приложения вынесены в файл appsettings.json, который копируется в каталог сборки. В текущей версии конфигурация содержит путь к базе данных; структура допускает добавление строк подключения, таймаутов и параметров внешних сервисов без изменения кода.

```
{
  "Database": {
    "Path": "data/repair-requests.db"
  }
}
```

Чтение конфигурации выполняется через ConfigurationBuilder и IConfiguration. Относительный путь преобразуется в абсолютный относительно каталога приложения; если каталог отсутствует, он создаётся. При необходимости путь можно переопределить переменной окружения или аргументом командной строки, что удобно при развёртывании на другом компьютере.

```
var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: true)
    .Build();

var relative = configuration["Database:Path"] ?? "data/repair-requests.db";
var databasePath = Path.Combine(AppContext.BaseDirectory, relative);
```

Секреты (пароли, ключи API) в файл не записываются. Для учебного проекта это приемлемо, поскольку база локальная, а демонстрационные пароли известны. В промышленной эксплуатации следует использовать переменные окружения или хранилище секретов, а файл appsettings.json исключать из репозитория при наличии конфиденциальных данных.

Таблица 4.2 — Параметры конфигурации

Параметр	Значение по умолчанию	Назначение
Database:Path	data/repair-requests.db	Путь к файлу SQLite
Logging (Serilog)	Information	Минимальный уровень журнала
RetainedFileCountLimit	14	Срок хранения журналов, дней
HttpClient.Timeout	6 с	Таймаут запроса курсов валют

4.3. Интеграция с внешним API курсов валют

Для расчёта стоимости запчастей, закупленных за рубежом, предусмотрена интеграция с API Центрального банка РФ. Используется публичный endpoint https://www.cbr-xml-daily.ru/daily_json.js, возвращающий JSON с курсами валют. Запрос выполняется асинхронно через HttpClient с таймаутом 6 секунд. Такой таймаут достаточен для получения небольших данных и не подвешивает интерфейс при отсутствии сети.

```
private static readonly HttpClient Http = new()
{
    Timeout = TimeSpan.FromSeconds(6)
};

public async Task<CurrencyRate> GetRatesAsync(CancellationToken ct = default)
{
    try
    {
        var json = await Http.GetStringAsync(Endpoint, ct);
        using var doc = JsonDocument.Parse(json);
        var usd = ReadValue(doc.RootElement, "USD");
        var eur = ReadValue(doc.RootElement, "EUR");
        return usd > 0 && eur > 0
            ? new CurrencyRate(date, usd, eur, IsFallback: false)
            : Fallback with { Date = date };
    }
    catch (Exception ex) when (ex is HttpRequestException
        or TaskCanceledException or JsonException)
    {
        return Fallback with { Error = $"Не удалось получить курс: {ex.Message}" };
    }
}
```

Обрабатываются три класса ошибок — отсутствие сети или недоступность сервера (HttpRequestException), превышение таймаута (TaskCanceledException) и некорректный JSON (JsonException). Во всех случаях возвращается резервный курс — 92,50 ₽ за доллар и 100,20 ₽ за евро с признаком IsFallback = true. Пользователь видит предупреждение о том, что использован резервный курс, и может продолжить работу.

Таблица 4.3 — Обработка ошибок внешнего API

Ситуация	Исключение	Поведение системы
Нет подключения к интернету	HttpRequestException	Резервный курс + предупреждение
Сервер не ответил за 6 с	TaskCanceledException	Резервный курс + предупреждение
Некорректный ответ	JsonException	Резервный курс + предупреждение
Курс получен	—	Актуальные значения, IsFallback = false

Пересчёт суммы в рубли выполняется методом ConvertToRub, который умножает сумму на курс и округляет результат до копеек. Метод покрыт

параметризованным тестом с различными суммами и курсами, что исключает ошибки округления. Интеграция с внешним API демонстрирует умение работать с HttpClient, JSON и обрабатывать отказы внешних сервисов, не нарушая работу основного приложения.

5. ТЕСТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

5.1. Модульные тесты

Тестирование построено на фреймворке xUnit. Тесты размещены в отдельном проекте RepairRequests.Tests, который ссылается на Core, но не зависит от WinForms. Это позволяет запускать их в любом окружении, включая сервер непрерывной интеграции. Всего реализовано 133 теста, в том числе параметризованные, которые проверяют несколько наборов входных данных в рамках одного метода.

Таблица 5.1 — Состав автоматических тестов

Класс тестов	Количество	Что проверяется
AuthServiceTests	17	Вход, регистрация, блокировка, дубликаты, пустые поля
PasswordHasherTests	15	Формат хэша, уникальность соли, проверка пароля, устойчивость к повреждённым строкам
DataValidatorTests	25	Валидация пользователей, автомобилей, заявок и неисправностей
RepairStatisticsTests	11	Среднее, медиана, мода, дисперсия, доверительный интервал, вероятности
AnalyticsServiceTests	12	Фильтрация по датам, группировки, месячный ряд, пустой период
RequestWorkflowTests	20	Допустимые и недопустимые переходы, временные метки, отмена с комментарием
CurrencyServiceTests	6	Резервный курс, округление, работа без сети
ExportTests	4	Excel из четырёх листов, PDF, пустой отчёт
RepositoriesIntegrationTests	23	Миграции, сиды, уникальность, каскады, фильтрация, транзакции
Итого	133	Все тесты проходят

Модульные тесты следуют схеме Arrange–Act–Assert. Для изоляции зависимостей используются простые тестовые двойники — например, AuthServiceTests применяет встроенную реализацию IUserRepository на списке в памяти, а RepairStatisticsTests работает с чистыми функциями без базы данных. Это делает тесты быстрыми и детерминированными.

```
[Fact]
public async Task Login_Fails_ForBlockedAccount()
{
    // Arrange
    var hasher = new PasswordHasher();
    var repo = new InMemoryUserRepository();
    repo.Seed(new User
    {
        Email = "blocked@remont.ru",
```

```
        PasswordHash = hasher.Hash("Client123!"),
        IsBlocked = true
    });
    var service = new AuthService(repo, hasher);

    // Act
    var result = await service.LoginAsync("blocked@remont.ru", "Client123!");

    // Assert
    Assert.False(result.Success);
    Assert.Contains("заблокирована", result.Error);
}
```

Параметризованные тесты применяются для проверки валидации — например, Vehicle_WithValidPlate_Passes принимает несколько корректных номеров, а Vehicle_WithInvalidPlate_Fails — несколько некорректных. Это позволяет одним методом покрыть кириллические и латинские буквы, разные длины региона и типичные ошибки ввода.

Результат запуска — 133 из 133 тестов пройдены, общее время выполнения — около 16 секунд. Сборка решения завершается без ошибок и предупреждений. Отчёт о тестировании формируется командой dotnet test --logger "trx;LogFileName=results.trx" и может быть приложен к материалам защиты.

5.2. Интеграционные тесты

Интеграционные тесты проверяют взаимодействие слоёв — EF Core, SQLite, репозитории и сервисы. Для каждого теста создаётся временная база данных в каталоге Temp, применяются миграции и заполняются данные. По завершении файл базы удаляется, поэтому тесты не влияют друг на друга и не требуют внешнего сервера.

```
await using var db = fixture.CreateContext();
await db.Database.MigrateAsync();
await DbSeeder.SeedAsync(db, new PasswordHasher());

var repository = new RepairRequestRepository(fixture.Factory);
var page = await repository.GetPagedAsync(new RequestFilter
{
    ClientId = clientId,
    Status = RequestStatus.Closed,
    PageSize = 10
});

Assert.Equal(10, page.Items.Count);
Assert.True(page.TotalCount >= 10);
```

Интеграционные тесты подтверждают, что миграция создаёт корректную схему, демонстрационные данные идемпотентны, уникальные

индексы работают, а внешние ключи запрещают удаление связанных записей. Отдельно проверяется каскадное удаление истории при удалении заявки и сохранение статуса вместе с записью истории.

Тест WorkflowUpdate_PersistsStatusAndHistory проходит полный путь заявки — создание, принятие, начало работ, завершение, закрытие. После каждого шага проверяются статус, временные отметки и количество записей истории. Это самый ценный интеграционный тест, поскольку он охватывает главный бизнес-процесс системы.

Тесты экспорта проверяют, что Excel-книга содержит четыре листа («Сводка», «Заявки», «Простои», «Статистика»), а PDF-файл начинается с сигнатуры %PDF- и завершается маркером %%EOF. Отдельный тест подтверждает, что экспорт пустого отчёта не вызывает исключения и создаёт корректные файлы.

5.3. Тест-план

Тест-план охватывает роли, фильтрацию, CRUD-операции и обработку ошибок. Полная версия с результатами приведена в приложении Г; основные сценарии перечислены в таблице 5.2.

Таблица 5.2 — Основные тест-кейсы

ID	Сценарий	Ожидаемый результат	Статус
TC-01	Вход с корректными данными администратора	Открывается главное меню	Пройден
TC-02	Вход с неверным паролем	Сообщение об ошибке, вход не выполнен	Пройден
TC-03	Вход заблокированного пользователя	Сообщение о блокировке	Пройден
TC-04	Регистрация с существующей почтой	Сообщение о дубликате	Пройден
TC-05	Регистрация с коротким паролем	Сообщение о длине пароля	Пройден
TC-06	Клиент открывает «Мои заявки»	Видны только его 15 заявок	Пройден
TC-07	Клиент пытается удалить заявку	Кнопка скрыта, операция недоступна	Пройден
TC-08	Создание заявки с корректными данными	Заявка сохранена, номер присвоен	Пройден
TC-09	Создание заявки с описанием короче 10 символов	Сообщение валидации	Пройден

ID	Сценарий	Ожидаемый результат	Статус
TC-10	Переход «Новая» → «В работе»	Переход запрещён	Пройден
TC-11	Отмена заявки без причины	Сообщение о необходимости причины	Пройден
TC-12	Полный цикл заявки менеджером	Статусы и история корректны	Пройден
TC-13	Поиск по номеру и госномеру	Найдены нужные заявки	Пройден
TC-14	Фильтр по статусу и датам	Выборка соответствует условиям	Пройден
TC-15	Сортировка по стоимости	Порядок убывания стоимости	Пройден
TC-16	Удаление автомобиля с заявками	Операция запрещена, сообщение	Пройден
TC-17	Удаление неисправности с заявками	Операция запрещена, сообщение	Пройден
TC-18	Добавление автомобиля с занятым номером	Сообщение о дубликате	Пройден
TC-19	Отчёт за период с данными	Показатели и таблицы заполнены	Пройден
TC-20	Отчёт за пустой период	Нулевые показатели без ошибок	Пройден
TC-21	Экспорт в Excel	Файл создан, четыре листа	Пройден
TC-22	Экспорт в PDF	Файл создан, открывается	Пройден
TC-23	Отключение сети при запросе курса	Резервный курс, предупреждение	Пройден
TC-24	Выход из системы	Возврат на форму входа	Пройден

Ручное тестирование выполнялось на чистой базе и на базе с демонстрационными данными. Проверялись разрешения экрана 1366×768 и 1920×1080, масштаб Windows 100 % и 125 %, работа без сети, повторный запуск, удаление файла базы и его повторное создание. Критических дефектов не выявлено; замечания по оформлению устранены в ходе рефакторинга.

6. ОТЧЁТНОСТЬ И ВИЗУАЛИЗАЦИЯ ДАННЫХ

6.1. Отчёт «Заявки за период»

Отчёт формируется в отдельной форме, доступной из главного меню. Пользователь выбирает начальную и конечную даты и нажимает «Сформировать». Приложение загружает заявки за период, вычисляет сводные показатели и заполняет таблицы. Для менеджера отчёт строится по всем заявкам, для клиента — только по его собственным.

Структура отчёта включает следующие блоки

— Ключевые показатели — всего заявок, выполнено и доля выполнения, активные, среднее время ремонта, средний простой, медиана, стандартное отклонение, суммарная стоимость;

— Распределение по статусам — количество и доля заявок каждого статуса;

— Распределение по приоритетам — количество и доля заявок каждого приоритета;

— Простои по категориям неисправностей — количество, средний общий простой, среднее время ремонта, среднее ожидание, максимальный простой;

— Простои по автомобилям — аналогичные показатели в разрезе транспортных средств;

— Динамика по месяцам — созданные и завершённые заявки, среднее время ремонта и сумма за месяц;

— Математическая модель — параметр λ и вероятности завершения ремонта за заданное время.

Внешний вид отчёта для клиента приведён на рисунке 6.1. Клиент видит только свои заявки; заголовок и состав показателей адаптированы, а кнопки экспорта доступны в том же виде, что и у менеджера.

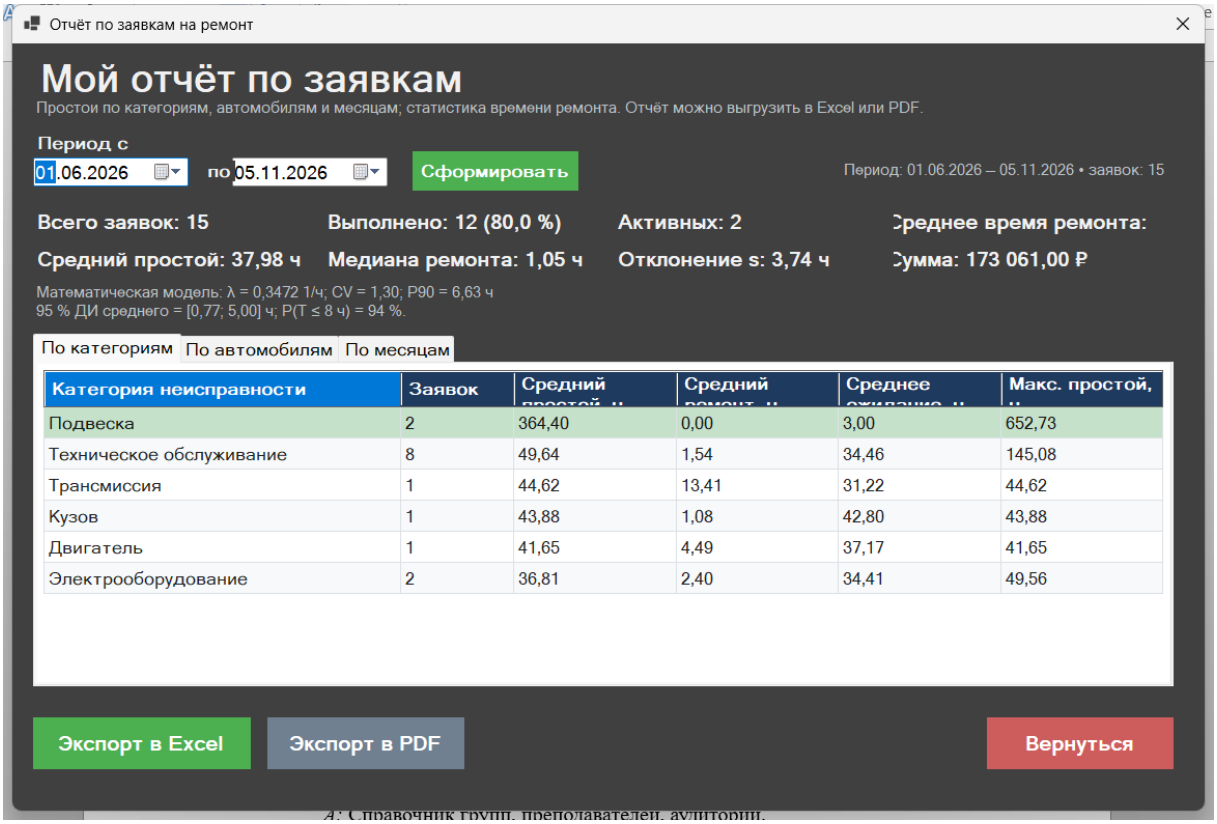


Рисунок 6.1 — Отчёт по заявкам клиента

По демонстрационным данным за период июнь–ноябрь 2026 года отчёт показывает 42 заявки, из них 34 выполнены и закрыты (81,0 %), 5 активны, 3 отменены, 3 просрочены. Суммарная стоимость составляет 560 193,00 ₽, средняя стоимость заявки — 13 337,93 ₽, средний простой — 30,75 часа, среднее время ремонта — 2,81 часа. Эти значения согласуются с данными, которые отображаются в форме, и с результатами SQL-запросов к базе.

Отчёт устойчив к граничным случаям — если период начинается и заканчивается в один день, учитываются все заявки этого дня; если данных нет, выводятся нули и пояснение. При большом количестве записей таблицы прокручиваются, а экспорт выполняется асинхронно, поэтому интерфейс не замораживается.

6.2. Экспорт отчёта в Excel и PDF

Экспорт реализован двумя независимыми сервисами. Excel-файл формируется библиотекой ClosedXML и содержит четыре листа (таблица 6.1). Каждый лист имеет заголовок, оформленные шапки, автоширину

колонок и числовые форматы. Объём файла с демонстрационными данными — около 17 КБ.

Таблица 6.1 — Структура Excel-отчёта

Лист	Содержимое
Сводка	Период, автор, ключевые показатели, распределение по статусам
Заявки	Построчный список заявок — номер, дата, автомобиль, клиент, неисправность, механик, приоритет, статус, стоимости, время ремонта
Простои	Группировка по категориям и автомобилям, средние и максимальные простои
Статистика	Описательная статистика, параметры экспоненциальной модели, вероятности, доверительный интервал

PDF-файл формируется библиотекой QuestPDF. Документ содержит титульный блок с названием, периодом и автором, затем разделы «Сводные показатели», «Распределение заявок», «Математическая модель времени ремонта», «Простои по категориям», «Динамика по месяцам» и таблицу заявок. Используется книжная ориентация А4, шрифты подключаются из системы, поэтому кириллица отображается корректно. Размер файла с демонстрационными данными — около 140 КБ.

Таблица 6.2 — Структура PDF-отчёта

Раздел PDF	Содержимое
Титульный блок	Название отчёта, период, дата формирования, автор
1. Сводные показатели	Таблица из 12 показателей в две колонки
2. Распределение заявок	Две таблицы — по статусам и по приоритетам
3. Математическая модель	Формулы, значения, вероятности, доверительный интервал
4. Простои и динамика	Группировки по категориям, автомобилям и месяцам
5. Реестр заявок	Таблица заявок с переносом на новые страницы

Оба сервиса принимают объект ReportData, поэтому формат вывода не зависит от формы и может использоваться в других сценариях. Экспорт покрыт тестами — проверяется количество листов Excel, наличие строк заявок, сигнатура PDF и корректность пустого отчёта. Пользователю предлагается выбрать путь сохранения, после успешной записи выводится предложение открыть файл.

Экспорт в Excel удобен для дальнейшей обработки — руководитель может строить сводные таблицы и диаграммы средствами Excel. PDF предназначен для печати и отправки заказчику — он сохраняет вёрстку и не требует установки офисного пакета.

6.3. Графики статистики

Для визуализации данных построены четыре диаграммы. Они создаются отдельным инструментом на основе реальной базы и вставляются в отчёт и презентацию. Графики позволяют быстро оценить структуру заявок, динамику, проблемные категории и распределение времени ремонта.

Распределение по статусам (рисунок 6.2) показывает, что большинство заявок — 30 из 42 — закрыты, 4 выполнены, 2 новые, 1 принята, 2 в работе, 3 отменены. Высокая доля закрытых заявок свидетельствует о том, что процесс доводится до конца, а небольшое число активных заявок соответствует текущей загрузке.

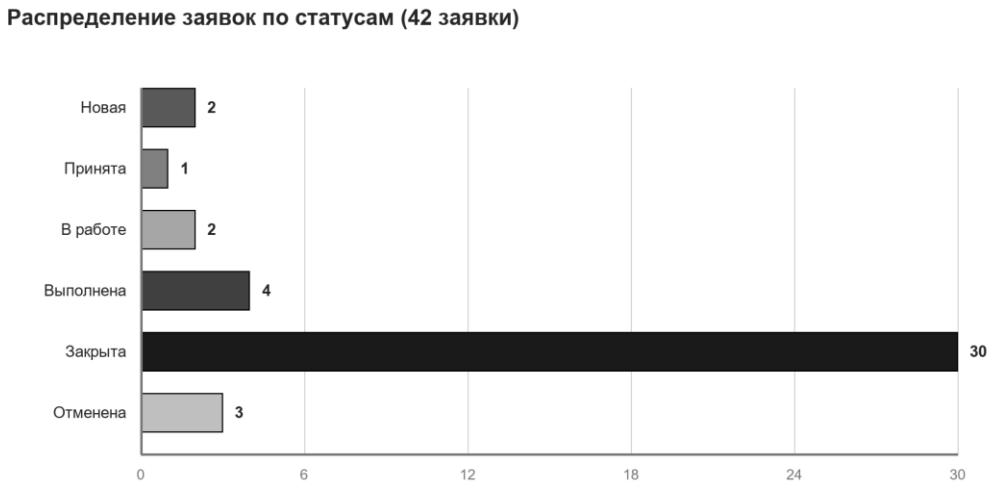


Рисунок 6.2 — Распределение заявок по статусам

Динамика по месяцам (рисунок 6.3) отражает количество созданных и завершённых заявок с июня по ноябрь 2026 года. Пик создания приходится на октябрь (9 заявок), завершения — на июль и сентябрь (по 8 заявок). Небольшое расхождение между рядами объясняется тем, что часть заявок октября и ноября ещё находится в работе.

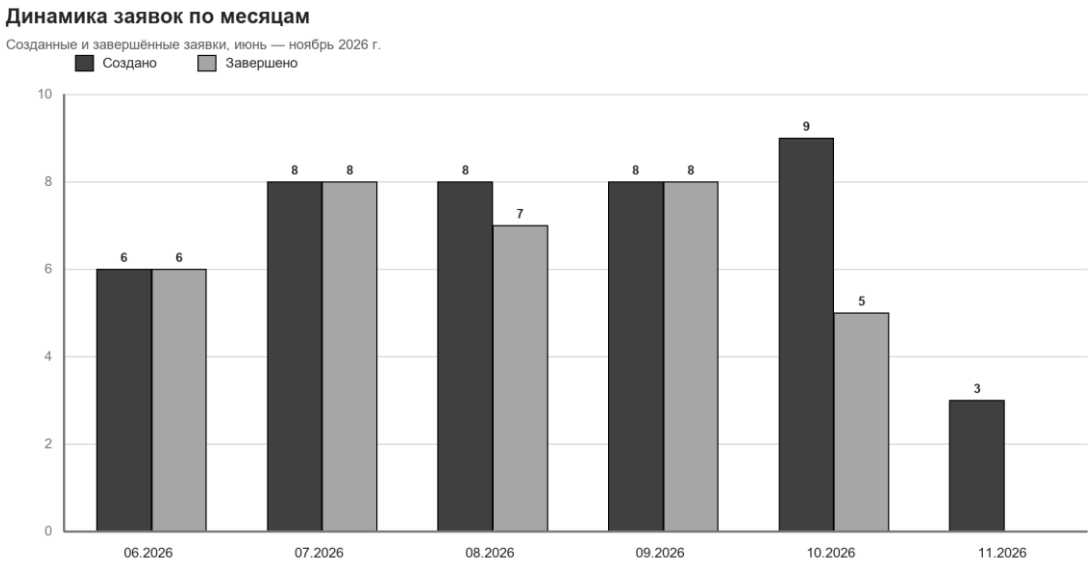


Рисунок 6.3 — Динамика заявок по месяцам

Средний простой по категориям неисправностей (рисунок 6.4) показывает, что наибольший простой дают кузовные работы (42,51 часа), затем подвеска (36,52) и двигатель (32,39). Это ожидаемо — кузовной ремонт и замена стекла требуют заказа деталей и сушки герметика. Наименьший простой — у тормозной системы (7,83 часа), поскольку работа выполняется быстро и не требует длительного ожидания запчастей.

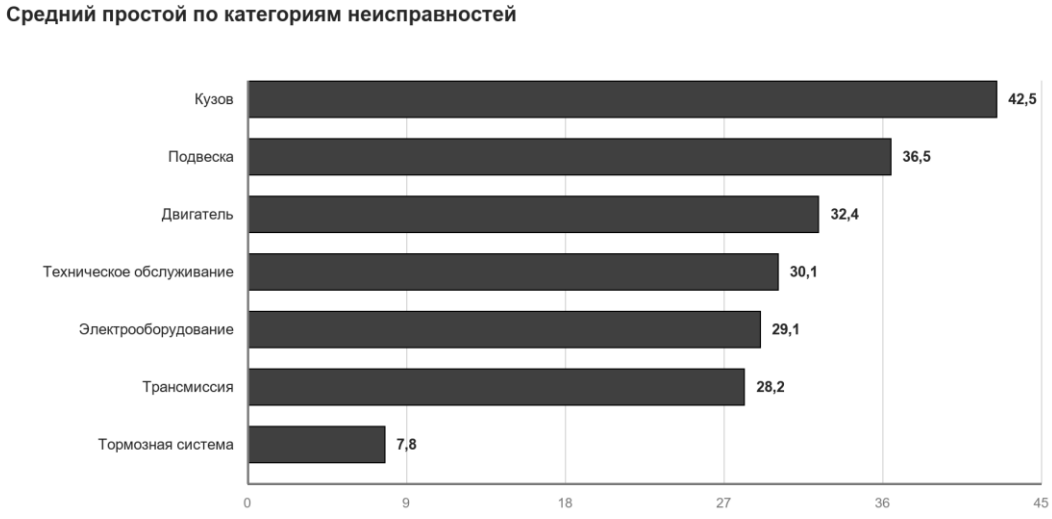


Рисунок 6.4 — Средний простой по категориям неисправностей

Гистограмма длительности ремонта (рисунок 6.5) показывает распределение 34 завершённых ремонтов по интервалам. Наибольшая частота приходится на интервал 1–2 часа (11 ремонтов), затем 0–1 час (8) и

2–4 часа (6). Длинные ремонты (свыше 9 часов) единичны. Форма распределения с длинным правым хвостом согласуется с экспоненциальной моделью и объясняет превышение среднего над медианой.

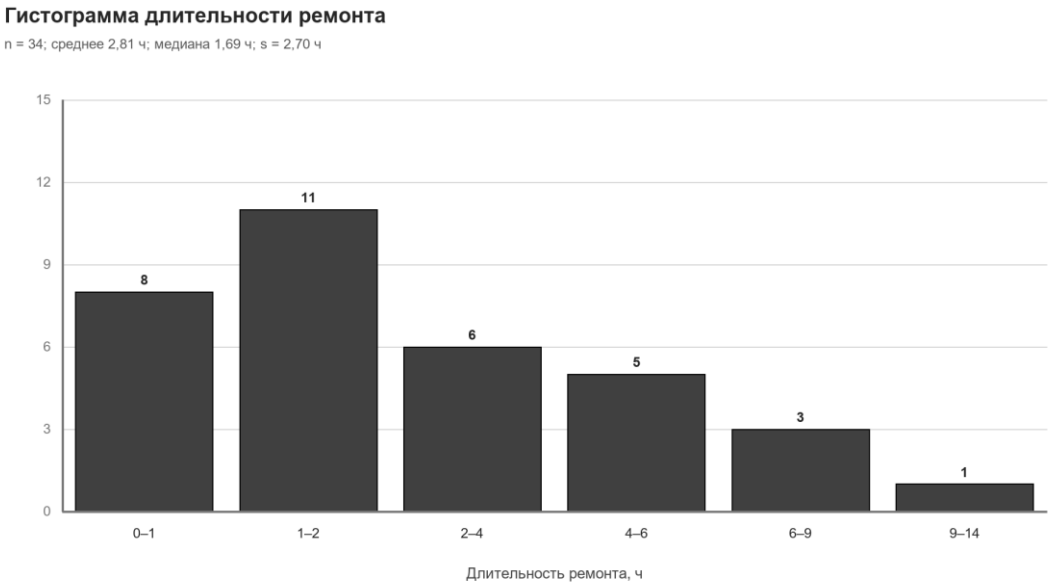


Рисунок 6.5 — Гистограмма длительности ремонта

Графики построены с учётом требований к печати — подписи осей и значений читаемы, цвета различимы в чёрно-белом варианте, единицы измерения указаны. Данные для графиков берутся из тех же запросов, что и для отчёта, поэтому расхождений между таблицами и диаграммами нет.

7. ДОКУМЕНТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ

7.1. Руководство пользователя

Руководство пользователя приведено в приложении Е и включает описание интерфейса, порядок входа и регистрации, работу со справочниками, создание и обработку заявок, формирование отчёта и экспорт, а также действия в аварийных ситуациях. Ниже приведены основные сведения.

Вход в систему. Запустите файл RepairRequests.WinForms.exe. Введите электронную почту и пароль, нажмите «Войти». Демонстрационные учётные записи — admin@remont.ru / Admin123!, manager@remont.ru / Manager123!, client@remont.ru / Client123!. При забытом пароле обратитесь к администратору — самостоятельное восстановление в текущей версии не предусмотрено.

Создание заявки. Откройте «Мои заявки» → «Добавить». Выберите автомобиль и неисправность, укажите приоритет, плановую дату и описание, нажмите «Сохранить». Номер заявки присваивается автоматически. Если автомобиля нет в списке, сначала добавьте его в разделе «Мой транспорт».

Обработка заявки. Менеджер открывает заявку двойным щелчком и выполняет доступные действия — «Принять заявку», «Начать работы», «Завершить ремонт», «Закрыть заявку». Для отмены необходимо указать причину. Все действия фиксируются в истории.

Отчёт. Выберите период и нажмите «Сформировать». Для сохранения используйте «Экспорт в Excel» или «Экспорт в PDF». Файл можно открыть сразу после сохранения.

Аварийные ситуации. Если приложение не запускается, проверьте наличие .NET Desktop Runtime 10 и права на запись в каталог программы. Если база повреждена, закройте приложение, восстановите файл из резервной копии или удалите файл data/repair-requests.db — при следующем запуске база будет создана заново с демонстрационными данными. Перед удалением обязательно сделайте копию.

Полное руководство содержит скриншоты, пошаговые инструкции и рекомендации по работе; оно приведено в приложении Е и передаётся пользователям вместе с программой.

7.2. Описание программы по ГОСТ 19.402-78

Документ «Описание программы» оформлен по ГОСТ 19.402-78 и включает сведения о назначении, условиях выполнения, требованиях к программному обеспечению, составе функций и обращении с программой. Ниже приведены его ключевые положения; полный текст вынесен в отдельный документ.

Назначение. Программа предназначена для учёта заявок на ремонт транспорта, ведения справочников, контроля статусов и формирования отчётности. Область применения — станции технического обслуживания с числом рабочих постов до десяти.

Условия выполнения. Программа работает под управлением Windows 10/11 x64, требует .NET Desktop Runtime 10, не менее 4 ГБ оперативной памяти, 200 МБ свободного дискового пространства и разрешения экрана не ниже 1366×768. Для получения курсов валют желателен доступ в интернет; при его отсутствии используется резервный курс.

Требования к программному обеспечению. Операционная система Windows 10/11, .NET Desktop Runtime 10, библиотека SQLite (поставляется вместе с приложением), офисный пакет — только для просмотра экспортированных файлов.

Состав функций. Вход и регистрация; разграничение ролей; ведение справочников транспорта и неисправностей; создание, редактирование и обработка заявок; история статусов; фильтрация, сортировка и пагинация; статистика и вероятностная модель; отчёт и экспорт в Excel/PDF; журналирование; интеграция с API курсов валют.

Обращение с программой. Запуск — двойным щелчком по исполняемому файлу. Вход — по учётной записи. Завершение — кнопкой

«Выход» на главном меню и затем «Выход» на форме входа. Резервное копирование — копированием файла базы при закрытом приложении.

7.3. Оформление листингов

В отчёте приведены ключевые фрагменты кода с комментариями — хэширование паролей, проверка допустимых переходов, генерация номера заявки, выборка с фильтрацией, расчёт статистики и экспорт. Полные листинги находятся в репозитории; ниже приведён ещё один фрагмент — расчёт доверительного интервала и вероятностей.

```
var margin = n > 1 ? 1.96 * std / Math.Sqrt(n) : 0;
var lambda = mean > 0 ? 1 / mean : 0;
var p90 = lambda > 0 ? -Math.Log(0.1) / lambda : 0;

return new RepairTimeStatistics(
    Count: n,
    MeanHours: Round(mean),
    MedianHours: Round(median),
    StandardDeviation: Round(std),
    LambdaPerHour: Round(lambda, 4),
    P90Hours: Round(p90),
    ConfidenceLowHours: Round(Math.Max(0, mean - margin)),
    ConfidenceHighHours: Round(mean + margin));
```

Комментарии в коде поясняют назначение методов и бизнес-правил, но не дублируют очевидные операции. Имена классов, методов и переменных соответствуют предметной области и соглашению PascalCase/camelCase, принятому в C#. Для публичных членов используется XML-документация, что позволяет IDE показывать подсказки.

7.4. Сборка релиза

Релиз собирается в конфигурации Release. Используется framework-dependent публикация под win-x64 — она требует установленного .NET Desktop Runtime, зато занимает меньше места. Команда публикации

```
dotnet publish src/RepairRequests.WinForms/RepairRequests.WinForms.csproj \
  -c Release -r win-x64 --self-contained false -o publish/win-x64
```

Результат — 55 файлов общим объёмом около 33,9 МБ. В каталоге публикации находятся исполняемый файл, библиотеки Core и сторонних пакетов, appsettings.json, ресурсы интерфейса и нативная библиотека SQLite. Каталоги data, logs и reports создаются при первом запуске.

Для компьютера без установленного .NET можно собрать self-contained версию, добавив параметр --self-contained true; её размер возрастает

примерно до 150 МБ. Для удобства распространения можно использовать PublishSingleFile=true, однако нативные библиотеки SQLite и QuestPDF при этом всё равно извлекаются во временный каталог.

Таблица 7.1 — Команды сборки и публикации

Команда	Назначение	Результат
dotnet build -c Release	Сборка без публикации	bin/Release
dotnet publish -c Release -r win-x64	Публикация framework-dependent	publish/win-x64
dotnet publish --self-contained true	Публикация с runtime	около 150 МБ
dotnet test	Запуск тестов	133 теста

Перед публикацией рекомендуется очистить решение (dotnet clean), убедиться в отсутствии предупреждений, прогнать тесты и проверить запуск на чистой базе. Версия приложения задаётся в файле проекта; для учебного проекта достаточно номера 1.0.0.

7.5. Инструкция по развёртыванию

Развёртывание выполняется в несколько шагов. Полная инструкция с иллюстрациями приведена в приложении Д.

1. Установите .NET Desktop Runtime 10 с официального сайта Microsoft.

2. Скопируйте каталог publish/win-x64 на целевой компьютер, например в C:\Program Files\RepairRequests.

3. Убедитесь, что у пользователя есть права на запись в каталог программы (для базы, журналов и отчётов).

4. При необходимости отредактируйте appsettings.json, указав другой путь к базе.

5. Запустите RepairRequests.WinForms.exe. При первом запуске база будет создана и заполнена демонстрационными данными.

6. Войдите под учётной записью администратора и проверьте работу разделов.

7. Настройте резервное копирование — ежедневно копируйте файл data/repair-requests.db в отдельный каталог или на сетевой диск.

Обновление. Закройте приложение, сохраните резервную копию базы, замените файлы программы новыми, запустите приложение. Миграции

применяются автоматически. Если новая версия содержит несовместимые изменения схемы, предварительно изучите примечания к выпуску.

Типичные ошибки. «Не удалось запустить приложение» — не установлен .NET Desktop Runtime; «База данных недоступна» — нет прав на запись или файл заблокирован другим процессом; «Неверный формат госномера» — проверьте соответствие шаблону; «Не удалось получить курс» — отсутствует сеть, используется резервный курс. Все ошибки записываются в журнал logs/app-ГГГГММДД.log.

Безопасность. Не передавайте файл базы посторонним — он содержит персональные данные клиентов. Ограничьте доступ к каталогу программы средствами NTFS. Пароли хранятся в виде хэшей, но демонстрационные учётные записи следует отключить или сменить пароли перед промышленной эксплуатацией.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики разработана информационная система учёта заявок на ремонт транспорта, охватывающая полный цикл обращения клиента — от регистрации и создания заявки до выполнения работ, закрытия и формирования отчётности. Система реализована на платформе C# WinForms с использованием .NET 10, Entity Framework Core и SQLite.

Все поставленные задачи выполнены

— изучена предметная область, описаны участники, процессы и требования;

— составлено техническое задание по ГОСТ 19.201-78;

— построены структурная схема, UML-диаграммы вариантов использования и классов, ER-диаграмма; схема приведена к третьей нормальной форме;

— настроены среда разработки, Git-репозиторий и трёхпроектное решение;

— создана база данных SQLite, выполнена миграция, подготовлены демонстрационные данные;

— реализованы авторизация с PBKDF2, разграничение ролей и CRUD-формы справочников;

— разработан модуль заявок с жизненным циклом, историей, фильтрацией, сортировкой и пагинацией;

— реализованы статистические расчёты и экспоненциальная модель времени ремонта;

— подключены Serilog, конфигурация и внешний API курсов валют;

— написаны 133 автоматических теста, все проходят;

— реализованы отчёт, экспорт в Excel/PDF и визуализация данных;

— подготовлены руководство пользователя, описание по ГОСТ 19.402-78, инструкция по развёртыванию и презентация.

Количественные результаты подтверждают работоспособность системы — в демонстрационной базе 42 заявки, 34 завершённых ремонта, среднее время ремонта 2,81 часа, медиана 1,69 часа, стандартное отклонение 2,70 часа, доверительный интервал 1,90–3,72 часа. Отчёт формируется за доли секунды, Excel-файл содержит четыре листа, PDF-файл корректно

открывается и имеет объём около 140 КБ. Все 133 теста проходят, сборка не содержит предупреждений.

Практическая значимость работы состоит в том, что система готова к использованию на небольшой станции технического обслуживания — она не требует сервера, поддерживает резервное копирование, разграничивает доступ и даёт руководителю объективную картину загрузки и сроков ремонта. Реализованная вероятностная модель позволяет обоснованно отвечать клиентам на вопрос о сроках выполнения работ.

Направления дальнейшего развития — интерфейс управления пользователями для администратора, уведомления клиентов по электронной почте или SMS, отдельные статистические модели по категориям неисправностей, прогнозирование загрузки постов, интеграция с бухгалтерскими системами и переход на серверную СУБД (PostgreSQL) при росте числа рабочих мест. Эти улучшения не меняют архитектуру и могут быть реализованы поэтапно.

Таким образом, цель работы достигнута, задачи выполнены в полном объёме, а полученные навыки — проектирование баз данных, разработка настольных приложений, работа с ORM, тестирование, документирование и развёртывание — имеют непосредственное практическое значение.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М. Стандартинформ, 2010. — 5 с.

2. ГОСТ 19.402-78. Единая система программной документации. Описание программы. — М. Стандартинформ, 2010. — 4 с.

3. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. — М. Стандартинформ, 2009.

4. ГОСТ Р 7.0.5-2008. Библиографическая ссылка. Общие требования и правила составления. — М. Стандартинформ, 2008.

5. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. — 8-е изд. — М. Вильямс, 2006. — 1328 с.

6. Мартин, Р. Чистый код — создание, анализ и рефакторинг / Р. Мартин. — СПб. Питер, 2019. — 464 с.

7. Фаулер, М. Рефакторинг — улучшение существующего кода / М. Фаулер. — 2-е изд. — СПб. Питер, 2020. — 448 с.

8. Троелсен, Э. Язык программирования C# 10 и платформа .NET 6 / Э. Троелсен, Ф. Джепикс. — 11-е изд. — М. Диалектика, 2023. — 1264 с.

9. Вентцель, Е. С. Теория вероятностей / Е. С. Вентцель. — 12-е изд. — М. Юстиция, 2018. — 658 с.

10. Кремер, Н. Ш. Теория вероятностей и математическая статистика / Н. Ш. Кремер. — 3-е изд. — М. ЮНИТИ-ДАНА, 2016. — 551 с.

11. Microsoft .NET documentation. — URL https://learn.microsoft.com/dotnet/ (дата обращения 05.11.2026).

12. Entity Framework Core documentation. — URL https://learn.microsoft.com/ef/core/ (дата обращения 05.11.2026).

13. SQLite Documentation. — URL https://www.sqlite.org/docs.html (дата обращения 05.11.2026).

14. Windows Forms documentation. — URL https://learn.microsoft.com/dotnet/desktop/winforms/ (дата обращения 05.11.2026).
15. Serilog documentation. — URL https://serilog.net/ (дата обращения 05.11.2026).
16. ClosedXML documentation. — URL https://github.com/ClosedXML/ClosedXML (дата обращения 05.11.2026).
17. QuestPDF documentation. — URL https://www.questpdf.com/ (дата обращения 05.11.2026).
18. xUnit.net documentation. — URL https://xunit.net/ (дата обращения 05.11.2026).
19. Pro Git book / S. Chacon, B. Straub. — URL https://git-scm.com/book/ru/v2 (дата обращения 05.11.2026).
20. draw.io documentation. — URL https://www.drawio.com/doc/ (дата обращения 05.11.2026).
21. OWASP Password Storage Cheat Sheet. — URL https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения 05.11.2026).
22. NIST Special Publication 800-63B. Digital Identity Guidelines. — URL https://pages.nist.gov/800-63-3/sp800-63b.html (дата обращения 05.11.2026).

ПРИЛОЖЕНИЕ А

Техническое задание на разработку

информационной системы учёта заявок на ремонт транспорта

1. Введение

1.1. Наименование разработки — информационная система учёта заявок на ремонт транспорта «АвтоМастер» (далее — Система).

1.2. Разработка выполняется в рамках учебной практики по специальности 09.02.07 «Информационные системы и программирование».

1.3. Основание для разработки — результаты анализа текущей деятельности СТО «АвтоМастер», выполненного в ходе учебной практики, и задание на учебную практику, утверждённое руководителем практики.

1.4. Разработка выполнена студентом-практикантом. В ходе анализа текущей деятельности СТО «АвтоМастер» рассмотрены приём и учёт заявок, ведение бумажного журнала и файлов Excel, выявлены потери заявок, повторный ввод данных об автомобилях и неисправностях, отсутствие истории статусов и ручное формирование отчётности. На основе анализа сформулированы требования к системе.

1.5. Сроки выполнения определяются календарным планом учебной практики.

2. Назначение и цели создания системы

2.1. Система предназначена для автоматизации приёма, обработки и учёта заявок на ремонт транспортных средств, ведения справочников, контроля статусов и формирования отчётности.

2.2. Цели создания Системы

— повышение оперативности обработки заявок и исключение их потери;

— обеспечение прозрачности статусов заявок для клиента, менеджера и руководства;

— исключение повторного ввода данных об автомобилях и неисправностях;

— автоматизация расчёта статистики времени ремонта и простоев;

— сокращение времени подготовки отчётов с нескольких часов до нескольких секунд;

— повышение достоверности данных за счёт валидации и ссылочной целостности.

2.3. Система относится к классу информационно-учётных систем и не предназначена для управления технологическим оборудованием.

3. Требования к системе

3.1. Требования к системе в целом. Система должна быть реализована как настольное приложение для Windows 10/11 x64, работать без выделенного сервера, использовать локальную базу данных SQLite и поддерживать одновременную работу одного пользователя на одном рабочем месте.

3.2. Требования к функциям. Перечень функций приведён в таблице А.1.

Таблица А.1 — Функции системы

Код	Функция	Описание
Ф-01	Вход в систему	Аутентификация по электронной почте и паролю, проверка блокировки, обновление даты последнего входа
Ф-02	Регистрация клиента	Создание учётной записи с ролью «Клиент», проверка уникальности почты и сложности пароля
Ф-03	Разграничение прав	Роли «Клиент», «Менеджер», «Администратор»; клиент видит только свои данные
Ф-04	Справочник транспорта	Просмотр, поиск, добавление, изменение, удаление автомобилей с контролем связанных заявок
Ф-05	Справочник неисправностей	Ведение видов работ с категорией, нормативной трудоёмкостью, базовой стоимостью и признаком активности
Ф-06	Создание заявки	Выбор автомобиля и неисправности, указание приоритета, плановой даты и описания, автоматическая генерация номера
Ф-07	Жизненный цикл заявки	Шесть статусов, допустимые переходы, временные отметки, история изменений с автором и комментарием
Ф-08	Фильтрация и поиск	Поиск по тексту, фильтры по статусу, приоритету, автомобилю, неисправности, клиенту, диапазону дат
Ф-09	Сортировка и пагинация	Сортировка по дате, номеру, приоритету, статусу, стоимости; постраничный вывод
Ф-10	Статистика времени ремонта	Среднее, медиана, мода, дисперсия, стандартное отклонение, коэффициент вариации, доверительный интервал
Ф-11	Математическая модель	Экспоненциальное распределение, параметр λ, вероятности завершения за заданное время, 90-й процентиль

Код	Функция	Описание
Ф-12	Отчёт за период	Ключевые показатели, распределение по статусам и приоритетам, простои по категориям и автомобилям, динамика по месяцам
Ф-13	Экспорт отчёта	Excel из четырёх листов и PDF с разделами и таблицами
Ф-14	Журналирование	Запись запуска, инициализации БД и необработанных ошибок в файл с суточной ротацией
Ф-15	Внешний API курсов валют	Получение курсов USD/EUR с таймаутом и резервным значением при отказе

3.3. Требования к видам обеспечения.

3.3.1. Математическое обеспечение — формулы описательной статистики, экспоненциальная модель распределения, доверительный интервал для среднего.

3.3.2. Информационное обеспечение — база данных SQLite из пяти таблиц (Users, Vehicles, FaultTypes, RepairRequests, RequestStatusHistory), нормативно-справочная информация, демонстрационные данные.

3.3.3. Программное обеспечение — .NET 10, Windows Forms, Entity Framework Core 10, ClosedXML, QuestPDF, Serilog, xUnit.

3.3.4. Техническое обеспечение — персональный компьютер с процессором не ниже двухъядерного, 4 ГБ оперативной памяти, 200 МБ свободного дискового пространства, монитор с разрешением не ниже 1366×768.

4. Требования к надёжности и безопасности

4.1. Система должна корректно обрабатывать пустые выборки, отсутствие сети, недоступность внешнего API и ошибки ввода, не завершаясь аварийно.

4.2. Пароли должны храниться только в виде хэша PBKDF2-HMAC-SHA256 с числом итераций не менее 120 000 и уникальной солью.

4.3. Должна быть обеспечена ссылочная целостность — запрет удаления справочных записей, на которые ссылаются заявки; каскадное удаление истории статусов.

4.4. Должно быть предусмотрено резервное копирование файла базы данных.

5. Требования к эргономике и технической эстетике

5.1. Интерфейс — на русском языке, единая цветовая схема, крупные элементы управления, понятные сообщения об ошибках.

5.2. Формы должны быть адаптированы под разрешение 1366×768 и выше, поддерживать масштабирование Windows до 125 %.

5.3. Все критические действия (удаление, отмена заявки, выход) должны требовать подтверждения.

6. Стадии и этапы разработки

Таблица А.2 — Стадии разработки

Этап	Содержание	Результат
1	Анализ предметной области, разработка ТЗ	Раздел 1 отчёта, приложение А
2	Проектирование системы и базы данных	Диаграммы, приложение Б
3	Настройка среды и репозитория	Решение из трёх проектов
4	Создание базы данных и миграций	Файл repair-requests.db
5	Реализация авторизации и справочников	Формы входа, транспорта, неисправностей
6	Реализация модуля заявок	Формы заявок, история, фильтрация
7	Реализация статистики и модели	Сервис статистики, раздел 3
8	Интеграции и журналирование	Serilog, конфигурация, API курсов
9	Тестирование	133 теста, тест-план
10	Отчётность и экспорт	Форма отчёта, Excel, PDF, графики
11	Документирование и защита	Отчёт, руководства, презентация

7. Порядок контроля и приёмки

7.1. Контроль осуществляется на каждом этапе — сборка решения, проверка отсутствия предупреждений, запуск автоматических тестов, ручная проверка сценариев по тест-плану.

7.2. Приёмка выполняется путём демонстрации работы Системы на демонстрационных данных и сверки результатов с требованиями настоящего ТЗ.

7.3. Критерии приёмки — все функции Ф-01…Ф-15 реализованы, 133 теста проходят, отчёт формируется и экспортируется, интерфейс соответствует требованиям раздела 5.

ПРИЛОЖЕНИЕ Б

Диаграммы проекта

Все диаграммы выполнены в редакторе draw.io и сохранены в виде редактируемых исходных файлов формата .drawio, а также экспортированы в PNG для включения в отчёт. Исходные файлы находятся в каталоге docs/diagrams.

Таблица Б.1 — Состав диаграмм

Файл	Содержание	Нотация
01-structural.drawio	Структурная схема предприятия	Организационная схема
02-usecase.drawio	Диаграмма вариантов использования	UML Use Case
03-class.drawio	Диаграмма классов	UML Class Diagram
04-er.drawio	Модель базы данных	ER (Crow's Foot)
05-3nf.drawio	Приведение к третьей нормальной форме	Схема декомпозиции
06-architecture.drawio	Архитектура приложения	Слоистая схема

Структурная схема предприятия отражает линейно-функциональную организацию СТО «АвтоМастер» — генеральному директору подчиняются отдел приёма заявок, производственный участок, склад запчастей, бухгалтерия и администратор информационной системы.

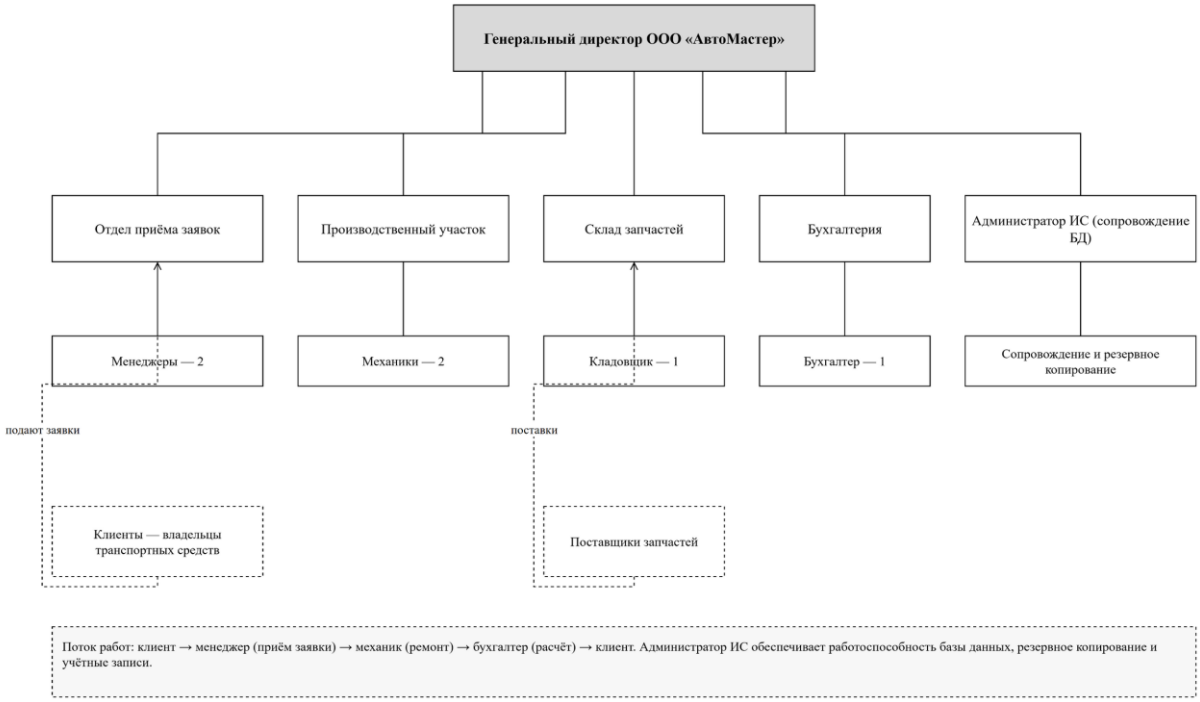


Рисунок Б.1 — Структурная схема предприятия

Диаграмма вариантов использования определяет границы системы и функции трёх акторов — клиента, менеджера и администратора.

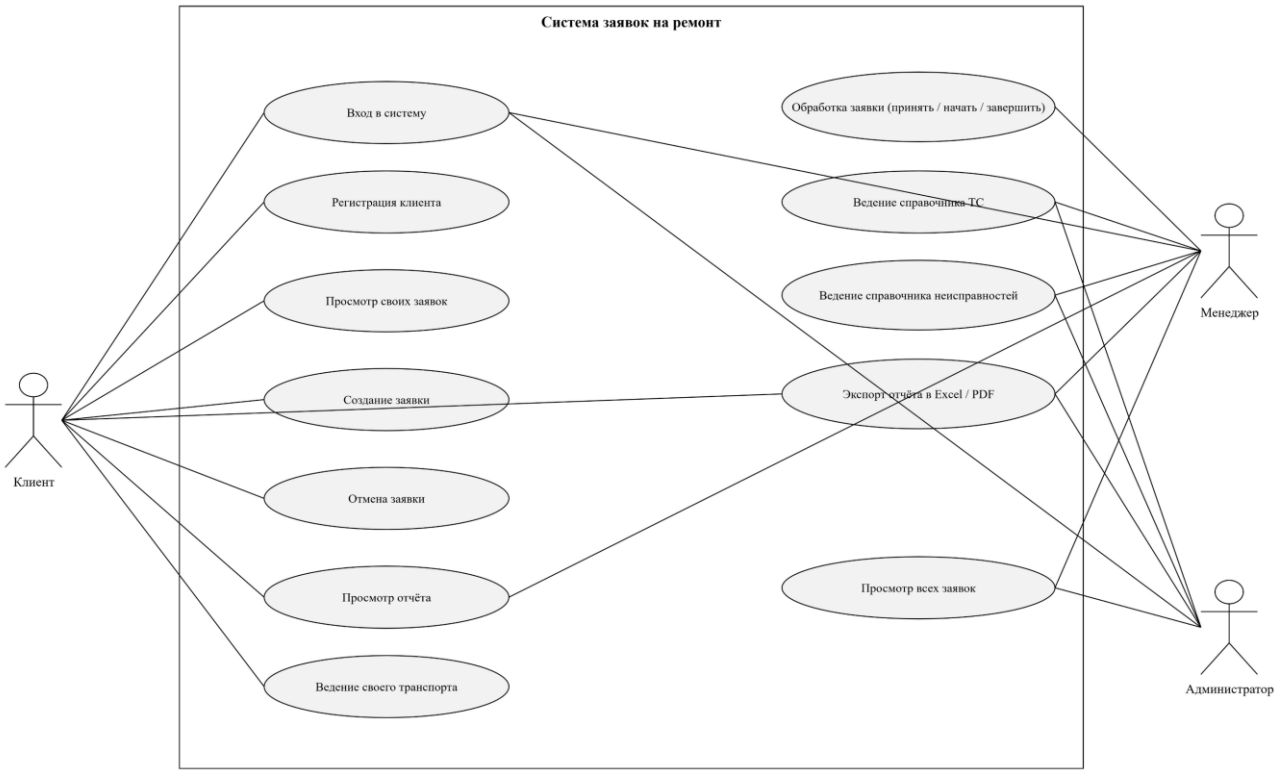


Рисунок Б.2 — UML-диаграмма вариантов использования

Диаграмма классов показывает пять основных сущностей предметной области, их атрибуты, методы и связи с кратностями.

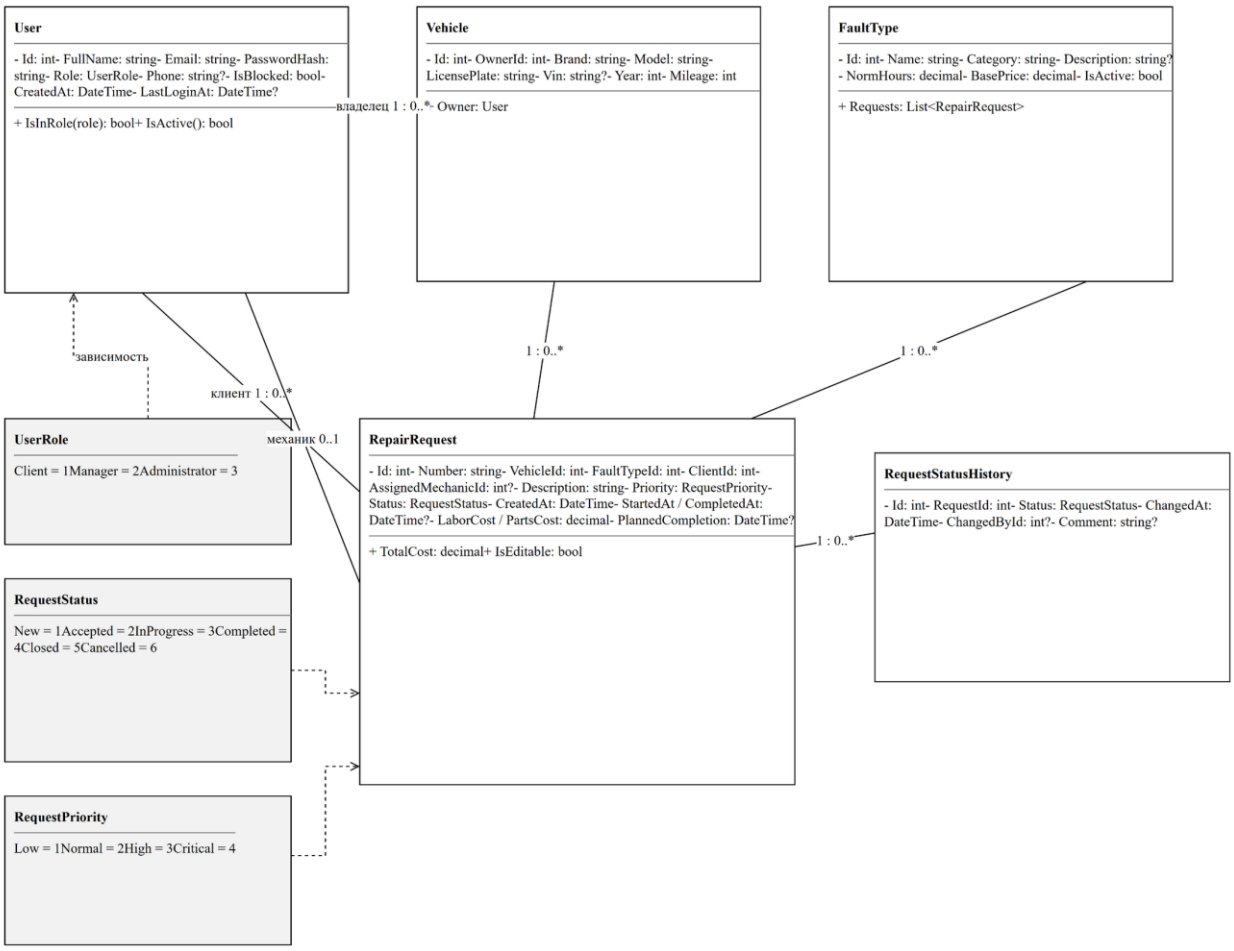


Рисунок Б.3 — UML-диаграмма классов

ER-диаграмма описывает физическую модель базы данных — таблицы, первичные и внешние ключи, уникальные индексы и правила удаления.

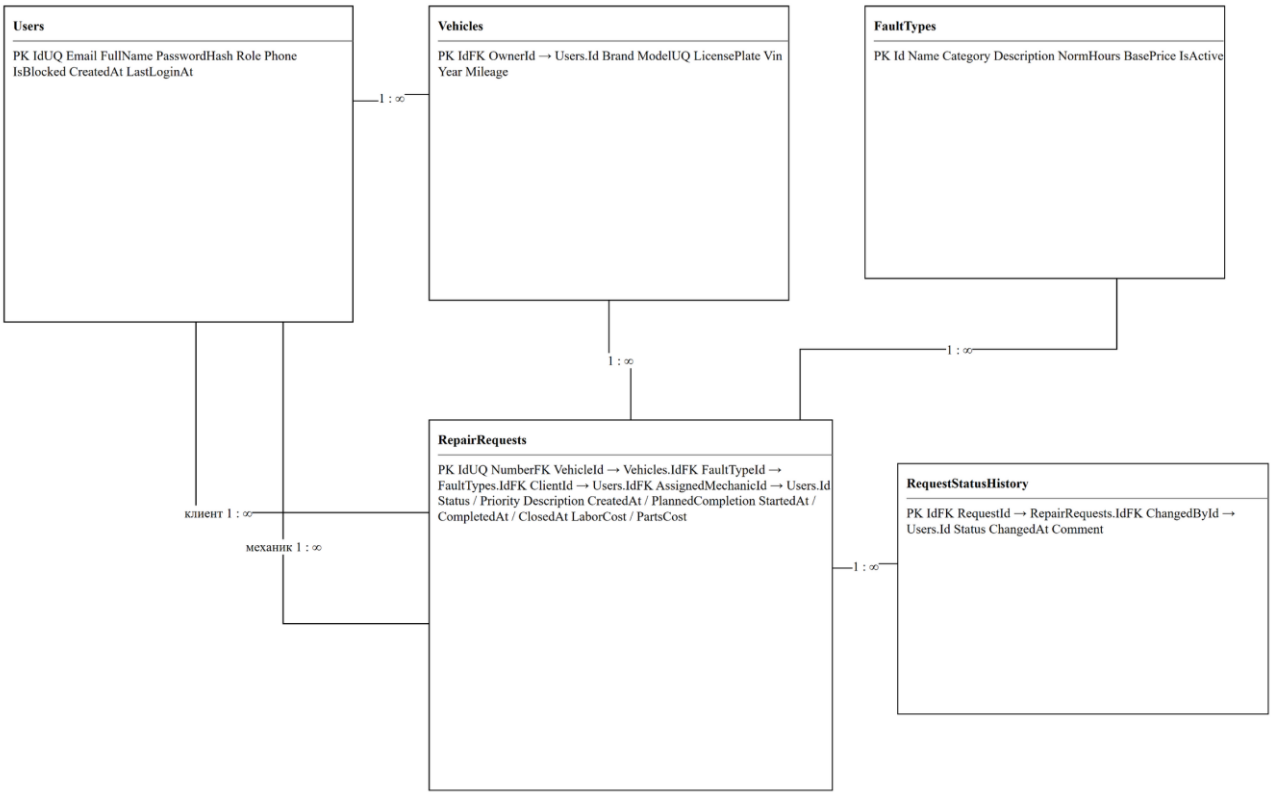


Рисунок Б.4 — ER-диаграмма базы данных

Схема нормализации демонстрирует переход от исходного ненормализованного отношения «Заявка» к четырём взаимосвязанным таблицам, находящимся в третьей нормальной форме.

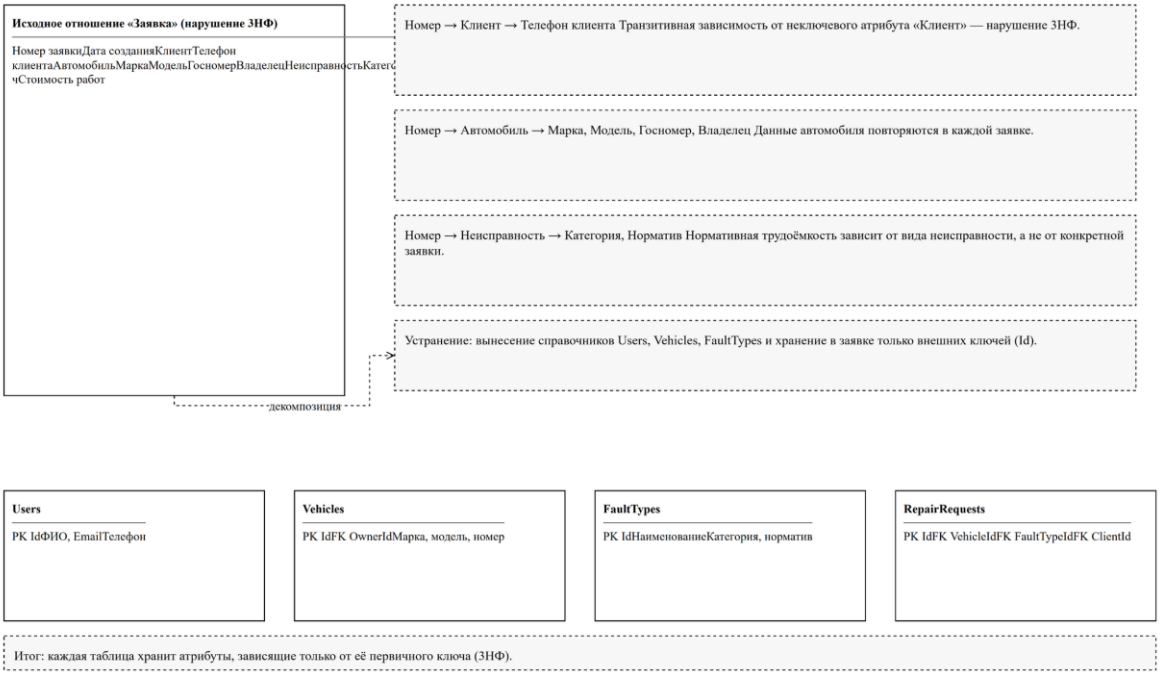


Рисунок Б.5 — Приведение схемы к третьей нормальной форме

Схема архитектуры отражает разделение решения на слои представления, бизнес-логики и доступа к данным, а также направления зависимостей между проектами и внешними библиотеками.

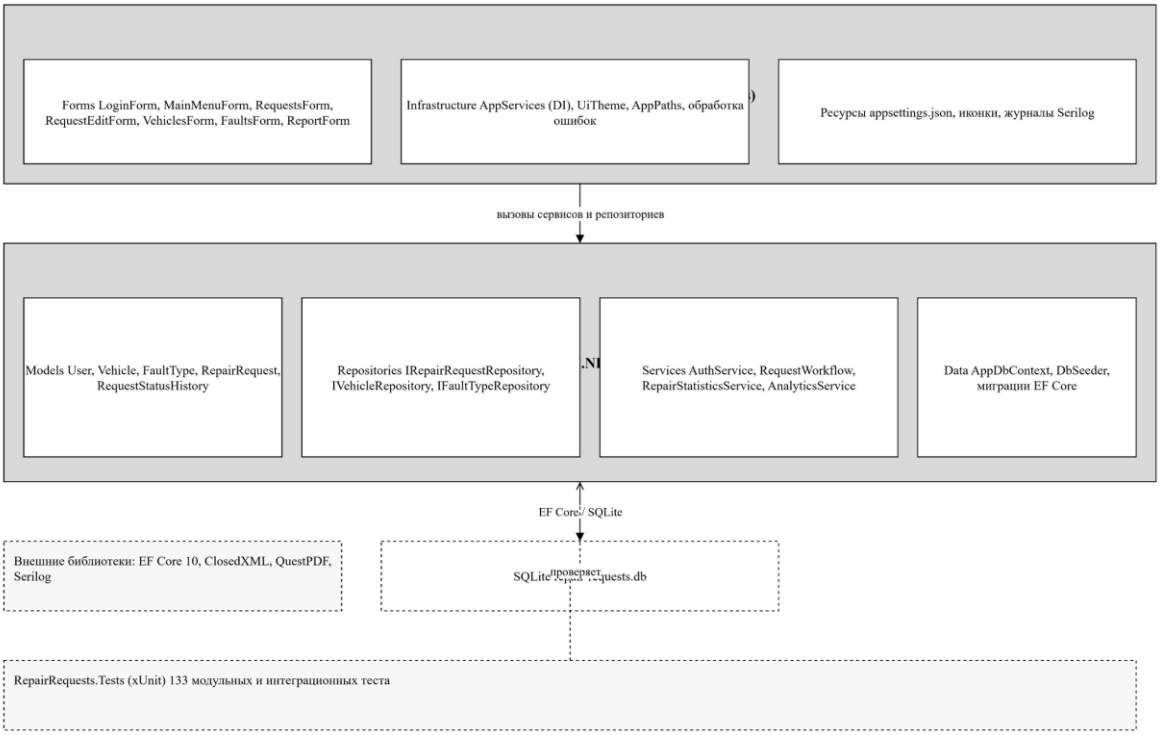


Рисунок Б.6 — Архитектура приложения

ПРИЛОЖЕНИЕ В

Скриншоты интерфейса

В приложении приведены скриншоты основных форм Системы, полученные при работе с демонстрационной базой данных. Скриншоты хранятся в каталоге docs/screens.

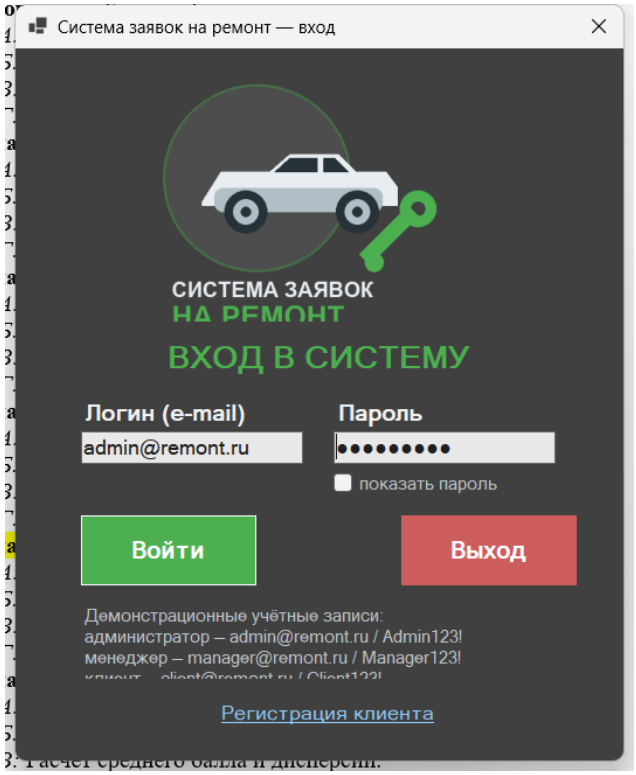


Рисунок В.1 — Форма входа в систему

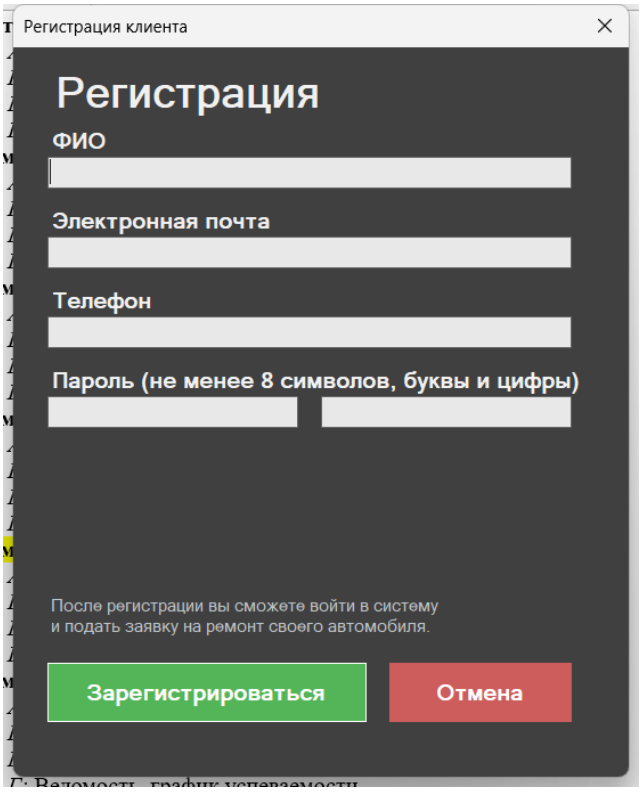


Рисунок В.2 — Форма регистрации клиента

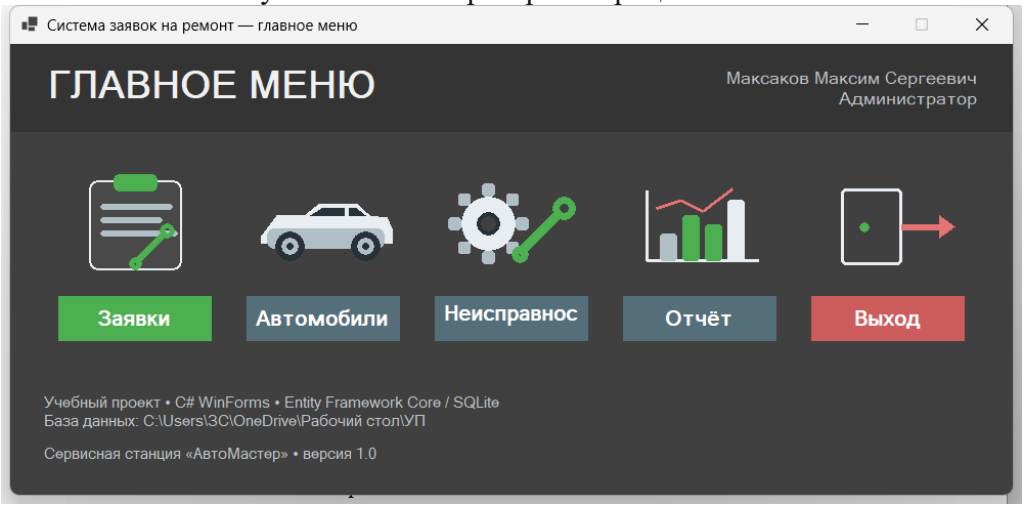


Рисунок В.3 — Главное меню менеджера (администратора)

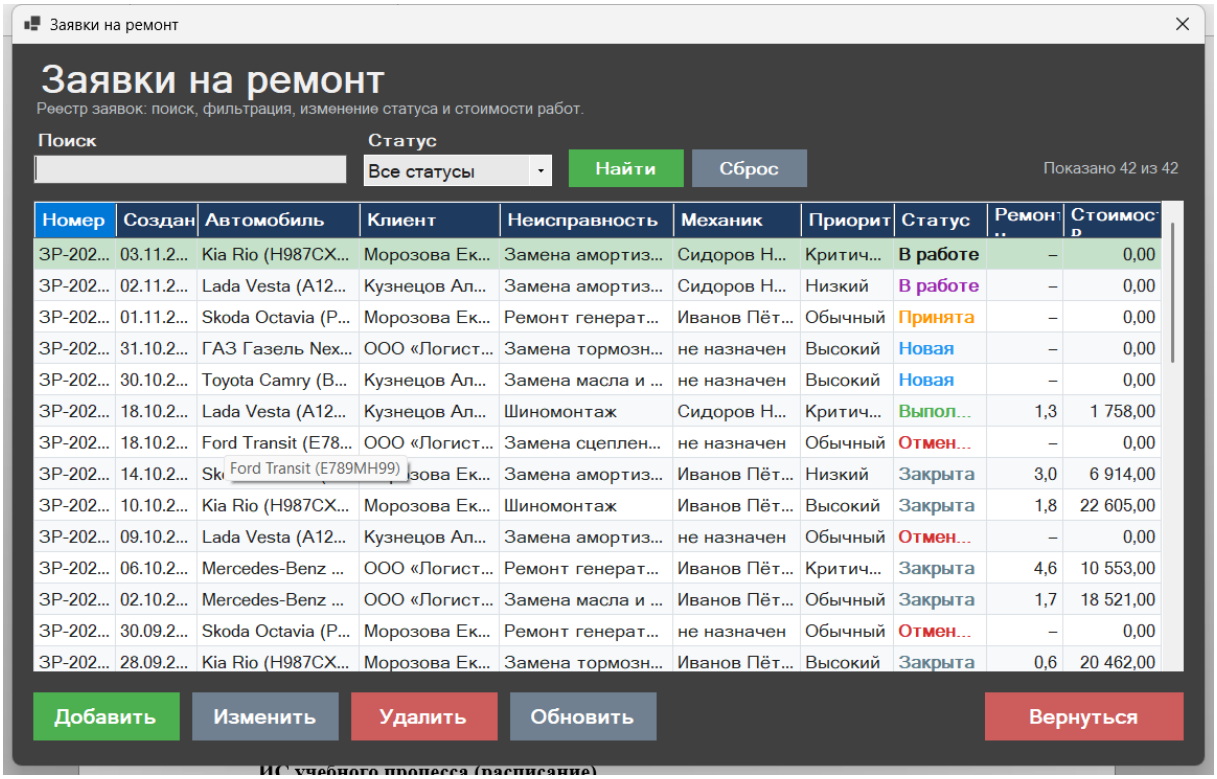


Рисунок В.4 — Список заявок для менеджера

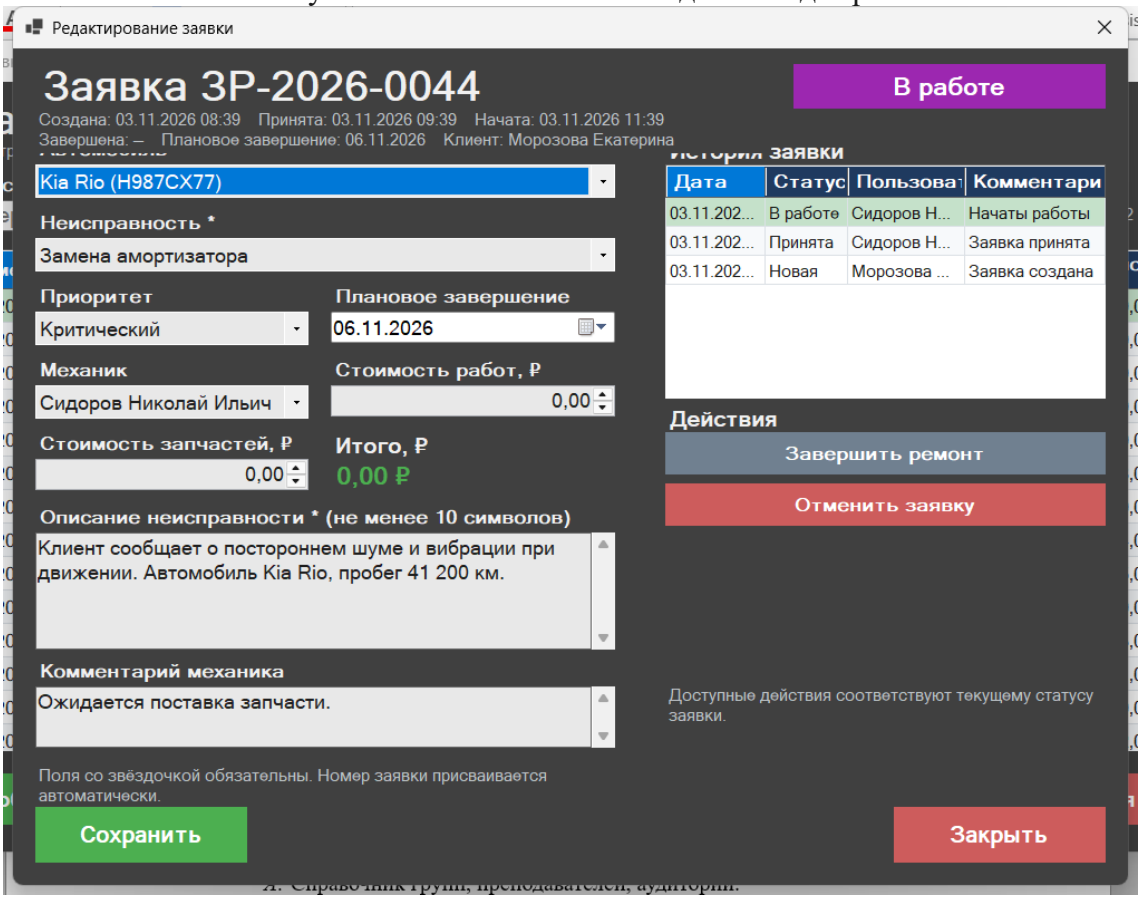


Рисунок В.5 — Редактирование заявки

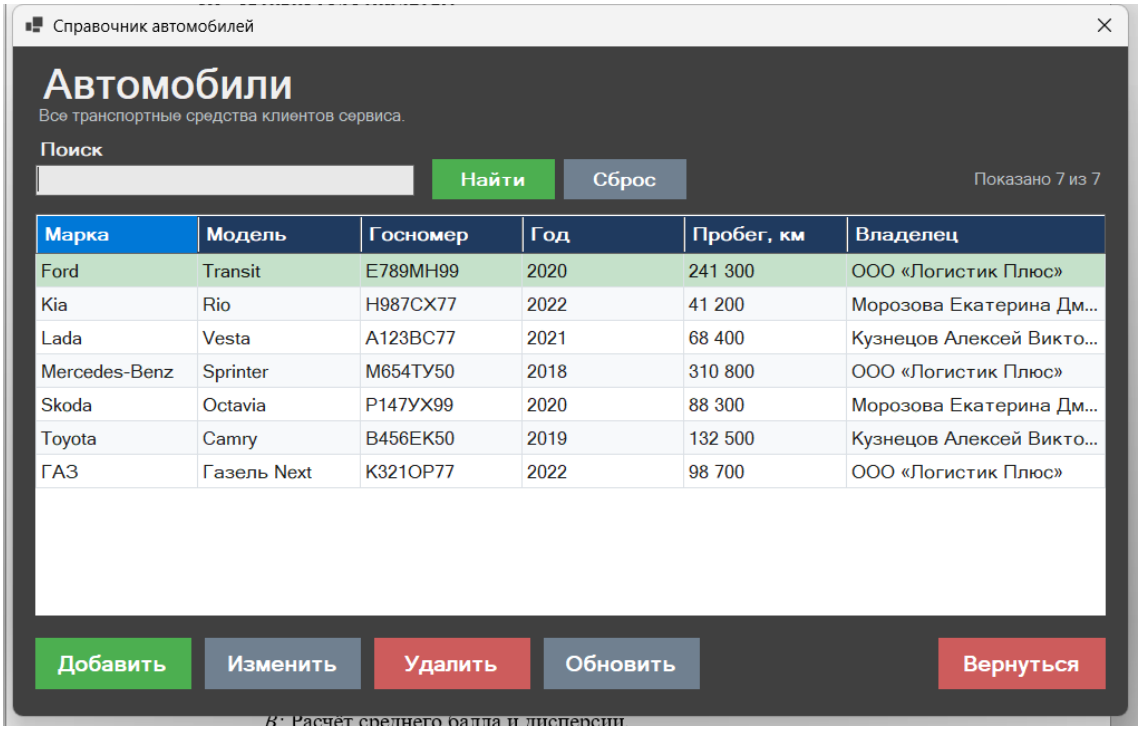


Рисунок В.6 — Справочник транспортных средств

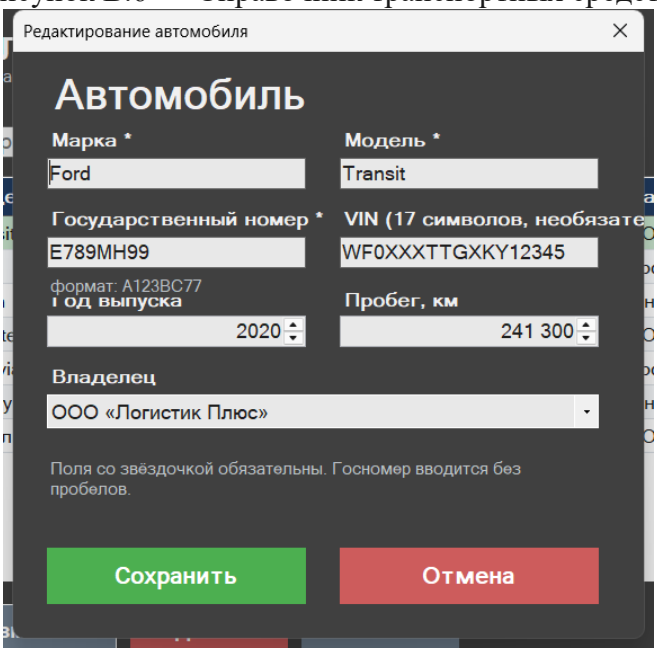


Рисунок В.7 — Редактирование автомобиля

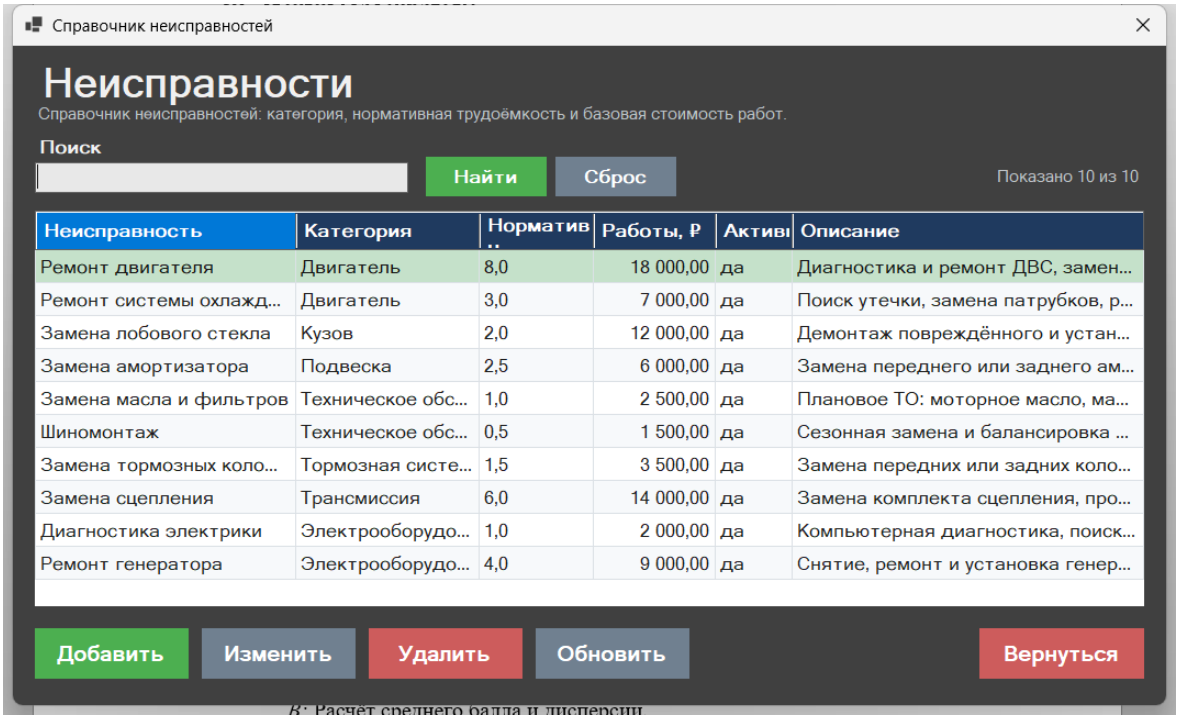


Рисунок В.8 — Справочник неисправностей

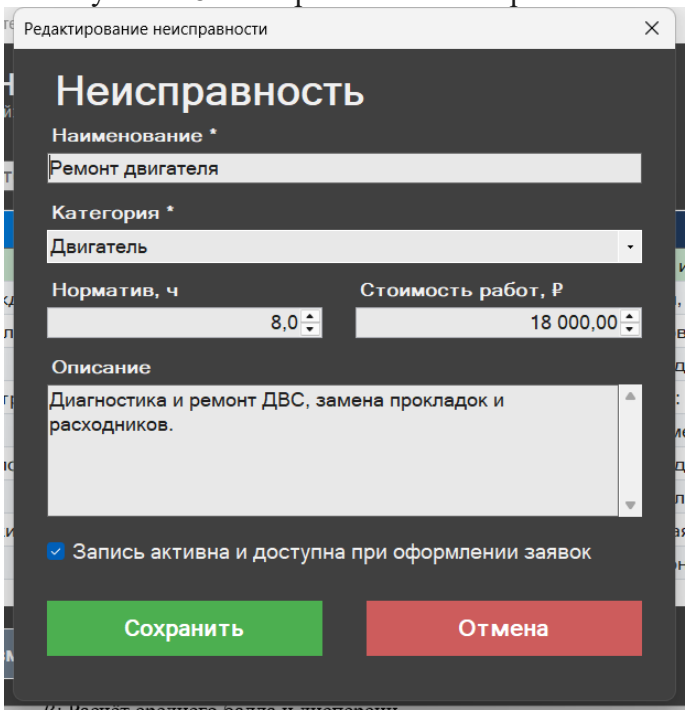


Рисунок В.9 — Редактирование неисправности

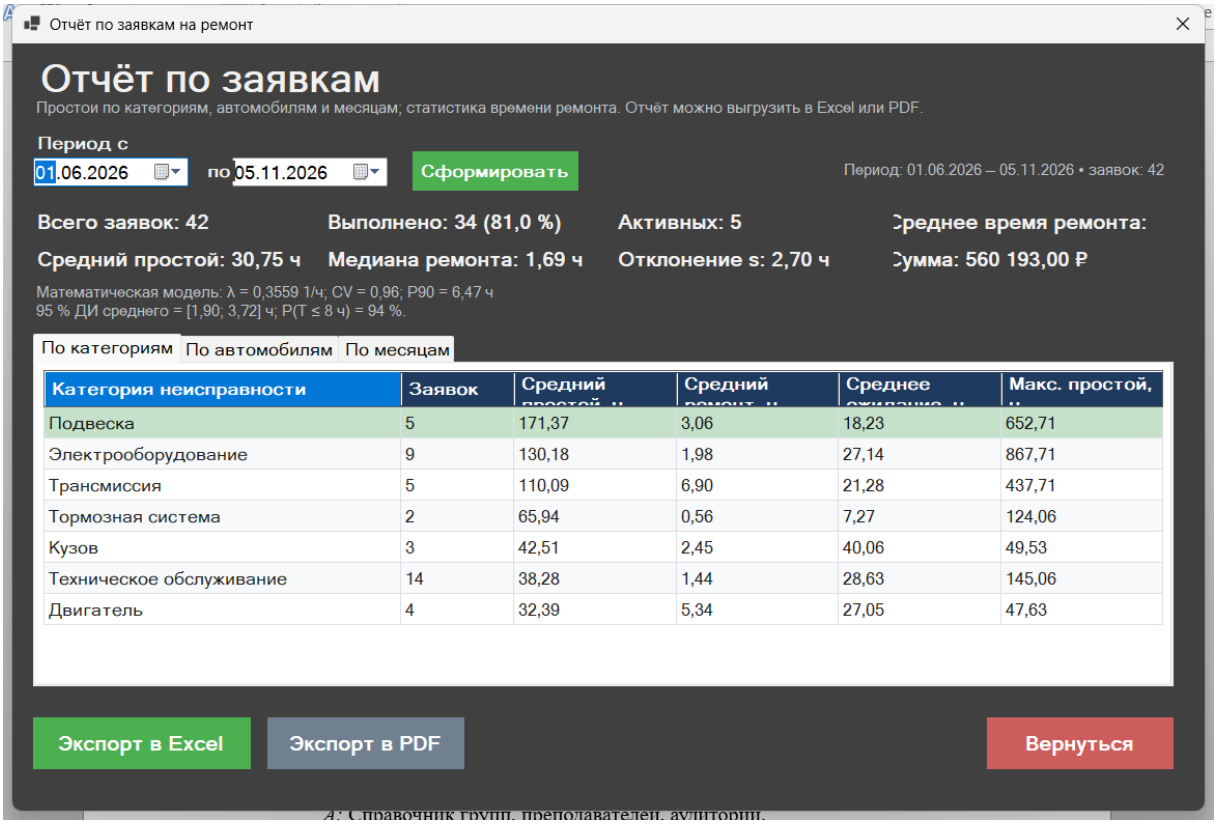


Рисунок В.10 — Отчёт по заявкам для менеджера

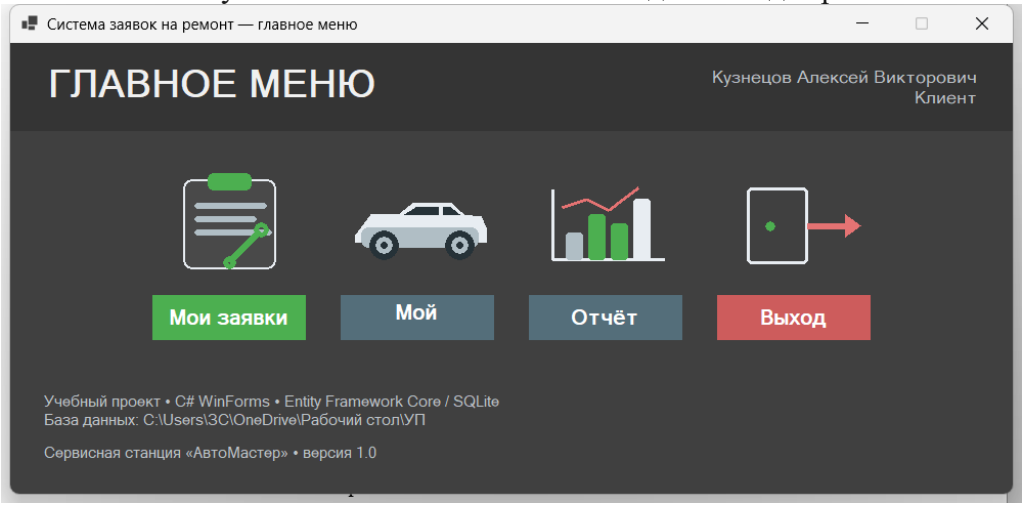


Рисунок В.11 — Главное меню клиента

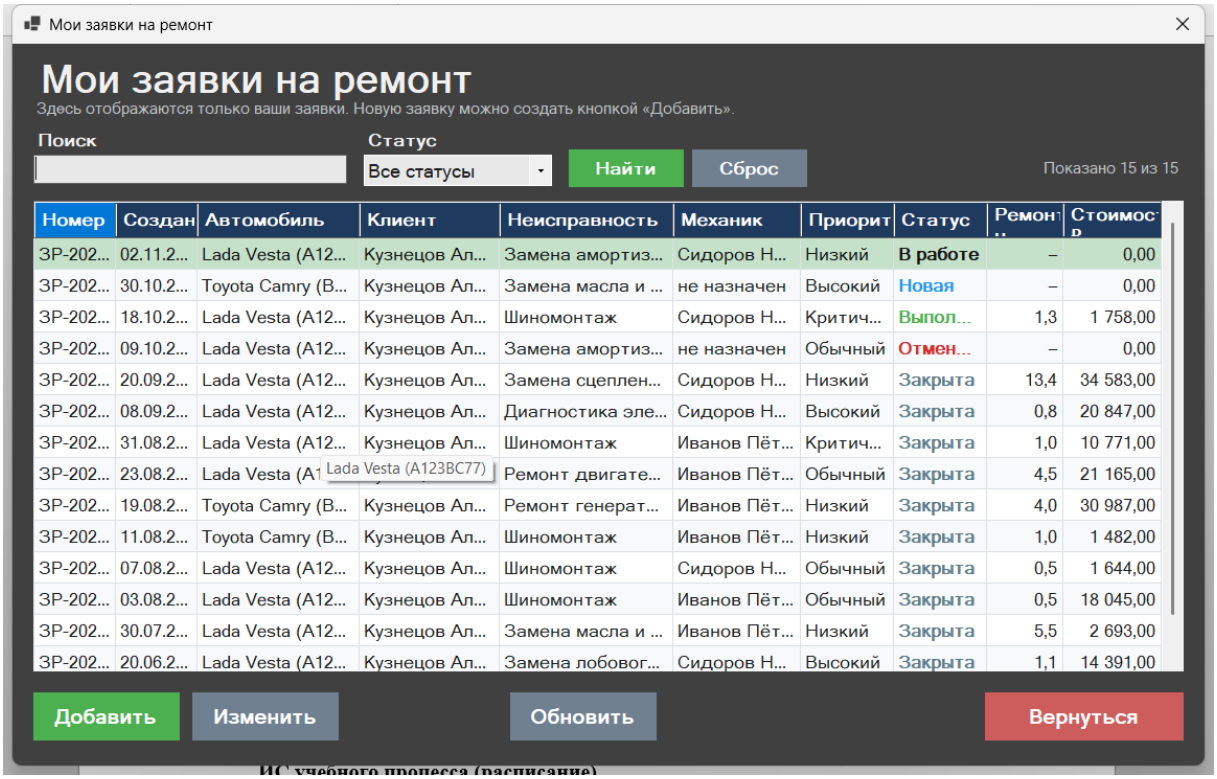


Рисунок В.12 — Список заявок клиента

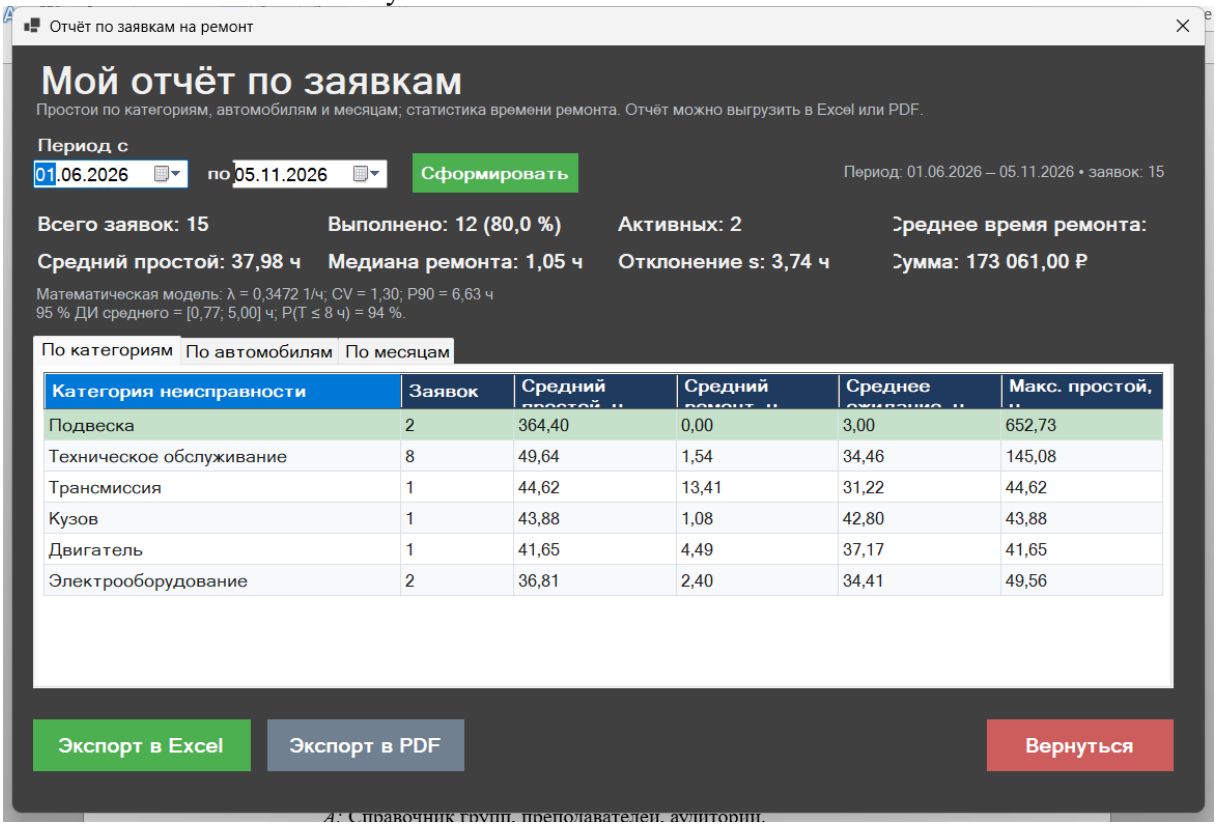


Рисунок В.13 — Отчёт по заявкам клиента

ПРИЛОЖЕНИЕ Г

Тест-план и результаты тестирования

Тест-план составлен по функциональным требованиям приложения А и охватывает авторизацию, разграничение ролей, справочники, заявки, статистику, отчётность и обработку ошибок. Ручные тест-кейсы выполнялись на чистой базе и на базе с демонстрационными данными; автоматические тесты запускались командой dotnet test.

Таблица Г.1 — Ручные тест-кейсы

ID	Сценарий	Шаги	Ожидаемый результат	Статус
TC-01	Вход администратора	Открыть приложение, ввести admin@remont.ru / Admin123!, нажать «Войти»	Открывается главное меню с пятью разделами	Пройден
TC-02	Вход с неверным паролем	Ввести корректную почту и неверный пароль	Сообщение «Неверная электронная почта или пароль», вход не выполнен	Пройден
TC-03	Вход заблокированного пользователя	Заблокировать учётную запись в БД, выполнить вход	Сообщение о блокировке, доступ запрещён	Пройден
TC-04	Регистрация с существующей почтой	Открыть регистрацию, указать client@remont.ru	Сообщение о дубликате, запись не создаётся	Пройден
TC-05	Регистрация с коротким паролем	Ввести пароль из 5 символов	Сообщение о минимальной длине 8 символов	Пройден
TC-06	Клиент открывает «Мои заявки»	Войти как client@remont.ru, открыть раздел	Отображаются только 15 заявок клиента	Пройден
TC-07	Клиент пытается удалить заявку	Открыть заявку клиента	Кнопка «Удалить» скрыта, операция недоступна	Пройден
TC-08	Создание заявки	Нажать «Добавить», выбрать автомобиль и неисправность, указать описание, сохранить	Заявка сохранена, присвоен номер ЗР-2026-NNNN	Пройден
TC-09	Создание заявки с коротким описанием	Ввести описание из 5 символов	Сообщение валидации, запись не сохраняется	Пройден
TC-10	Недопустимый переход статуса	Открыть новую заявку и попытаться перевести в «В работе»	Переход запрещён, доступна только кнопка «Принять»	Пройден
TC-11	Отмена заявки без причины	Нажать «Отменить заявку», оставить комментарий пустым	Сообщение о необходимости указать причину	Пройден
TC-12	Полный цикл заявки	Последовательно выполнить приём, начало работ, завершение и закрытие	Статусы и временные отметки корректны, история содержит 5 записей	Пройден
TC-13	Поиск заявок	Ввести номер или госномер в поле поиска	Отображаются только подходящие заявки	Пройден
TC-14	Фильтр по статусу и датам	Выбрать статус «Закрыта» и период 01.09–30.09	Выборка соответствует обоим условиям	Пройден
TC-15	Сортировка по стоимости	Выбрать сортировку «по стоимости»	Заявки упорядочены по убыванию суммы	Пройден
TC-16	Удаление автомобиля с заявками	Выбрать Lada Vesta и нажать «Удалить»	Сообщение о невозможности удаления, запись сохранена	Пройден

ID	Сценарий	Шаги	Ожидаемый результат	Статус
ТС-17	Удаление неисправности с заявками	Выбрать используемую неисправность и нажать «Удалить»	Сообщение о невозможности удаления	Пройден
ТС-18	Добавление автомобиля с занятым номером	Ввести существующий госномер	Сообщение о дубликате	Пройден
ТС-19	Отчёт за период с данными	Выбрать 01.06.2026–30.11.2026, нажать «Сформировать»	Показатели и таблицы заполнены, ошибок нет	Пройден
ТС-20	Отчёт за пустой период	Выбрать 01.01.2020–31.01.2020	Нулевые показатели, пояснение, приложение не падает	Пройден
ТС-21	Экспорт в Excel	Нажать «Экспорт в Excel», сохранить файл	Файл создан, содержит 4 листа	Пройден
ТС-22	Экспорт в PDF	Нажать «Экспорт в PDF», сохранить файл	PDF создан, открывается, кириллица корректна	Пройден
ТС-23	Отключение сети	Отключить сеть и запросить курс валют	Использован резервный курс, выведено предупреждение	Пройден
ТС-24	Выход из системы	Нажать «Выход» в главном меню	Возврат на форму входа, приложение работает	Пройден
ТС-25	Полное закрытие	Нажать «Выход» на форме входа	Приложение завершается корректно	Пройден

Автоматические тесты распределены по девяти классам и покрывают бизнес-логику, валидацию, хэширование, статистику, аналитику, workflow и работу с базой данных.

Таблица Г.2 — Результаты автоматических тестов

Класс тестов	Количество	Результат
AuthServiceTests	17	Все пройдены
PasswordHasherTests	15	Все пройдены
DataValidatorTests	25	Все пройдены
RepairStatisticsTests	11	Все пройдены
AnalyticsServiceTests	12	Все пройдены
RequestWorkflowTests	20	Все пройдены
CurrencyServiceTests	6	Все пройдены
ExportTests	4	Все пройдены
RepositoriesIntegrationTests	23	Все пройдены
Итого	133	100 %

Команда запуска и фрагмент вывода

```
dotnet test tests/RepairRequests.Tests/RepairRequests.Tests.csproj -v q --nologo

Тестовый запуск для ...\RepairRequests.Tests.dll (.NETCoreApp,Version=v10.0)
Успешно! — не удалось: 0, пройдено: 133, пропущено: 0, всего: 133
Длительность: 16 с
```

ПРИЛОЖЕНИЕ Д

Инструкция по развёртыванию и технике безопасности

Д.1. Требования к рабочему месту

Таблица Д.1 — Требования к рабочему месту

Компонент	Минимальные требования	Рекомендуемые
Операционная система	Windows 10 x64 (21H2)	Windows 11 x64 (23H2 и новее)
Процессор	2 ядра, 2,0 ГГц	4 ядра, 2,5 ГГц и выше
Оперативная память	4 ГБ	8 ГБ и более
Дисковое пространство	200 МБ	1 ГБ с учётом резервных копий
Разрешение экрана	1366×768	1920×1080
Платформа	.NET Desktop Runtime 10	.NET Desktop Runtime 10
Доступ в интернет	Не требуется	Желателен для курсов валют

Д.2. Порядок установки

1. Скачайте и установите .NET Desktop Runtime 10 с официального сайта Microsoft. Проверьте установку командой dotnet --list-runtimes — в списке должна присутствовать строка Microsoft.WindowsDesktop.App 10.x.

2. Скопируйте каталог publish/win-x64 на целевой компьютер, например в C:\Program Files\RepairRequests. Для работы не требуются права администратора, если выбрана папка в профиле пользователя.

3. Проверьте права на запись в каталог программы — приложению нужны подкаталоги data, logs и reports. При необходимости выдайте разрешение через свойства папки или командой icacls.

4. При необходимости отредактируйте appsettings.json, указав другой путь к базе данных в разделе Database:Path.

5. Создайте ярлык на рабочем столе для файла RepairRequests.WinForms.exe.

6. Запустите приложение. При первом запуске автоматически создаются каталоги, применяется миграция InitialCreate и заполняется демонстрационная база.

7. Войдите под учётной записью администратора admin@remont.ru / Admin123! и проверьте доступность всех разделов меню.

8. Смените демонстрационные пароли или заблокируйте неиспользуемые учётные записи перед началом реальной эксплуатации.

Д.3. Настройка резервного копирования

База данных представляет собой один файл data/repair-requests.db. Для резервного копирования достаточно скопировать этот файл при закрытом приложении либо использовать команду SQLite .backup, которая безопасна и при работающем приложении.

```
# Ежедневное копирование планировщиком задач Windows
Copy-Item "C:\Program Files\RepairRequests\data\repair-requests.db" `
    "D:\Backup\repair-requests_$(Get-Date -Format yyyyMMdd).db"
```

Рекомендуется хранить не менее 30 ежедневных копий и одну месячную. Копии следует размещать на другом носителе или сетевом ресурсе. Восстановление выполняется заменой файла базы при закрытом приложении.

Д.4. Обновление версии

1. Закройте приложение и убедитесь, что процесс RepairRequests.WinForms завершён.
2. Создайте резервную копию файла базы данных и каталога appsettings.json.
3. Замените файлы программы новыми, сохранив каталог data.
4. Запустите приложение — миграции применятся автоматически.
5. Проверьте журнал logs/app-ГГГГММДД.log на отсутствие ошибок и выполните дымовые тесты (вход, список заявок, отчёт).

При несовместимых изменениях схемы необходимо предварительно изучить примечания к выпуску и выполнить миграцию вручную командой dotnet tool run dotnet-ef database update.

Д.5. Типичные неисправности и способы устранения

Таблица Д.2 — Типичные неисправности

Симптом	Вероятная причина	Действие
Приложение не запускается, сообщение о среде выполнения	Не установлен .NET Desktop Runtime 10	Установить runtime с сайта Microsoft
«База данных недоступна»	Нет прав на запись или файл занят другим процессом	Проверить права, закрыть лишние экземпляры приложения
«Неверный формат госномера»	Номер не соответствует шаблону А123ВС77	Исправить номер, использовать кириллицу или латиницу
«Не удалось получить курс валют»	Отсутствует сеть или API недоступен	Проверить подключение; приложение использует резервный курс

Симптом	Вероятная причина	Действие
Отчёт пуст	Выбран период без заявок	Изменить период, проверить фильтры
Файл Excel не открывается	Файл повреждён или занят	Повторно выполнить экспорт
Приложение зависло	Длительная операция или сбой	Завершить процесс, изучить журнал, перезапустить

Д.6. Техника безопасности при работе с ПЭВМ

При эксплуатации Системы необходимо соблюдать требования СанПиН 1.2.3685-21 и Типовой инструкции по охране труда при работе с персональным компьютером.

— Перед началом работы осмотрите рабочее место, убедитесь в отсутствии повреждений кабелей, вилок и корпуса компьютера; при обнаружении неисправности не включайте оборудование и сообщите ответственному лицу.

— Непрерывная работа за компьютером не должна превышать двух часов; через каждые 45–60 минут устраивайте перерыв на 10–15 минут, выполняйте гимнастику для глаз и опорно-двигательного аппарата.

— Расстояние от глаз до экрана должно составлять 60–70 см, верхний край монитора — на уровне глаз или чуть ниже. Освещённость рабочего места — 300–500 лк, не допускается блёсткость на экране.

— Не допускается работа на оборудовании с открытыми корпусами, а также самостоятельный ремонт при включённом питании.

— Запрещается размещать на системном блоке жидкости, посторонние предметы, загромождать вентиляционные отверстия.

— При появлении запаха гари, искрения, дыма немедленно отключите оборудование от сети и сообщите руководителю.

— Перед завершением работы корректно закройте приложение и операционную систему, приведите рабочее место в порядок.

Ответственность за соблюдение требований охраны труда возлагается на руководителя подразделения и самих работников. Инструктаж проводится при приёме на работу и повторно не реже одного раза в шесть месяцев с фиксацией в журнале инструктажей.

Д.7. Защита персональных данных

База данных содержит персональные данные клиентов (ФИО, телефон, электронную почту, сведения о транспортных средствах), поэтому при эксплуатации необходимо соблюдать требования Федерального закона № 152-ФЗ «О персональных данных»

— ограничить доступ к каталогу программы средствами NTFS и не передавать файл базы третьим лицам;

— назначить ответственного за обработку персональных данных и получить согласие клиентов на обработку;

— хранить резервные копии в защищённом месте и уничтожать их по истечении срока хранения;

— при увольнении сотрудника блокировать его учётную запись и менять общие пароли;

— не использовать демонстрационные учётные записи с известными паролями в промышленной эксплуатации.

Выполнение перечисленных требований обеспечивает конфиденциальность, целостность и доступность данных, а также снижает риск инцидентов информационной безопасности.

ПРИЛОЖЕНИЕ Е

Руководство пользователя

В приложении приведено руководство пользователя информационной системы учёта заявок на ремонт транспорта. Документ описывает назначение системы, роли пользователей, порядок входа, работу клиента и сотрудника, формирование отчётов и действия в аварийных ситуациях.

Е.1. Общие сведения о системе

Е.1.1. Назначение системы

Информационная система «Заявки на ремонт» предназначена для учёта обращений клиентов станции технического обслуживания, ведения справочников транспортных средств и неисправностей, контроля статусов заявок и формирования отчётности. Система позволяет отказаться от бумажного журнала приёмки и получать статистику времени ремонта по накопленным данным.

Основные возможности

— регистрация клиентов и авторизация пользователей;

— ведение справочника автомобилей клиента;

— справочник неисправностей с нормативами трудоёмкости и стоимостью работ;

— оформление заявок и назначение механика;

— смена статусов заявки с сохранением истории;

— расчёт среднего времени ремонта и показателей разброса;

— отчёт за период с распределением по категориям, автомобилям и месяцам;

— выгрузка отчёта в Excel и PDF.

Е.1.2. Роли пользователей

В системе предусмотрены три роли. Права ролей различаются составом доступных разделов и объёмом видимых данных.

Таблица Е.1 — Роли пользователей

Роль	Доступные разделы	Объём данных
Клиент	Мои заявки, мой транспорт, отчёт	Только свои автомобили и заявки
Менеджер	Заявки, автомобили, неисправности, отчёт	Все данные станции
Администратор	Те же разделы, что у менеджера	Все данные станции

Е.1.3. Требования к рабочему месту

Для работы системы требуется компьютер под управлением Windows 10 или 11 (64-разрядная версия), среда .NET Desktop Runtime 10, не менее 4 ГБ оперативной памяти и 200 МБ свободного места на диске. Рекомендуемое разрешение экрана — не ниже 1366×768. Постоянное подключение к сети не требуется; внешний сервис курсов валют используется только как дополнительная возможность.

Е.2. Начало работы

Е.2.1. Запуск и вход в систему

Запуск выполняется файлом RepairRequests.WinForms.exe из папки приложения. При первом запуске автоматически создаётся база данных и загружается демонстрационный набор записей. Открывается форма входа.

Введите электронную почту и пароль, затем нажмите кнопку «Войти». Поле пароля скрывает символы; при необходимости установите флажок «показать пароль». Вход также выполняется нажатием клавиши Enter. При неверных данных выводится сообщение «Неверная электронная почта или пароль».

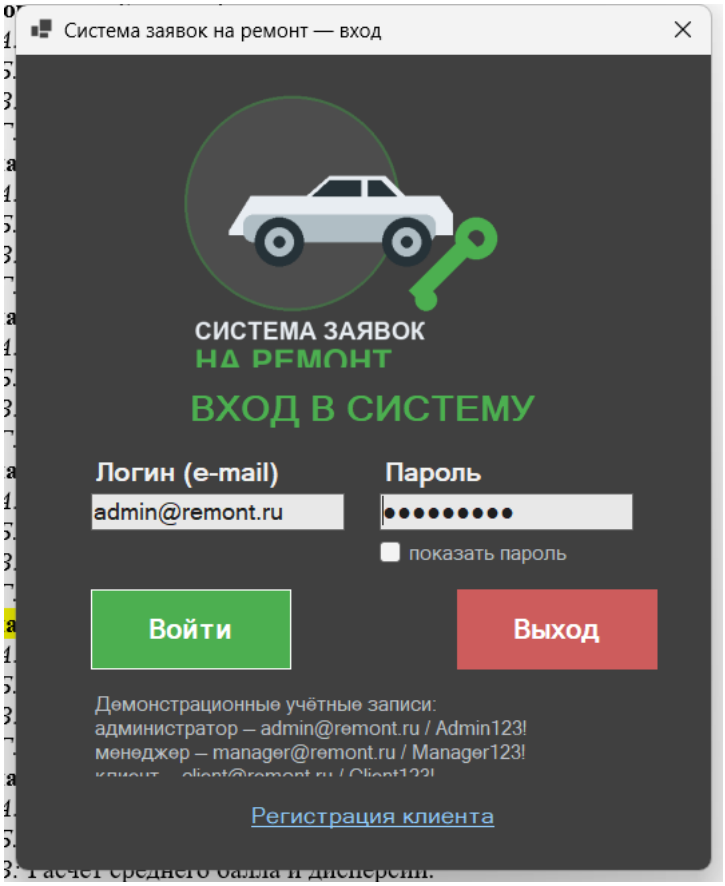


Рисунок Е.1 — Форма входа в систему

Е.2.2. Регистрация клиента

Новый пользователь может зарегистрироваться по ссылке «Регистрация клиента». Необходимо указать ФИО, адрес электронной почты, телефон и пароль. Пароль должен содержать не менее восьми символов, включать буквы и цифры. Пароль вводится дважды для исключения опечатки.

После успешной регистрации пользователю присваивается роль клиента, и он может войти в систему. Если почта уже используется, выводится сообщение о необходимости указать другой адрес.

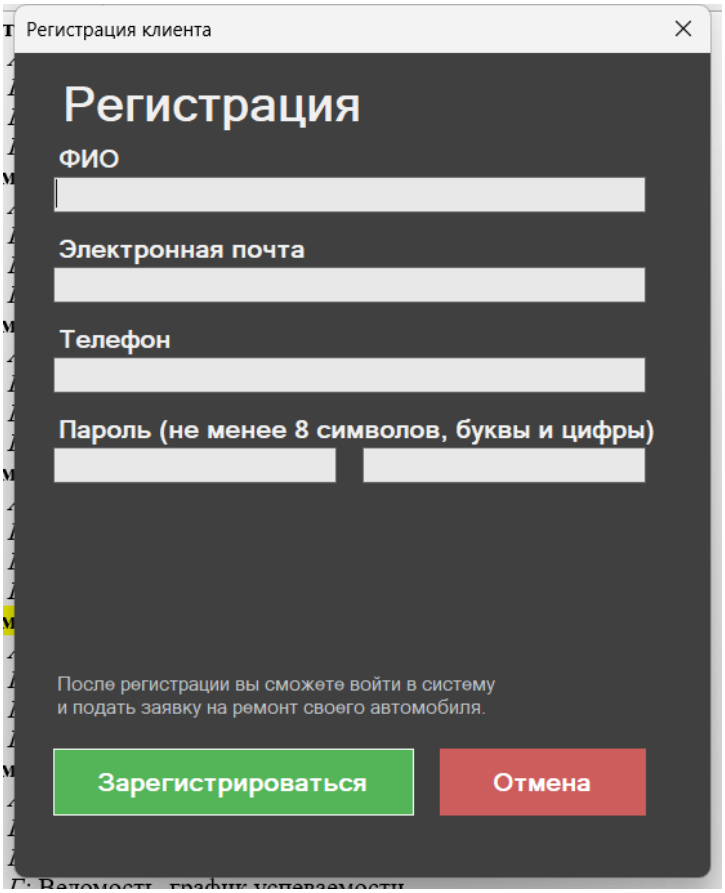


Рисунок Е.2 — Регистрация клиента

Е.2.3. Главное меню

После входа открывается главное меню с крупными кнопками разделов. Состав кнопок зависит от роли — клиент видит «Мои заявки», «Мой транспорт», «Отчёт» и «Выход», сотрудник — «Заявки», «Автомобили», «Неисправности», «Отчёт» и «Выход». В верхней части окна отображаются ФИО и роль пользователя, в нижней — путь к файлу базы данных.

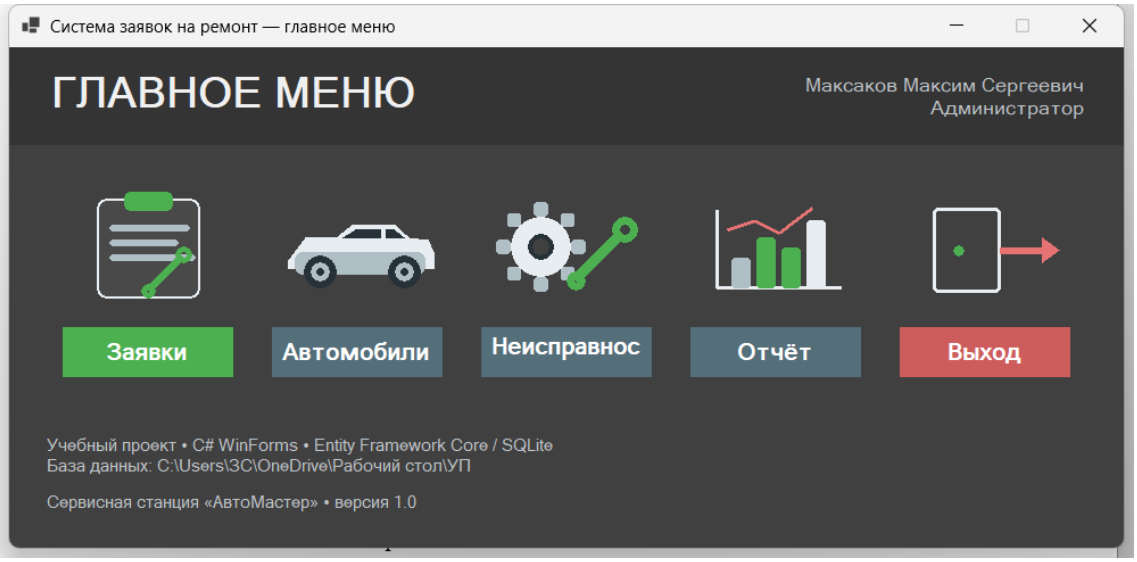


Рисунок Е.3 — Главное меню

Е.3. Работа клиента

Е.3.1. Добавление автомобиля

Раздел «Мой транспорт» содержит список автомобилей клиента. Для добавления нажмите кнопку «Добавить» и заполните поля — марка, модель, государственный номер, VIN, год выпуска и пробег. Государственный номер вводится в формате А123ВС77, VIN должен содержать 17 символов.

После нажатия кнопки «Сохранить» запись появляется в списке. Если поле заполнено неверно, форма остаётся открытой и показывает сообщение об ошибке. Изменение записи выполняется кнопкой «Изменить», удаление — кнопкой «Удалить» с подтверждением.

Е.3.2. Оформление заявки

В разделе «Мои заявки» нажмите «Добавить». В форме заявки выберите автомобиль и неисправность, опишите проблему, при необходимости укажите приоритет, плановую дату завершения и стоимость работ. Стоимость работ подставляется по справочнику и может быть изменена.

Номер заявки присваивается автоматически в формате ЗР-2026-0001. После сохранения заявка получает статус «Новая» и появляется в списке.

Клиент может отслеживать её состояние, но не может изменить статус или назначить механика.

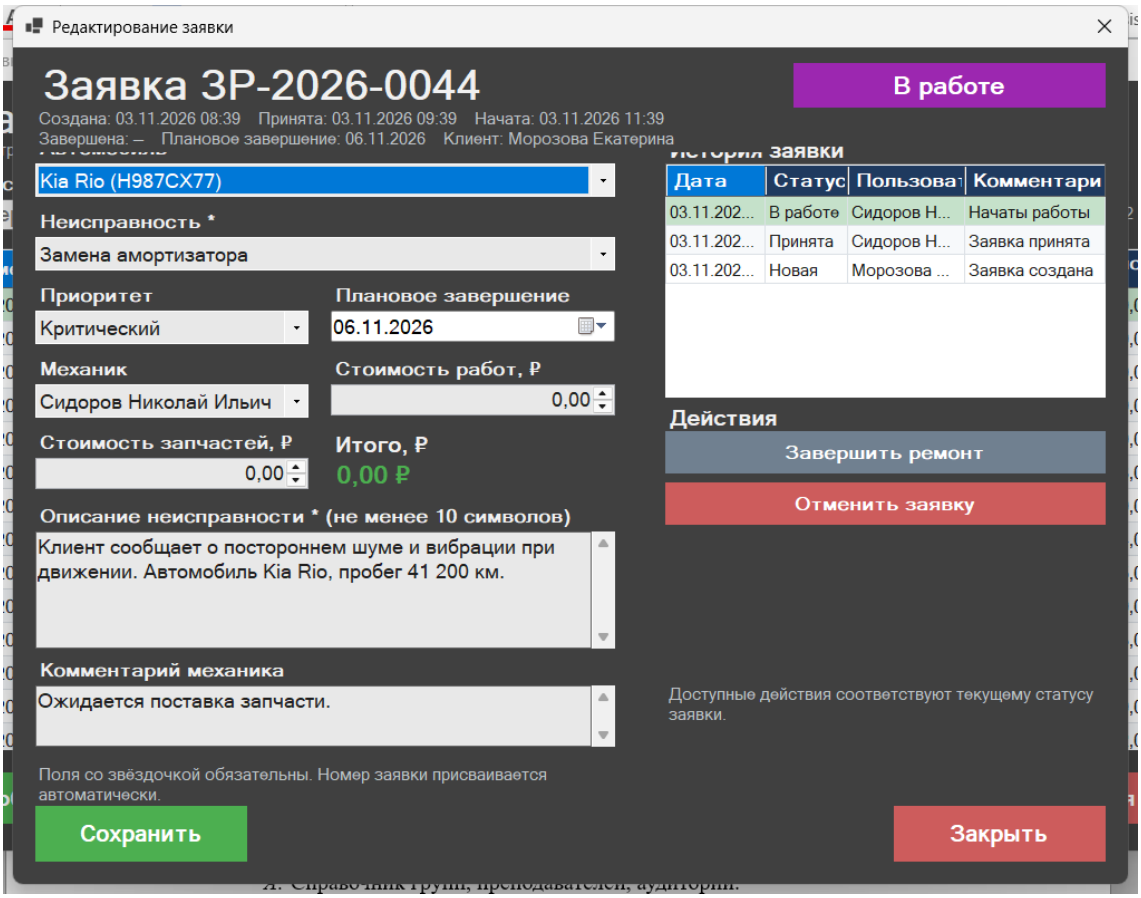


Рисунок Е.4 — Форма заявки на ремонт

Е.3.3. Просмотр состояния заявок

Список заявок содержит номер, дату создания, автомобиль, неисправность, приоритет, статус, время ремонта и стоимость. Строку можно отфильтровать по статусу и найти по номеру, автомобилю или неисправности. Двойной щелчок открывает заявку для просмотра.

Статусы «Новая», «Принята», «В работе», «Выполнена», «Закрыта» и «Отменена» отображаются разными цветами. История изменений доступна в нижней части формы заявки.

Е.3.4. Отчёт по своим заявкам

Раздел «Отчёт» позволяет выбрать период и сформировать отчёт только по своим заявкам. В отчёте отображаются количество заявок, среднее

время ремонта, медиана, суммарная стоимость и распределение по категориям и автомобилям.

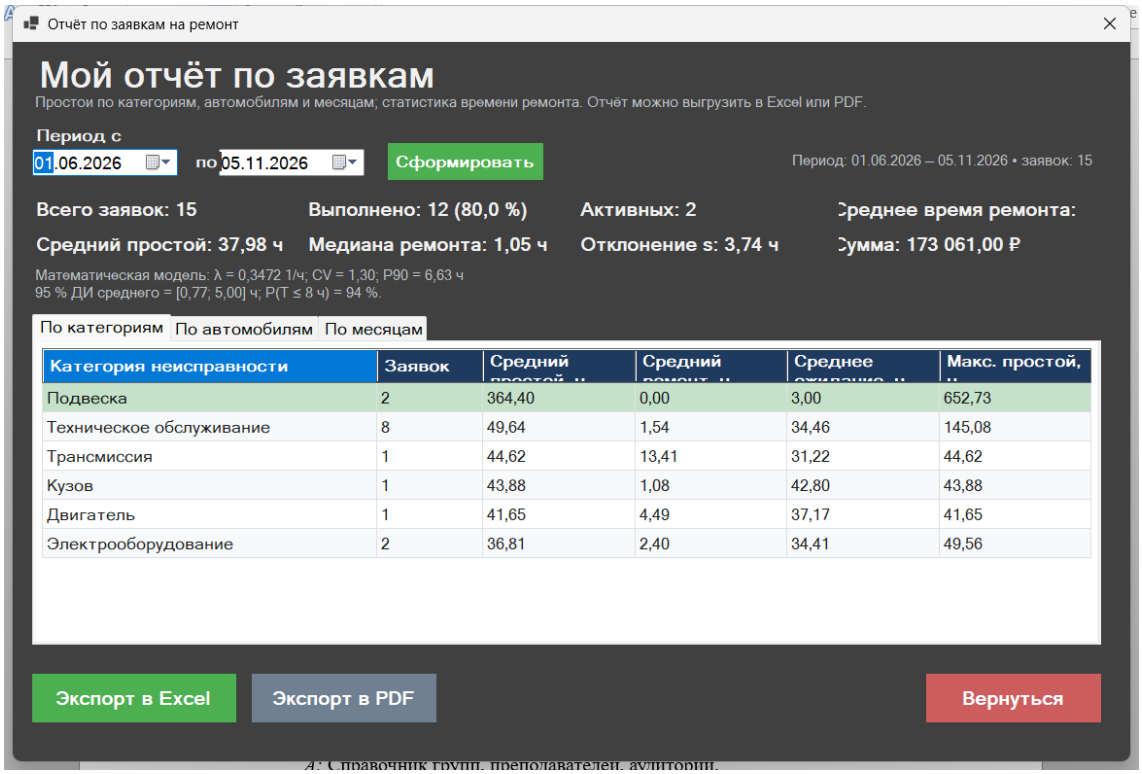


Рисунок Е.5 — Отчёт клиента

Е.4. Работа сотрудника

Е.4.1. Список заявок и поиск

Сотрудник видит все заявки станции. Список можно отфильтровать по статусу и найти по номеру, автомобилю, клиенту или неисправности. Кнопки «Добавить», «Изменить» и «Удалить» расположены над таблицей. Количество показанных и общее число записей отображается в нижней части окна.

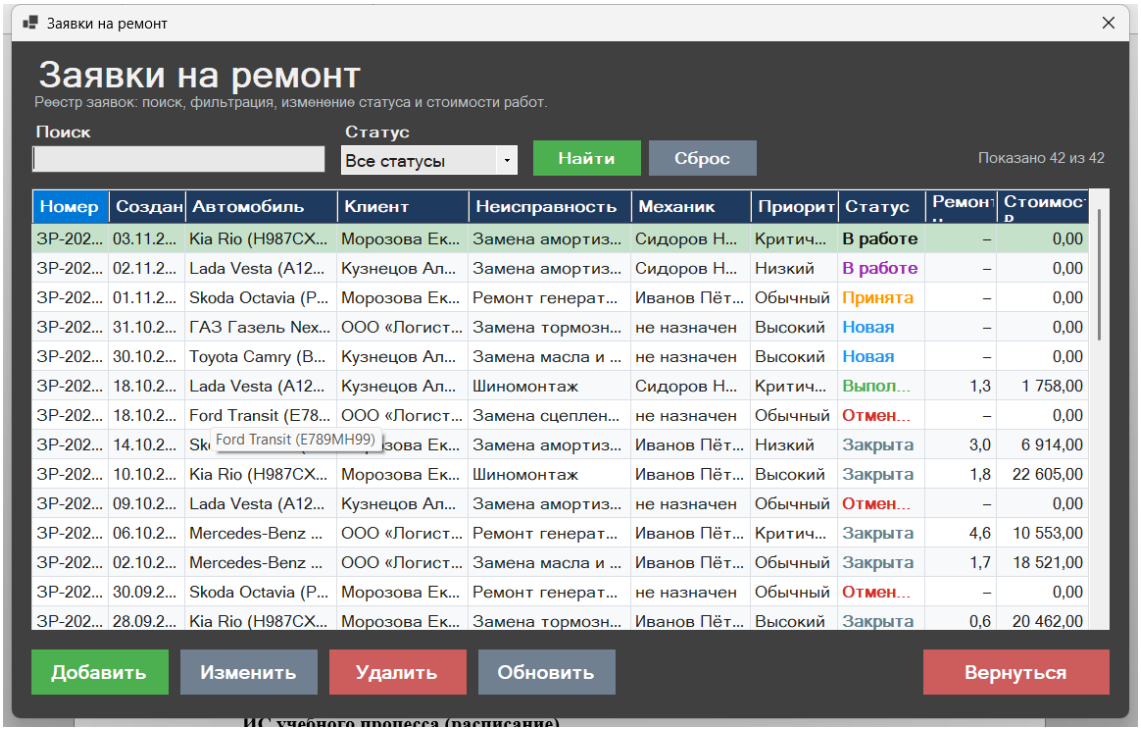


Рисунок Е.6 — Список заявок

Е.4.2. Обработка заявки

Откройте заявку двойным щелчком. В форме доступны выбор механика, изменение приоритета и плановой даты, стоимости работ и запчастей, комментарий. Смена статуса выполняется кнопками действий; система разрешает только допустимые переходы, например «Новая» → «Принята» → «В работе» → «Выполнена» → «Закрыта».

При смене статуса запрашивается подтверждение. В историю автоматически записываются дата, новый статус, автор изменения и комментарий. Если заявка закрыта или отменена, дальнейшее изменение статуса недоступно.

Е.4.3. Назначение механика

Механик выбирается из списка пользователей с ролью менеджера. Назначение фиксируется в заявке и отображается в списке. При необходимости исполнителя можно изменить до завершения работ.

Е.4.4. Справочники

Раздел «Автомобили» позволяет просматривать и редактировать автомобили всех клиентов. Раздел «Неисправности» содержит наименование, категорию, описание, норматив трудоёмкости и базовую стоимость работ. Неисправность, которая больше не используется, можно сделать неактивной — она сохранится в истории, но не будет предлагаться при оформлении новых заявок.

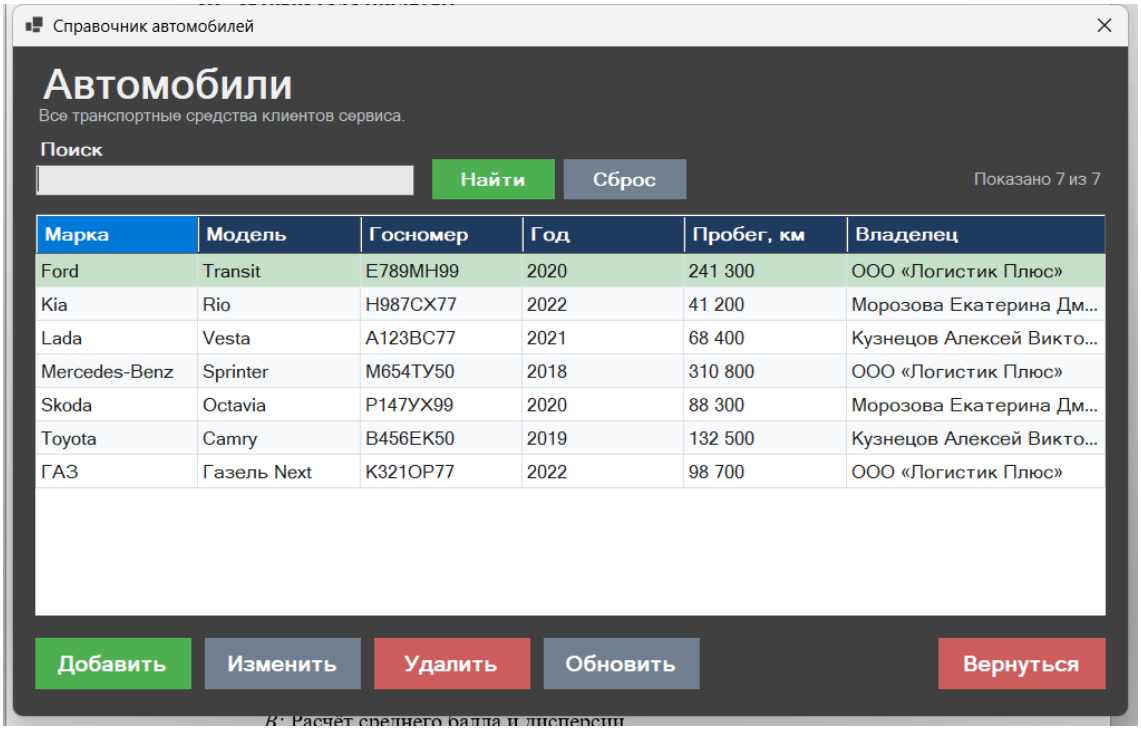


Рисунок Е.7 — Справочник автомобилей

Е.5. Отчёты и экспорт

Е.5.1. Формирование отчёта за период

В разделе «Отчёт» укажите начало и окончание периода и нажмите «Сформировать». Начало периода не может быть позже окончания. Отчёт строится по заявкам, созданным в выбранном периоде; для сотрудника — по всем заявкам, для клиента — только по своим.

В верхней части отображаются сводные показатели — всего заявок, выполнено, активных, среднее время ремонта, медиана, отклонение, суммарная стоимость и средний простой. Ниже расположены вкладки «По категориям», «По автомобилям» и «По месяцам».

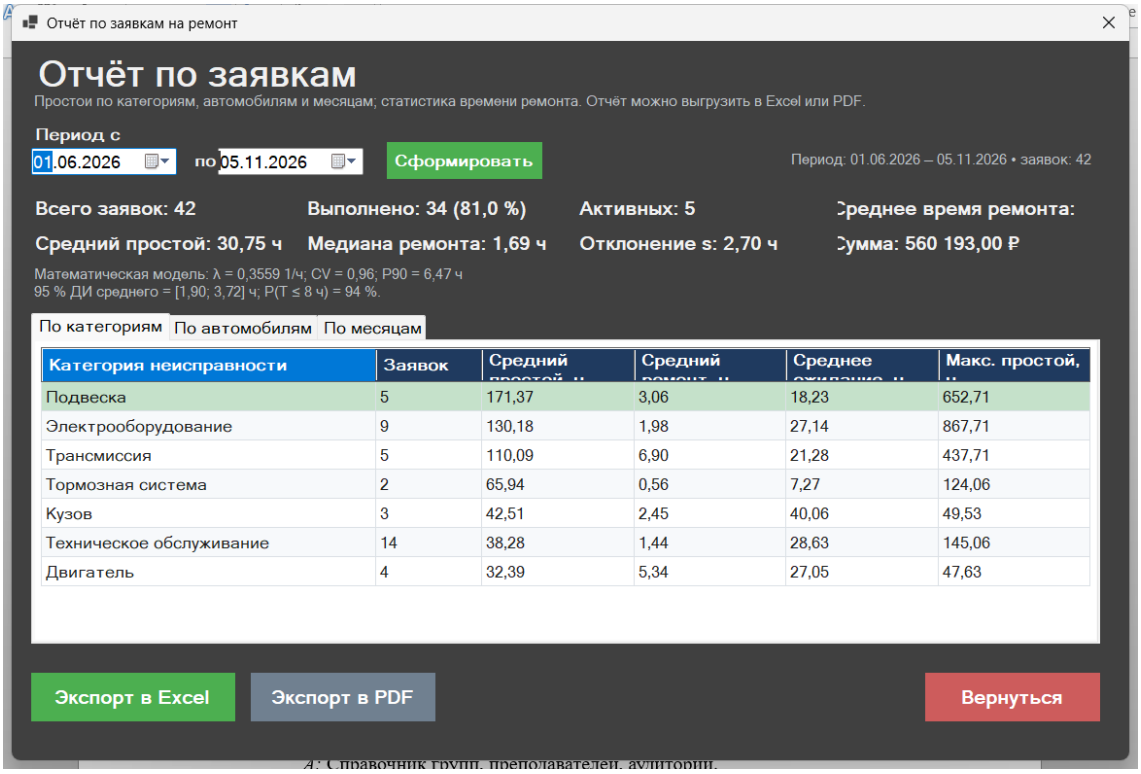


Рисунок Е.8 — Отчёт за период

Е.5.2. Показатели статистики

Для оценки сроков ремонта рассчитываются среднее значение, медиана, мода, дисперсия, стандартное отклонение и коэффициент вариации. Дополнительно строится экспоненциальная модель — интенсивность $\lambda = 1 / \bar{x}$ и вероятность завершить ремонт за заданное время по формуле $F(t) = 1 - e^{(-\lambda t)}$.

Если за период нет завершённых заявок, статистика не рассчитывается — выводятся нулевые значения и сообщение о недостатке данных. Это нормальная ситуация, а не ошибка.

Е.5.3. Экспорт в Excel

Кнопка «Excel» сохраняет книгу с четырьмя листами — «Сводка», «Заявки», «Простои» и «Статистика». Имя файла содержит дату и время формирования. Книга открывается в Microsoft Excel или совместимом редакторе.

Числовые значения выводятся с двумя знаками после запятой, даты — в формате ДД.ММ.ГГГГ. Ширина столбцов подобрана автоматически, заголовки имеют границы.

Е.5.4. Экспорт в PDF

Кнопка «PDF» формирует готовый к печати документ формата А4. Он содержит сводные показатели, распределение заявок, описание математической модели и перечень заявок. В колонтитулах указаны название системы, дата формирования и номер страницы.

Е.6. Аварийные ситуации

В таблице приведены типичные ситуации и порядок действий пользователя.

Таблица Е.2 — Аварийные ситуации

Ситуация	Причина	Действия
Не удаётся войти	Неверная почта или пароль	Проверить раскладку и регистр, повторить ввод
Учётная запись заблокирована	Решение администратора	Обратиться к администратору
Не создаётся база данных	Нет прав на запись в каталог	Запустить из папки с правами записи
Не удаляется автомобиль	Есть связанные заявки	Закрыть или удалить заявки
Отчёт пустой	Нет заявок за период	Выбрать другой период
Недоступен курс валют	Нет подключения к сети	Используется резервный курс
Файл Excel не открывается	Повреждён или занят	Повторить выгрузку, закрыть файл

Е.7. Рекомендации по работе

— регулярно копируйте файл базы данных при закрытом приложении;

— не изменяйте файлы в каталоге data вручную;

— используйте сложные пароли и не передавайте их другим сотрудникам;

— перед обновлением версии сохраняйте копию базы данных;

— закрывайте приложение перед выключением компьютера;

— при повторяющейся ошибке сохраните файл журнала из каталога logs.

Соблюдение этих рекомендаций снижает риск потери данных и упрощает диагностику неисправностей.